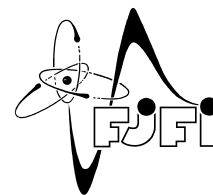




ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE
Fakulta jaderná a fyzikálně inženýrská



**Název práce česky název práce česky název práce
česky název práce česky**

**Title of the Work in English Title of the Work in
English Title of the Work in English**

Bakalářská práce

Autor: **Jméno Autora**
Vedoucí práce: **prof. Ing. Jméno Školitele, DrSc.**
Konzultant: **doc. RNDr. Jméno Konzultanta, CSc.** (pouze pokud konzultant byl jmenován.)
Akademický rok: 2024/2025

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: Jméno: Osobní číslo:
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávací katedra/ústav:
Studijní program:
Specializace:

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Šablona

Název bakalářské práce anglicky:

Template

Pokyny pro vypracování:

1. první bod zadání
2. druhý bod zadání
3. atd.

**Místo tohoto souboru použijte
soubor PDF se všemi podpisy,
který lze stáhnout z KOS, resp.
jste jej obdrželi e-mailem.**

Seznam doporučené literatury:

- [1] reference č. 1
- [2] reference č. 2
- [3] atd.

Jméno a pracoviště vedoucí(ho) bakalářské práce:

prof. Ing. Jméno Školitele, DrSc.

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

doc. RNDr. Jméno Konzultanta, CSc.

Datum zadání bakalářské práce: **31.10.2023**

Termín odevzdání bakalářské práce: **05.08.2024**

Platnost zadání bakalářské práce: **30.09.2025**

(jméno vedoucího - bude doplněno)
podpis vedoucí(ho) práce

(jméno vedoucího - bude doplněno)
podpis vedoucí(ho) ústavu/katedry

doc. Ing. Václav Čuba, Ph.D.
podpis děkana

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Podpis studenta

Místo tohoto souboru použijte
soubor PDF se všemi podpisy,
který lze stáhnout z KOS, resp.
jste jej obdrželi e-mailem.

PROHLÁŠENÍ

Já, níže podepsaný

Příjmení, jméno studenta: John Doe
Osobní číslo: 123456
Název programu: Matematické inženýrství

prohlašuji, že jsem bakalářskou práci s názvem

... název práce ...

vypracoval samostatně a uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

V Praze dne

John Doe

.....
podpis studenta

**Místo tohoto souboru použijte prohlášení
vygenerované a podepsané v systému KOS.**

Poděkování:

Chtěl bych zde poděkovat především svému školiteli za pečlivost, ochotu, vstřícnost a odborné i lidské zázemí při vedení mé diplomové práce. Dále děkuji svému konzultantovi za

Název práce

Studijní program: Celý název studijního programu (nikoliv zkratka)

Druh práce: Bakalářská práce

Vedoucí práce: prof. Ing. Jméno Školitele, DrSc., pracoviště školitele (název instituce, fakulty, katedry...)

Konzultant: doc. RNDr. Jméno Konzultanta, CSc., pracoviště konzultanta. Pouze pokud konzultant byl jmenován.

[illegible]

Klíčová slova: klíčová slova (nebo výrazy) seřazená podle abecedy a oddělená čárkou

Title:

Title of the Work

Author: Author's Name

[illegible]

Key words: keywords in alphabetical order separated by commas

Obsah

Úvod	13
1 Problematika výběru modelů	14
1.1 Obecný princip hledání modelu	14
1.2 Bayesův přístup k výběru modelu	15
1.3 GP regrese	16
1.4 Výběr modelu pro GP regresi	18
1.4.1 Variační inference	18
1.4.2 Type II – MLE	18
1.4.3 Markov Chain Monte Carlo	19
2 Prior–Data Fitted Networks	21
2.1 Úvod do Prior–Data Fitted Networks	21
2.2 Praktická motivace k PFN	23
2.3 Vnitřní architektura PFN	23
2.3.1 Fáze 1 - Meta–Learning	24
2.3.2 Fáze 2 - Vstup do modelu	25
2.3.3 Fáze 3 – Transformer vrstvy	27
2.3.4 Fáze 4 – Dekóder, převod embeddingu na logity.	29
2.4 Teoretické statistické vlastnosti PFN	32
2.4.1 Statistický model	32
2.4.2 Od PPD k PFN	34
2.4.3 Učení PFN In–Context	35
2.4.4 PFN transformer	37
3 Experimentální průzkum vlastností PFN	40
3.1 Vnitřní struktura PFN transformera a jeho základní principy	40
3.1.1 Struktura attention matic	41
3.1.2 Matematický rámec pro srovnávací experimenty	44
3.1.3 Dělá PFN něco podobného jako GP s RBF kernelem?	44
3.1.4 Jak přesný je PFN na skutečné GP realizaci?	47
3.1.5 Konvergence střední hodnoty a kalibrace rozptylu	49
3.2 PFN nad rodinou Gaussovských procesů: náhodné hyperparametry	51
3.2.1 Trénink nad rozdělením hyperparametrů	51
3.2.2 Struktura attention matic	52
3.2.3 Attention vs RBF kernel	52
3.2.4 Kalibrace rozptylu	54

3.2.5	Přesnost napříč korelačními délkami	56
3.2.6	Identifikace korelační délky z kontextu	56
3.2.7	Predikce na skutečné realizaci a role kontextu	59
3.2.8	Shrnutí: co přináší náhodné hyperparametry	62
3.3	Specializace vrstev: čtení labelů a kernelové porovnávání	63
3.3.1	Korelační analýza attention napříč vrstvami	63
3.3.2	Které hlavy model skutečně potřebuje: head-knockout	66
3.3.3	Rozklad chyby a role hloubky	69
3.3.4	Shrnutí: co se která vrstva naučila	71
3.4	Hloubka jako iterace řešiče: PFN versus gradientní sestup	72
3.4.1	Hloubka snižuje chybu, ale ne jako obyčejná Neumannova řada	73
3.4.2	Rozklad chyby: hloubka odbourává bias i rozptyl	74
3.4.3	Férové srovnání: skutečný gradientní sestup	74
3.4.4	Necitlivost PFN na podmíněnost	76
3.4.5	Shrnutí: hloubka jako preconditioned iterace	76

Závěr

79

Úvod

vns. fvffdbeb bdsbf
Text úvodu....

Kapitola 1

Problematika výběru modelů

V této kapitole uvedeme čtenáře do problematiky výběru modelu. Ukážeme si zde jaké různé přístupy výběru modelu při modelování dat existují, jaké potíže se s tím vážou a lze-li tyto potíže elegantně vyřešit. V praxi se často setkáváme se situacemi, kdy musíme pro daná data zvolit nejvhodnější model z několika kandidátů. Například při analýze časových řad můžeme mít několik různých modelů, které by mohly popsat pozorovaná data, a potřebujeme rozhodnout, který konkrétní model z funkčního prostoru je nejvhodnější. Dnes existuje celá řada metod, jak tento konkrétní model určit, jak najít vhodné parametry modelu, ovšem všechny tyto metody jsou zpravidla dost výpočetně náročné. Avšak se to změnilo s příchodem prudkého rozvoje neuronových sítí a hlubokého učení. O Neuronových sítích víme, že to je dobrý aproximační nástroj a ukázalo se totiž, že abychom mohli co nejefektivněji aproximovat hledaný model pro naše data, lze použít architekturu Transformer, která, jak se postupem času dozvíme, je nejpresnější a nejrychlejší řešení tohoto problému. Zavedeme nejdříve obecnou problematiku výběru modelu, poté Bayesovský princip výběru modelu a nakonec si ukážeme, jak nám v tom může pomoci architektura Transformer.

1.1 Obecný princip hledání modelu

V této sekci si ukážeme obecný princip výběru modelu. Představme si následující situaci: máme k dispozici naměřená data, např. z fyzikálního experimentu nebo ze sociální studie a chceme najít model, který tato data co nejlépe popisuje. Tento model může být například lineární nebo nelineární regrese, rozhodovací strom nebo neuronová síť. Dalším účelem modelu je schopnost generalizace na nová, neviděná data, tj. možnost predikce nebo možnost otestovat pravdivost modelu. V tomto okamžiku před námi stojí několik otázek:

- Jakou strukturu modelu musíme zvolit (např. lineární vs. polynomiální)
- Jak správně zvolit hodnoty parametrů a hyperparametrů modelu (např. síla regularizace nebo počet vrstev v neuronové síti)
- Jak moc složitý model zvolit, aby dobře fitoval trénovací data, ale zároveň dobře generalizoval na nová data

Nakonec z toho vyplývá i formulace daného problému: proces výběru modelu je proces systematického výběru optimálního modelu. Během tohoto procesu vzniká i základní trade-off dilema. Zvolíme-li příliš jednoduchý model, nebude schopen zachytit strukturu dat a bude mít špatný fit i špatnou generalizaci (underfitting). Naopak zvolíme-li příliš složitý model, bude perfektně fitovat (interpolovat) trénovací

data, ale bude špatně generalizovat na nová data (overfitting). Cílem je najít optimální model z těchto hledisek.

Dnes existuje celá řada přístupů k výběru modelu. Jsou to tři hlavní přístupy: Bayesovský přístup (marginal likelihood), cross-validace a informační kritéria (AIC, BIC). Cross-validace je praktický a intuitivní přístup, který odhaduje generalizační chybu pomocí validačních dat. Jinými slovy, data rozdělíme na několik částí, pro každou část trénujeme model na ostatních datech a testujeme na ní. Nakonec průměrujeme chyby přes všechny části a vybereme model s tzv. nejnižší cross-validační chybou [Rasmussen]. Dalším velmi populárním přístupem je Bayesovský přístup, který spočívá v tom, že spočítáme pravděpodobnost modelu daná daty. To znamená, že spočítáme tzv. marginal likelihood, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. Tento přístup má výhodu v tom, že automaticky penalizuje složité modely a nabízí principiální způsob, jak balancovat fit a složitost modelu. Posledním přístupem jsou informační kritéria, jako je Akaike Information Criterion (AIC) a Bayesian Information Criterion (BIC). Tyto metody poskytují rychlou a jednoduchou aproximaci marginal likelihood, která penalizuje složitost modelu. Nás avšak bude v této práci nejvíce zajímat Bayesovský přístup, protože nám poskytuje nejvíce principů pro výběr modelu a je základem pro náš další postup.

1.2 Bayesův přístup k výběru modelu

Zde ukážeme *Bayesův* princip při výběru modelu. Je zvykem použít hierarchický princip při zavádění Bayesovského výběru modelu [Rasmussen].

Označme tedy $\mathbf{w} \in \mathcal{W}$ jako parametry modelu, kde $\mathcal{W}$ je prostor parametrů modelu, mohou to být např. váhy v lineární regresi nebo v neuronové síti. Dále nechť $\boldsymbol{\theta} \in \Theta$ jsou hyperparametry modelu, kde Θ je prostor hyperparametrů modelu, např. počet vrstev v neuronové síti. Nakonec označme $\mathcal{H}_i \in \mathcal{H}$ jako různé struktury modelu, kde $\mathcal{H}$ je prostor modelů, např. různé architektury neuronových sítí. V Bayesovském přístupu k výběru modelu používáme tzv. Bayesovskou inferenci k odhadu pravděpodobnostního rozdělení parametrů, hyperparametrů a modelů na základě pozorovaných dat $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, kde $\mathbf{x}_i$ jsou vstupní data a y_i jsou odpovídající cílové hodnoty, anebo alternativně lze označit jako $\mathcal{D} = \{(\mathbf{X}, \mathbf{y})\}$. Tj. cílem je na základě pozorovaných dat dojít k závěru, který model s kterými parametry a hyperparametry nejlépe vysvětluje pozorovaná data.

Na první úrovni hierarchie odhadneme pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ pro dané (fixní) hyperparametry $\boldsymbol{\theta}$ a model $\mathcal{H}_i$. Zde používáme pojem posteriorního rozdělení, tj. ptáme se, jaké je pravděpodobnostní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k pozorovaným datům $\mathbf{X}$, fixním hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$. Předpokládejme, že chceme získat predikci pro vstupní data $\mathbf{y}$. Potom podle Bayesovy věty platí:

$$P(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \frac{P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)}, \quad (1.1)$$

kde $P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost výstupních dat $\mathbf{y}$ podmíněná vstupními daty $\mathbf{X}$, parametry modelu $\mathbf{w}$, hyperparametry $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$ (tzv. likelihood), $P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)$ je apriorní rozdělení parametrů modelu $\mathbf{w}$ vzhledem k hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$, a $P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je tzv. evidence dat.

Na druhé úrovni hierarchie odhadneme pravděpodobnostní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ pro daný (fixní) model $\mathcal{H}_i$. Podobně jako na první úrovni používáme Bayesovu větu:

$$P(\boldsymbol{\theta}|\mathbf{y}, \mathbf{X}, \mathcal{H}_i) = \frac{P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\boldsymbol{\theta}|\mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)}, \quad (1.2)$$

kde $P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$ je pravděpodobnost dat vzhledem k vstupním datům $\mathbf{y}$, hyperparametrům $\boldsymbol{\theta}$ a modelu $\mathcal{H}_i$ (tzv. likelihood), $P(\boldsymbol{\theta}|\mathcal{H}_i)$ je apriorní rozdělení hyperparametrů modelu $\boldsymbol{\theta}$ vzhledem k modelu $\mathcal{H}_i$, a $P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je evidence dat.

Na třetí úrovni hierarchie odhadneme pravděpodobnostní rozdělení modelů $\mathcal{H}_i$. Opět používáme Bayesovu větu:

$$P(\mathcal{H}_i|\mathbf{y}, \mathbf{X}) = \frac{P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)P(\mathcal{H}_i)}{P(\mathbf{y}|\mathbf{X})}, \quad (1.3)$$

kde $P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i)$ je pravděpodobnost dat vzhledem k výstupním hodnotám $\mathbf{y}$ podmíněná vstupními daty $\mathbf{X}$ a modelu $\mathcal{H}_i$ (tzv. marginal likelihood), $P(\mathcal{H}_i)$ je apriorní rozdělení modelů $\mathcal{H}_i$, a $P(\mathbf{y}|\mathbf{X})$ je evidence dat.

Celý tento postup nám umožňuje systematicky vyhodnotit různé modely, jejich hyperparametry a parametry na základě pozorovaných dat a vybrat ten, který nejlépe vysvětluje data podle Bayesovského principu. Zde je dokonce vidět, proč jsme použili hierarchický přístup - na každé úrovni hierarchie odhadujeme pravděpodobnostní rozdělení na základě předchozí úrovně, což nám umožňuje efektivně zachytit nejistotu na různých úrovních modelování. Postupujeme tedy od nejvyšší úrovně nejistoty, což je vůbec výběr modelu, přes střední úroveň, což je výběr hyperparametrů, až k nejnižší úrovni, což je odhad parametrů modelu, tj. se náš problém na každé úrovni zúžuje, tedy jdeme do hloubky nastavení modelu.

Avšak v praxi při výpočtech hustot pravděpodobností na jednotlivých úrovních hierarchie můžeme narazit na problém, že ten integrál neumíme spočítat analyticky, např. protože tato hustota není normalizovaná (neboli tzv. nevlastní). Takovým integrálem je např. normalizační konstanta na první úrovni hierarchie:

$$P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \int_{\mathcal{W}} P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)d\mathbf{w}, \quad (1.4)$$

anebo na druhé úrovni hierarchie:

$$P(\mathbf{y}|\mathbf{X}, \mathcal{H}_i) = \int_{\Theta} P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)P(\boldsymbol{\theta}|\mathcal{H}_i)d\boldsymbol{\theta} \quad (1.5)$$

Tyto integrály je nutné aproximovat numericky pomocí takových metod jako Markov chain Monte Carlo (MCMC) nebo pokročilejším No-U-Turn Sampler (NUTS), pokud chceme odhadnou integrál z nenormalizovaných hustot pravděpodobnostní, jako je integrál (1.5), nebo např. Type II maximum likelihood (Type II – MLE), chceme-li odhadnout pouze posteriorní pravděpodobnost pro hyperparametry modelu, což spočívá v Laplaceově aproximaci integrálu (1.4) a následné maximalizaci $P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$. Všechny tyto metody jsou však dost výpočetně náročné a může trvat značné množství času, než se podaří najít vhodný model pro daná data. Jedním z účelů této práce je ukázat, jak lze tyto pravděpodobnostní hustoty efektivně aproximovat pomocí úplně nového postupu, a to pomocí architektury Transformer.

1.3 GP regrese

Zde si ukážeme, jak aplikovat Bayesovský přístup výběru modelu na konkrétním příkladě, což bude prokládání dat pro nelineární regresi pomocí Gaussovských procesů (dále jen GP). Dále si ukážeme i základní metody pro hledání vhodného modelu, o kterých jsme zmínili v předchozí sekci, a to jsou Type II – MLE a MCMC. Avšak předtím zadefinujeme základní pojmy GP regresi.

Představme si tedy, že máme před sebou následující problém. Někdo nám donesl sadu dat, mohou to být biomedicínská data, data z ekonomických trhu nebo dokonce i data z fyzikálního experimentu a chce po nás sestavit model, který by popisoval tato data a pomocí kterého bychom mohli také stanovit závěry nebo predikce pro budoucí výsledky na základě aktuálních. Je zcela přirozené, že chceme najít co nejpřesnější model, který by tato data popisoval. Jedním z přístupů, jak toho dosáhnout, je použití Gaussovských procesů (GP) pro regresi. Nejdříve ve zkratce zavedeme pojem GP regresi.

Zvykem je definovat Gaussovský proces pomocí vícerozměrného normálního rozdělení. Mějme tedy vícerozměrnou náhodnou veličinu $\mathbf{X} = (X_1, \dots, X_n)$ a necht' $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ pro $\forall i = 1 \dots n$, pak říkáme, že $\mathbf{X}$ má vícerozměrné normální rozdělení s vektorem středních hodnot $\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)$ a kovarianční maticí $\boldsymbol{\Sigma}$, pokud její hustota pravděpodobnosti je dána vztahem:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right), \quad (1.6)$$

kde $|\boldsymbol{\Sigma}|$ je determinant matice $\boldsymbol{\Sigma}$, vektor $\mathbf{x}$ je vektor realizací, značíme $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Z toho vztahu plyne i definice Gaussovského procesu [Rasmussen]:

Definice 1.1 (Gaussovský proces). Gaussovský proces je soubor náhodných veličin, libovolný konečný počet kterých má sdružené normální rozdělení.

Dalším klíčovým pojmem je tzv. kovarianční funkce (nebo jádro), která určuje vlastnosti GP. Ta se definuje jako funkce $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$, která pro každé dva vstupy $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ vrací hodnotu $k(\mathbf{x}, \mathbf{x}')$, která udává míru podobnosti mezi těmito dvěma vstupy. Je to jistá modifikace kovarianční matice, která je částí klasického vícerozměrného normálního rozdělení, a které vznikne po aplikaci tzv. Kernel triku. Máme-li nějaký proces $f(\mathbf{x})$, teď stejný proces chceme zkoumat i pro jiný vstup, tj. nás zajímá $f(\mathbf{x}')$. Ted' je před námi otázka, jaká je kovariance mezi $f(\mathbf{x})$ a $f(\mathbf{x}')$? Potom se skrz funkční prostor vah a kovarianční matice $\boldsymbol{\Sigma}$ ukáže, že kovarianční funkce je definovaná jako:

$$k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\Sigma} \boldsymbol{\phi}(\mathbf{x}'), \quad (1.7)$$

kde $\boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\Sigma} \boldsymbol{\phi}(\mathbf{x}')$ je součin v prostoru vah s kovarianční maticí dvou různých vstupů.

Nakonec platí, že Gaussovský proces je plně definován svou střední funkcí $m(\mathbf{x})$ a kovarianční funkcí $k(\mathbf{x}, \mathbf{x}')$. Střední funkci a kovarianční funkci tohoto procesu definujeme jako:

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})], \quad k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))], \quad (1.8)$$

a následně říkáme, že proces $f(\mathbf{x})$ je Gaussovský proces a značíme $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$.

Platí dále, že kovarianční funkce, které se také říká kernel (jádro), může mít různé formy. Mezi nejčastěji používané patří např. Radial Basis Function (RBF) kernel (někdy squared exponential kernel), který je definován jako:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right), \quad (1.9)$$

kde σ_f^2 je rozptyl funkce a l je délka škály. Pak existuje spousta dalších kernelů, jako je Matérn kernel, Periodic kernel nebo Linear kernel, každý s různými vlastnostmi a aplikacemi, ale pro naše účely postačí základní RBF kernel. RBF kernel má řadu výhod: je hladký, nekonečně diferencovatelný a má tendenci dobře aproximovat širokou škálu funkcí. Současně platí, že každý datový bod můžeme představit jako střed Gaussovského rozdělení. Z předchozí myšlenky plyne i velmi intuitivní myšlenka o tom, jak funguje GP regrese: každý datový bod přispívá k predikci nového bodu podle své vzdálenosti od něj, přičemž váha tohoto příspěvu je určena kovarianční funkcí (jádro). Tím pádem, čím blíže je nějaký bod k jinému bodu, tím větší vliv na něj má: body blízko sebe mají hodnotu blízkou 1, vzdálené body mají hodnotu blízkou 0 a funguje to jako "měřítko podobnosti" mezi body. Pokud bychom tedy chtěli predikovat např. počasí, tak je jasno, že pokud před 5 hodinami bylo -6 stupňů Celsia, před dvěma hodinami -4 stupňů Celsia a před hodinou -5 stupňů Celsia, tak je zjevné, že tyto hodnoty budou mít vliv na predikci počasí za hodinu a ta teplota bude někde mezi -6 a -4 stupňů Celsia a určitě nestoupne nebo neklesne o víc než 5 hodnot.

1.4 Výběr modelu pro GP regresi

V předchozí kapitole jsme si avšak ukázali, že hledat matematické modely, které přesně popisují data, není vůbec jednoduché nebo dokonce nemožné a je potřeba použít různé metody při hledání vhodného modelu. Hlavní potíží je výpočet integrálů, které se objevovaly při sestavování Bayesovské hierarchie. Zde si ukážeme tři přístupy: nejprve obecný rámec variační inference, poté Type II – MLE jako jeho speciální případ a nakonec MCMC jako alternativní metodu.

1.4.1 Variační inference

Variační inference (také Variační Bayes) je obecný přístup k aproximaci těžce vypočítatelných posteriorních rozdělání, které jsme uvedli v předchozí sekci. Místo přímého výpočtu posteriorního rozdělání $P(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)$, který dle (1.4) vyžaduje integraci přes celý prostor parametrů, hledáme jeho aproximaci z vhodné parametrické rodiny rozdělání.

Konkrétně zavádíme variační rozdělání $q(\mathbf{w})$ z rodiny $\mathcal{Q}$ (typicky např. normálních rozdělání se strukturovanou kovarianční maticí) a snažíme se ho co nejvíce přiblížit skutečnému posteriornímu rozdělání minimalizací KL divergence:

$$q^*(\mathbf{w}) = \arg \min_{q \in \mathcal{Q}} \text{KL}(q(\mathbf{w}) \| P(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)). \quad (1.10)$$

Avšak přímá minimalizace KL divergence je opět náročná pro výpočet, neboť vyžaduje znalost normalizační konstanty (1.4). Namísto toho se maximalizuje tzv. *Evidence Lower BOund* (ELBO):

$$\mathcal{L}(q, \boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{w})}[\log P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)] - \text{KL}(q(\mathbf{w}) \| P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}_i)). \quad (1.11)$$

Lze totiž ukázat, že:

$$\log P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q(\mathbf{w}) \| P(\mathbf{w}|\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i)) \geq \mathcal{L}(q, \boldsymbol{\theta}), \quad (1.12)$$

kde nerovnost plyne z nezápornosti KL divergence. ELBO tedy tvoří dolní mez log-evidence a maximalizace $\mathcal{L}(q, \boldsymbol{\theta})$ přes q odpovídá minimalizaci KL divergence od skutečného posteriorního rozdělání – přičemž normalizační konstantu znát už nepotřebujeme.

Nevýhoda variační inference spočívá v tom, že výsledná aproximace je omezena volbou rodiny $\mathcal{Q}$. Příliš úzká rodina nemusí skutečné posteriorní rozdělání dobře zachytit.

1.4.2 Type II – MLE

Pokud bychom chtěli rychlejší a jednodušší řešení, a namísto hledání celé distribuce nám stačí pouze bodový MLE odhad parametrů, použijeme tzv. Type II maximum likelihood (Type II – MLE), což je speciálním případem variační inference. Tato metoda spočívá v tom, že maximalizujeme tzv. marginal likelihood, což je pravděpodobnost dat daná modelem, integrovaná přes všechny možné hodnoty parametrů modelu. V případě GP regrese je marginal likelihood dána vztahem:

$$P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}) = \int_{\mathcal{W}} P(\mathbf{y}|\mathbf{w}, \mathbf{X}, \boldsymbol{\theta}, \mathcal{H}) P(\mathbf{w}|\boldsymbol{\theta}, \mathcal{H}) d\mathbf{w}, \quad (1.13)$$

kde $\boldsymbol{\theta}$ jsou hyperparametry modelu, $\mathcal{H}$ je struktura modelu, $\mathbf{w}$ jsou parametry modelu, $\mathbf{X}$ jsou vstupní data a $\mathbf{y}$ jsou odpovídající cílové hodnoty. To znamená následující, vzpomeňme si na 2. úroveň hierarchie v Bayesovském přístupu, tj. na rovnici 1.2. Pak posteriorní rozdělání odhadujeme takto:

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta} \in \Theta} P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) \quad (1.14)$$

Principem tohoto přístupu je, že na základě napozorovaných dat hledáme hyperparametry pro náš model, se kterými on popisuje naše data co nejlépe pro dané modely. Tato metoda ale má řadu nevýhod. Náš odhad totiž skončí v lokálním maximu a nemusí vždy dosáhnout optimálních hodnot. To samozřejmě vynucuje nás zkoušet různé startovní body, zkoumat citlivost odhadu a patrně ladit parametry hledání. To už nemluvíme o ne úplně jednoduchých výpočtech, které také vyžadují jisté množství výkonu a času.

Ze statistiky víme, že jedním ze způsobů, jak najít ML odhad, je použít logaritmickou transformaci a pak najít maximum této funkce. Ukážeme to na konkrétním příkladě a pro to použijeme již definovaný v oddílu 1.3 Gaussovský proces s RBF kernelem. Zkusme tedy rozepsat výraz $\log P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta})$ (člen $\mathcal{H}_i$ můžeme vynechat pro lepší čitelnost). Vzpomeňme si, že při definici Gaussovského procesu jsme použili vícerozměrnou Gaussovskou hustotu 1.6. Zlogaritmujeme tento výraz a obdržíme:

$$\log p(\mathbf{y}) = -\frac{1}{2}\mathbf{y}^\top \boldsymbol{\Sigma}^{-1} \mathbf{y} - \frac{1}{2} \log |\boldsymbol{\Sigma}| - \frac{n}{2} \log 2\pi \quad (1.15)$$

Jediné, co nám tedy zbývá v případě Gaussovského procesu, dosadit za kovarianční matici kernelovou matici $\mathbf{K}_y = \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}$, kde $\mathbf{K}(\mathbf{X}, \mathbf{X})_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ a σ_n^2 je rozptyl šumu, čímž obdržíme:

$$\log P(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}, \mathcal{H}_i) = -\frac{1}{2}\mathbf{y}^\top \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_y| - \frac{n}{2} \log 2\pi \quad (1.16)$$

Chceme-li najít extrém této funkce, musíme tento výraz derivovat podle hyperparametrů, čímž dostáváme:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) &= \frac{1}{2}\mathbf{y}^\top \mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \text{tr} \left(\mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right) \\ &= \frac{1}{2} \text{tr} \left((\boldsymbol{\alpha} \boldsymbol{\alpha}^\top - \mathbf{K}_y^{-1}) \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right), \end{aligned} \quad (1.17)$$

kde $\boldsymbol{\alpha} = \mathbf{K}_y^{-1} \mathbf{y}$. Zde je dokonce vidět, proč tento model není optimální z výpočetního hlediska. Je totiž známo, že najít inverzní matici je dost náročná operace. Pro standardní algoritmy platí, že jejich složitost je $O(n^3)$, kde n je velikost matice. V praxi ovšem n nabývá dost velkých hodnot. Avšak lze vyhnout výpočtu přímé inverze, místo toho se může použít např. Choleského rozklad, který **tento** výpočet může usnadnit. Máme-li spočítanou inverzi, pak je složitost zbylých výpočtů $O(n^2)$. Další nevýhodou je to, že nemusíme spadnout do globálního optima a skončit v lokálním optimu, což opět vyžaduje větší pozornost.

Type II – MLE obrázky a rozepsat členy toho logaritmu.

1.4.3 Markov Chain Monte Carlo

Další metodou je Markov Chain Monte Carlo (MCMC). Ta je založena na teorii Markovských procesů. Její princip spočívá v tom, že se snažíme náhodným vzorkováním z rozdělení aproximovat integrál, který neumíme vyřešit analyticky. Konkrétně už jsme si říkali, že integrály 1.4 a 1.5 nemusíme umět spočítat kvůli jejich příliš velké dimenze nebo kvůli tomu, že tyto hustoty nemusí být vlastní. A tak místo přímého výpočtu budeme postupně generovat vzorky z těchto rozdělení a postupně zjistíme alespoň hrubou představu o skutečné podobě těchto rozdělení, dokonce můžeme i aproximovat jakékoliv statistiky tohoto rozdělení. Dalším podstatným bodem je to, že MCMC je založeno na tzv. Markovových řetězcích což je posloupnost náhodných veličin, pro kterou platí $p(\theta_{t+1}|\theta_t, \theta_{t-1}, \dots, \theta_0) = p(\theta_{t+1}|\theta_t)$, kde θ_i pro $i \in \{1, \dots, T\}$ značí vzorky generované z MCMC. Klíčovým požadavkem je, aby řetězec:

1. byl ergodický, tj. ireducibilní a aperiodický,

2. měl stacionární rozdělení $\pi(\theta)$.

Pak vzorky konvergují k $\pi(\theta)$ a můžeme použít pro odhady:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T f(\theta_t) = \mathbb{E}_{\pi}[f(\theta)],$$

kde $f(\cdot)$ je v tomto případě skutečná hustota. Dnes již existuje celá řada různých variant MCMC algoritmů, jsou to např. *Metropolis-Hastings algoritmus*, *Gibbs sampling* nebo modernější *Hamiltonian Monte Carlo*. Avšak není účelem této práce rozebírat všechny tyto metody dopodrobna. Nicméně o jedné metody toho řekneme trochu víc, a to je varianta *No U-Turn Sampler* využívající princip Hamiltonian Monte Carlo. Tato varianta totiž je velmi užívána a relativně účinná při hledání aposteriorních distribucí. Hamiltonian Monte Carlo řeší problém tím, že využívá gradient aposteriorního rozdělení. Základní myšlenkou je pohyb v parametrickém prostoru.

Tento proces vnímáme jako fyzikální systém:

- θ = poloha částice,
- zavádíme pomocnou momentum proměnnou $\mathbf{r}$ (někdy $\mathbf{p}$),
- pohyb řídí Hamiltonovy rovnice s energií:

$$H(\theta, \mathbf{r}) = -\log p(\theta|\mathcal{D}) + \frac{1}{2} \mathbf{r}^T \mathbf{M}^{-1} \mathbf{r},$$

kde první člen je potenciální energie (negativní log-posterior) a druhý je kinetická energie.

Rovněž si musíme uvědomit, že částice má tendenci se pohybovat po hladinách konstantní energie, a proto zkoumá prostor efektivněji než náhodné procházky, které používají předchozí algoritmy!

Algoritmus 1 Hamiltonian Monte Carlo

- 1: Vzorkuj náhodný momentum: $\mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$
 - 2: Simuluj Hamiltonovy rovnice pomocí leapfrog integrace na L kroků délky ϵ
 - 3: Metropolis accept/reject krok (zachovává přesnou stacionaritu)
-

Hamiltonian Monte Carlo avšak má dva hyperparametry, které je těžké nastavit:

- délka kroku ϵ (step size)
- počet kroků L (*trajectory length*)

NUTS řeší oba problémy tím, že automaticky volí délky trajektorie. To znamená, že pokračuje v simulaci, dokud trajektorie nezačne dělat tzv. *U-turn*, což znamená, že částice se po trajektorii otočí a začne se vracet zpět směrem k místu, odkud vyšla. Toto *U-turn kritérium* lze vyjádřit takto: směr pohybu začne být opačný než směr od startovní pozice, matematicky to znamená $(\theta_{\text{forward}} - \theta_{\text{backward}}) \cdot \mathbf{r}_{\text{forward}} < 0$, kde skalární součin vektoru mezi krajními body trajektorie a momentem je záporný, což signalizuje, že se pohybujeme zpět. A tento princip má dobrý smysl, jakmile se částice vrací zpět, už není efektivní pokračovat, znamená to, že jsme plně prozkoumali danou "energetickou hladinu". Další kroky by jen ztrácely výpočetní čas tím, že by procházely oblastí, které už jsme navštívili. NUTS tedy automaticky zastaví simulaci v optimálním bodě, když jsme prozkoumali maximum užitečného prostoru, ale ještě před tím než začneme ztrácet efektivitu návratem zpět.

Kapitola 2

Prior–Data Fitted Networks

V této kapitole uvedeme samotnou architekturu Prior–Data Fitted Networks (dále jenom PFN). Začneme krátkou motivací, kde popíšeme hlavní ideu této Deep Learning architektury. Následně ukážeme mechanismy, které se schovávají uvnitř PFN, jinými slovy, jak PFN postupně zpracovává vstupní hodnoty, a to vše na příkladě Gaussovských procesů.

2.1 Úvod do Prior–Data Fitted Networks

Prior–Data Fitted Networks je Deep Learning (dále jenom DL) architektura, kterou v roce 2022 uvedli Samuel Muller a Noah Hollmann v článku (Müller et al., 2022). Hlavní motivací pro vznik tohoto modelu je snaha posunout techniky Bayesovské statistiky na další úroveň spolu se strmým rozvojem v oblasti strojového učení. PFN je založen na proslulé architektuře transformer, která je viníkem dnešního pokroku všemožných Deep Learning modelu, a o niž víme díky velkým jazykovým modelům, a není to náhoda, existuje totiž jisté spojení mezi těmito architekturami. Ačkoli samotnou architekturu PFN a její technické nuance budeme podrobně diskutovat až v následující sekci, zde jenom uvedeme její základní charakteristiky. PFN je tzv. *encoder-only* architektura (tj. neobsahuje *dekóder* blok, ale pouze *enkóder* blok transformeru), na kterou je pak napojena softmax hlava, pro vyhodnocování výsledků ve tvaru pravděpodobností. Na vstup PFN dostává sekvenci dat. V případě velkých jazykových modelu by se jednalo o slovy nebo věty, umístěné v jistém pořadí. Tady je ta myšlenka shodná, ale namísto slov dostane síť neuspořádanou sekvenci dat, které získáme z prior rozdělení. Právě toto je důležitý intuitivní pojetí, na kterém je založená celá ta architektura. Stejně jako po jazykovém modelu, který dostal sekvenci slov, budeme chtít aby odhadl celkový kontext a jako výsledek např. i následující neznáme slovo, které by mohlo následovat, budeme i po PFN chtít, aby po předložení sekvenci dat, zkusil uchopit celkový trend (neboli kontext) a předpovědět, co by mohlo následovat dále. Což je velmi relevantní, když zpracováváme časové řady a chceme řešit regresní problém, jak tomu je např. během práce s Gaussovskými procesy.

Celý princip fungování této metodiky lze popsat následujícím způsobem. Jak již jsme diskutovali dříve, účelem Bayesovského přístupu ve statistice je využití apriorní znalosti nebo informace o jistém jevu v přírodě. A tento princip se dobře prolíná s obecnou úlohou aproximace funkcí pomocí neuronových sítí, protože při jejich návrhu vždy musíme enkódovat apriorní znalosti o této funkci. To se dělá buď přímo skrz architekturu konkrétního modelu nebo za pomoci různých regularizací. Každopádně je to velmi netriviální úloha, kterou je třeba vždy řešit než začneme řešit kteroukoliv úlohu pomocí DL nástrojů. Jako jedno z možných řešení se nabízí právě tzv. Bayesovská inference, jejíž hlavním předpokladem je, že se jistý jev nebo proces vyskytuje v reálném světě a disponujeme informacemi o jeho chování. Tento předpoklad nám umožňuje definovat následující postup. Uvažujme jev nebo celý proces (to jest prior, pro jednoduchost si pod tím můžeme představit obyčejnou množinu dat), ze kterého mů-

žeme neomezeně vzorkovat simulované datové sady $\mathcal{D}$ reprezentující různé prediktivní úlohy (body). Bez újmy na obecnosti předpokládejme, že tyto datové sady obsahují vstupní příznaky $\mathbf{x} \in \mathbb{R}^d$ a k nim příslušné výstupní třídy $y \in \mathcal{Y}$. Během trénování sítě PFN náhodně vygenerujeme dávku (batch) obsahující K takových datových sad. S každou sadou v dávce pak naložíme podle algoritmu 2.

Algoritmus 2 Příprava jedné trénovací dávky pro PFN

- 1: **for** $i = 1, \dots, K$ **do**
 - 2: Zvol náhodně velikost kontextu N_i
 - 3: Rozděl vygenerovanou datovou sadu na kontextovou (trénovací) část $\mathcal{D}_{train}^{(i)} = (\mathbf{x}_j^{(i)}, y_j^{(i)})_{j=1}^{N_i}$ a testovací část obsahující (pro jednoduchost) jeden testovací vzorek $(\mathbf{x}_{test}^{(i)}, y_{test}^{(i)})$
 - 4: Předlož modelu data ve tvaru $D^{(i)} = \mathcal{D}_{train}^{(i)} \cup \{\mathbf{x}_{test}^{(i)}, y_{test}^{(i)}\}$
 - 5: **end for**
-

Následně natrénujeme náš model q_θ tak, že mu na vstup předložíme kontextovou sadu společně s testovacím příznakem a učíme ho predikovat správný výstup. K tomu minimalizujeme ztrátovou funkci Negative Log-Likelihood (NLL) přes celou dávku K vzorků ve tvaru:

$$L(\theta) = - \sum_{i=1}^K \log q_\theta(y_{test}^{(i)} | \mathbf{x}_{test}^{(i)}, \mathcal{D}_{train}^{(i)})$$

Optimalizací této funkce (např. pomocí gradientního sestupu) hledáme parametry $\hat{\theta} = \arg \min_{\theta} L(\theta)$, čímž se síť učí provádět inferenci in-context na základě poskytnutých dat.

Toto schéma nám umožňuje aproximovat aposteriorní prediktivní rozdělení (PPD) pro libovolný problém, pro který jsme schopni definovat apriorní rozdělení (prior) a vzorkovat z něj data. Zde je namíste si ujasnit terminologii. **Aposteriorní rozdělení (Posterior)** popisuje naši nejistotu ohledně samotného generativního modelu p (nebo jeho parametrů) poté, co jsme pozorovali trénovací data $\mathcal{D}_n$. Matematicky se jedná o podmíněné rozdělení modelu vzhledem k těmto datům, které se značí jako $\Pi(p|\mathcal{D}_n)$. **Aposterioorní prediktivní rozdělení (PPD - Posterior Predictive Distribution)** naproti tomu popisuje nejistotu ohledně nových dat, které model dosud neviděl, jako je například predikce testovacího štítku y pro nový vstupní příznak x , na základě již pozorovaných dat $\mathcal{D}_n$. Vypočítá se jako průměr všech možných podmíněných rozdělení $p(y|x)$ vážených jejich aposteriorní pravděpodobností. Matematicky je definováno pomocí integrálu:

$$\pi(y|\mathbf{x}, \mathcal{D}_n) = \int p(y|\mathbf{x}) d\Pi(p|\mathcal{D}_n).$$

Takže účelem PFN je aproximovat právě PPD.

V praxi to znamená, že model je trénován na široké rodině úloh vygenerovaných z tohoto prioru. Tím se PFN naučí implicitně provádět bayesovskou inferenci nad daným prostorem funkcí.

V případě Gaussovských procesů to vypadá následovně: Jelikož známe přesný tvar rozdělení (např. Gaussovský proces s RBF kernelem), dokážeme z něj nekonečněkrát vzorkovat nová data. Navíc můžeme hyperparametrům RBF kernelu přiřadit vlastní rozdělení (tzv. hierarchický prior), čímž úlohu pro PFN zesložitíme. V této chvíli se model musí naučit generalizovat (tj. najít obecné řešení) přes celou rodinu funkcí pro všemožné hodnoty hyperparametrů. Potom stačí takto natrénovanému modelu předložit několik známých bodů (tzv. kontext) a on nám v jediném průchodu sítě vrátí rozdělení, které chceme predikovat a snaží se co nejvíce aproximovat původní funkci.

2.2 Praktická motivace k PFN

Abychom navázali na předchozí kapitolu, kde jsme uvedli dnes již standardní metody pro aproximaci PPD, ukážeme motivační příklady s porovnáním PFN s MCMC metody a variační inferencí. Muller ve své práci tuto problematiku uvedl a na ní představil světu PFN architekturu (Müller et al., 2022). Hlavní výhodou PFN je dle něj to, že na rozdíl od variační inference nebo metod Monte Carlo, že se učí aproximovat PPD přímo z datových vzorků. Jediným a velmi důležitým předpokladem ale je, abychom mohli rychle a výpočetně levně vzorkovat z apriorního rozdělení. Mezitím MCMC požaduje nenormované posteriorní rozdělení. Musíme umět jeho spočítat likelihood a také mít prior jako hustotu, ne jenom možnost vzorkovat. Pro variační inferenci potřebujeme znát sdruženou distribuci parametru a dat, často tyto distribuce jsou těžce spočitatelné. Navíc často je optimalizace takových řešení výpočetně náročná (výpočet gradientů, ELBO optimalizace) a kvalita závisí na volbě variační rodiny rozdělení. Dalším úskalím je, že pro každý nový dataset je nutné spustit celý proces znovu, což jenom zhoršuje časovou a výpočetní náročnost při řešení jistého problému. Výhodou těchto klasických řešení je, že např. MCMC konverguje k pravému posterioru v limitě nekonečně mnoha vzorků. Type II – MLE (speciální případ variační inference) konverguje k pravým hyperparametrům v limitě nekonečně dat za předpokladu korektně specifikovaného modelu. Navíc pro všechny modely platí, že systematická chyba takových aproximací, tzv. bias, jde k nule.

PFN obrací situaci. Model je předtrénován offline na obrovské mase syntetických datasetů z prioru a inference se pak provádí jediným dopředním průchodem (forward passem) sítí. Strukturální požadavek je výrazně slabší – stačí umět vzorkovat z apriorní distribuce, není potřeba hustota ani její gradient. To otevírá třídu priorů, kterou klasické metody pojmout neumí, např. BNN s libovolnou architekturou nebo ručně psané generativní procedury. Inference je řádově rychlejší, protože model je předtrénovaný. Müller ukazuje cca 200× zrychlení proti Type II – MLE a přibližně 1 000× až 8 000× proti NUTS, navíc na rozdíl od NUTS, PFN neprovádí žádné optimalizace před inferencí (tj. žádný warmup, žádná adaptace, žádné chain diagnostics). Konečně, PFN není trénován na rekonstrukci posterioru bod po bodu, ale přímo na predikční kvalitě skrz cross-entropy s PPD. Nevýhody jsou strukturální. Jak se ukáže dále, nemáme vždy zaručeno to, že bias bude klesat k nule. Naopak např. u architektury transformer bias nikdy nezmizí. To je kvalitativně jiná situace než u NUTS nebo Type II – MLE, které teoreticky konvergují přesně k prioru a jejich bias postupně mizí. Další nevýhodou je např. to, že pokud testovací dataset pochází z jiné distribuce než trénovací prior, PFN může selhat nebo minimálně výsledky nebudou tak přesné. NUTS je oproti tomu robustní v tom smyslu, že jeho výstupem je vždy korektní posterior pro právě ten model, který byl explicitně specifikován, bez ohledu na původ dat. Nakonec je třeba mít na paměti, že PFN musí být předem natrénovaný na konkrétním problému a s obrovským množstvím dat. Např. pokud bychom chtěli aproximovat Gaussovský proces, je třeba PFN předem natrénovat na dostatečně velké rodině funkcí a pestrout distribucí parametrů/hyperparametrů. Pokud by samozřejmě někdo to udělal za nás, a používali bychom předtrénovaný PFN jako hotový produkt (via dnešní LLM), pak bychom pro libovolný problém dostávali okamžitou aproximaci.

2.3 Vnitřní architektura PFN

V této sekci se pokusíme popsat procesy, které se dějí uvnitř PFN transformeru. Naším účelem není poskytnout kompletní popis architektury transformer obecně, za co odpovídají jednotlivé komponenty nebo parametry. Předpokládáme, že čtenář je již seznámen se základy architektury transformer. Pokud tak tomu není, doporučujeme nastudovat alespoň základní pojmy a principy této architektury. Potřebné zájemce jenom odkážeme na příslušnou kapitolu v knize (Prince, 2023). Přesto pevně věříme, že našemu výkladu budou rozumět i laici v oblasti strojového učení, neboť zde nepotřebujeme pokročilé znalosti

nebo složité pojmy a zkusíme vše popsat co nejjednodušeji. Pokud se takové ve výkladu objeví, slouží jenom pro odborníky v této oblasti a na porozumění hlavních principů nebudou mít velký vliv. Dále opět podotýkáme, že účelem práce není rozebírat konkrétní implementaci PFN a související se s tím nuance. Navíc PFN je relativně mladá architektura, které se neustále vylepšuje a aktualizuje, proto pro nás nemá smysl jít do takových detailů. Aktuální verzi zdrojového kódu PFN naleznete zde (<https://github.com/automl/PFNs>).

V předchozí sekci jsme již zmínili, že PFN je transformer, který se trénuje na obrovském množství syntetických datasetů, které vzorkujeme z prioru. Tímto způsobem se PFN naučí provádět Bayesovskou inferenci a po natrénování stačí jeden forward pass sítí a model vrátí kompletní PPD pro nové testovací body na vstupu. V této sekci budeme uvažovat, že naším priorem je Gaussovský proces s RBF kernelem. Dále chceme upozornit, že celý proces, který se odehrává uvnitř PFN od vstupu po výstup, popíšeme s použitím konkrétní konfigurace, kterou jsme běžně používali během zkoumání PFN:

1. Dimenze embeddingu $EMSIZE = 512$.
2. Počet *attention hlav* v každé vrstvě $NHEAD = 8$.
3. Dimenze vnitřních skrytých vrstev dopředných neuronových sítí $NHID = 1024$.
4. Počet vrstev $NLAYERS = 6$.

Jelikož se vše chystáme navíc ukázat pro případ 1D GP regrese, použijeme úplně klasické nastavení PFN, které uvádějí Muller a Hollmann v (Müller et al., 2022), konkrétně s těmito parametry:

- `features_per_group = 1` – nastaven na jedničku, protože řešíme 1D regresi.
- `attention_between_features = False` – slouží pro zpracování tabulkových dat; pokud je zapnutý, jedná se o architekturu *TabPFN*, kterou v této práci neuvažujeme (viz Hollmann et al., 2022).

Nakonec uvažujme, že provádíme inferenci pro GP regresi s 20 trénovacími a 50 testovacími body. Teď už máme nachystané formální parametry a nastavení, což nám umožňuje se pustit na samotný popis. Celý proces si rozdělíme na několik fází.

2.3.1 Fáze 1 - Meta-Learning

Pro obecnost ukážeme, jak probíhá učení pro případ, kdy hyperparametry pro RBF kernel nejsou známy a pochází z nějakého rozdělení. Případ fixních hyperparametrů je pouze speciálním případem tohoto problému a všechno, co napíšeme dále, platí i pro tento speciální případ. Meta-Learning je paradigma strojového učení, které spočívá v tom, že namísto toho, abychom model naučili řešit jeden konkrétní fixní problém, naučíme ho řešit celou třídu podobných úkolů. Konkrétně v našem případě by to znamenalo, že model naučíme na základě dat z GP s RBF kernelem, kde mu ukážeme celé spektrum všemožných hyperparametrů, které se v RBF kernelu vyskytují. Potom stačí jenom předložit modelu těch 20 kontext bodů a hned bude schopen aproximovat GP, ze kterého tyto body pocházejí. V předchozí sekci jsme již drobně popsali, jak funguje trénink PFN, formálněji a mnohem podrobněji lze tento proces zapsat takto:

Algoritmus 3 Meta-trénink sítě PFN na syntetických GP datech

Require: Model q_θ , apriorní rozdělení hyperparametrů Π , velikost kroku η , velikost dávky m

- 1: **while** není dosaženo konvergence **do**
- 2: **for** $j = 1, \dots, m$ **do**
- 3: Vzorkuj hyperparametry GP: $(\ell_j, \sigma_j, \sigma_{n,j}) \sim \Pi$
- 4: Vzorkuj vstupní body $x_1, \dots, x_N$ rovnoměrně z $[0, 1]$
- 5: Vzorkuj výstupy z GP s daným kernelem: $(y_1, \dots, y_N) \sim \mathcal{GP}(0, k_{\ell_j, \sigma_j}) + \mathcal{N}(0, \sigma_{n,j}^2 I)$
- 6: Rozděl na kontextovou sadu $\mathcal{D}_j = \{(x_i, y_i)\}_{i=1}^n$ a testovací body $\{(x_i^*, y_i^*)\}_{i=1}^t$
- 7: **end for**
- 8: Předej q_θ kontexty $\mathcal{D}_j$ a testovací vstupy x_i^* ; model vrátí prediktivní distribuce přes biny
- 9: Spočítej ztrátu (NLL):

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{j=1}^m \sum_{i=1}^t \log q_\theta(y_i^* | x_i^*, \mathcal{D}_j)$$

- 10: Aktualizuj: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
 - 11: **end while**
 - 12: **return** $\hat{\theta}$
-

Tímto způsobem se dostáváme do situace, kdy model vidí miliony různých datasetů generovaných z GP prioru a učí se pravidla, dle kterých pak musí uhádnout, jak vypadá posteriorní predikce na základě předložených dat. Tohle je velmi důležitá myšlenka, PFN se neučí aproximovat konkrétní funkce, učí se inferenční pravidlo.

2.3.2 Fáze 2 - Vstup do modelu

Během inference model dostane tři tenzory:

1. Souřadnice trénovacích bodů `x_train`.
2. Naměřené hodnoty na trénovacích bodech `y_train`.
3. Souřadnice testovacích bodů `x_test`.

Jak už plyne z logiky věci, `x_train` a `y_train` jsou dva různé objekty, první říká, kde jsme měřili (souřadnice na ose x), a druhý říká, co jsme v těchto místech naměřili. Tak dohromady tvoří datové body (x_i, y_i) . Souřadnice `x_test` nám říká, kde chceme ty predikce, přitom model by měl vrátit pravděpodobnostní rozdělení pro příslušné y -hodnoty, ale k tomu se dostaneme později v našem výkladu. Následně se `x_train` a `x_test` spojí do jedné sekvenci a model s nimi pracuje jako s jedním objektem. Přičemž modelu současně explicitně řekneme, že prvních 20 souřadnic jsou trénovací hodnoty a zbytek jsou testovací (50 testovacích souřadnic), jak jsme si to definovali na začátku sekce. Pro y -hodnoty je ta situace podobná, vytvoříme jednu sekvenci pro `y_train` a `y_test` hodnoty, až na to, že místo test hodnot vložíme NaN hodnoty, protože právě ty musí model odhadnout a nechceme je do modelu vkládat v žádné podobě. Pro zájemce uvedeme, jak to pak vypadá v kódu. V metodě `forward()` se `x_train` a `x_test` spojí pomocí funkce konkaténace:

```
1 x = torch.cat((x_train, x_test), dim=1) # (1, 70, 1)
```

Jedná se o tenzor, kde první dimenze je počet datasetů, které zpracováváme naraz, druhá dimenze je počet bodů v sekvenci a třetí dimenze je počet příznaků, který jsme na začátku této sekce nastavili na 1. Pro y -hodnoty analogicky:

```
1 y = torch.cat((y_train, NaN * torch.zeros(1, 50, 1)), dim=1) # (1, 70, 1)
```

Zde je explicitně vidět, že test hodnoty se nahradí za NaN. Dále si myslíme, že je namístě říct jedno upřesnění, aby nedošlo k nedorozumění. To, že modelu v y -hodnotách předkládáme NaN neznamena, že pak budeme chtít tyto hodnoty nahradit za odhady někde v průběhu zpracovávání, je to čistě technická záležitost. Zůstanou NaN po celou dobu. Výsledky obdržíme v jiné podobě, opět, budeme o tom diskutovat dále.

Dále následuje tzv. *Feature grouping*, o parametru, který určuje počet příznaku jsme také zmiňovali na začátku sekce. Připomínáme, že pro náš jednoduchý případ 1D regrese je naven na 1. Tento mechanismus slouží hlavně pro případ, kdy chceme zpracovávat tabulkové hodnoty, anebo pracovat s vícerozměrnými daty, ale to se týká hlavně verze modelu *TabPFN*. V tabulkových datech s 12 sloupci (věk, plat, výška, ...) model potřebuje tyto sloupce nějak zpracovat. Parametr `features_per_group` určuje, kolik sloupců se seskupí do jednoho *tokenu*. Pro náš 1D GP je grouping triviální: máme 1 příznak, tedy 1 grupu. Tensor x se tak přeformátuje z $(1, 70, 1)$ na $(1, 70, 1, 1)$.

Dalším krokem je převod dat na vstupu na tzv. *embedding*, tj. do řeči, které rozumí transformer. Ve skutečnosti se jedná o vytváření nového prostoru, ve kterém pak PFN bude operovat s daty, a kterému rozumí víc, než kdyby musel pracovat pouze se sekvencí dat.

X-encoder:

Zde to je jednoduchá lineární vrstva, která vezme jednu konkrétní číselnou hodnotu x -souřadnice a vynásobí ji maticí vah 1×512 (parametr velikosti embeddingu jsme nastavili nahoře). Výsledkem je 512-dimenzionální vektor (*embedding*). Tímto způsobem jsme zakódovali polohu bodu v tom novém prostoru, ze kterého model umí jednoduše číst a operovat v něm. Výstupem je `embedded_x` tvaru $(1, 70, 1, 512)$

Y-encoder:

Pro y -hodnoty je ta situace zajímavější, jelikož v předchozím kroku, jsme si řekli, že pro testovací hodnoty modelu předložíme NaN hodnoty. A teď otázkou je, jak tyto hodnoty zakódovat do *embeddingu*. K tomu slouží metoda `NanHandlingEncoderStep`, která nejdříve transformuje skalární y -hodnotu na vektor délky 2:

1. Pro trénovací pozice (y existuje): $[y, -1]$ – skutečná hodnota a indikátor "data jsou k dispozici".
2. Pro testovací pozice ($y = \text{NaN}$): $[0, +1]$ – nulová hodnota a indikátor "data chybí".

Pak lineární vrstva opět převede dvourozměrný vektor do 512-dimenzionálního prostoru. Výstupem je `embedded_y` tvaru $(1, 70, 512)$, tady je o jeden parametr méně, protože příznaky mezi skupinami se nevztahují na y -hodnoty.

V této chvíli máme dva embeddingy, cílem ale je, mít jeden hromadný embedding, který v sobě nese informace o souřadnici x a odpovídající ji hodnotě y . Zde opět existují dva způsoby implementace v PFN: pro `attention_between_features=False` a `attention_between_features=True`, kde poslední verze je opět určena pro práci s vícedimenzionálními daty. Pro tu první verzi je postup velmi jednoduchý – `embedded_x` a `embedded_y` jednoduše sečteme. Výsledkem je jeden *token* pro jednu konkrétní pozici, tj. tento *token* kombinuje informaci o poloze x a hodnotě y v jednom vektoru. Sčítání tady funguje, protože oba embeddingy jsou již stejného tvaru a jsou ve stejném 512-dimenzionálním prostoru a model bude umět z tohoto součtu dekodovat obě informační položky díky předchozímu postupu. V kódu to pak formálně vypadá následovně:

```
1 embedded_input = embedded_x + embedded_y.unsqueeze(2)
2 # (1, 70, 1, 512) + (1, 70, 1, 512) -> (1, 70, 1, 512)
```

Druhý režim zde již popisovat nebudeme, zájemce odkážeme na zdrojové kódy na GitHub, které jsme uvedli na začátku kapitoly.

2.3.3 Fáze 3 – Transformer vrstvy

Tenzor `embedded_input` pak musí projít 6 transformer vrstvami (jejich počet jsme též definovali na začátku sekce). Jakmile embeddingy vstupují do transformer vrstev je zvykem jim říkat *token*. To znamená, že objektům `embedded_x` a `embedded_y` bychom říkali `x_token` a `y_token`. Ačkoliv jsme tyto dva embeddingy sloučili do jednoho, pořád musíme na ně nahlížet jako na samostatné objekty, protože naším cílem je nakonec získat `y_token`, který v sobě bude obsahovat predikce pro testovací hodnoty. Sloučení se tedy provádí pouze pro technické potřeby architektury. Vraťme se ale zpátky k věci. Každá vrstva v naší konfiguraci provádí dvě hlavní operace:

1. *Self-attention* přes datové body (neplést s *attention between features*)
2. Feedforward MLP

Self-attention:

Tenzor se transponuje na $(1, 1, 70, 512)$, aby sekvence 70 bodů byla na dimenzi $\text{dim}=2$ (standardní konvence pro *attention*). Multi-head *attention* s 8 hlavami pak počítá:

Pro každou hlavu h (z 8):

$$\mathbf{Q}^{(h)} = \mathbf{x} \cdot \mathbf{W}_Q^{(h)}, \quad \mathbf{K}^{(h)} = \mathbf{x} \cdot \mathbf{W}_K^{(h)}, \quad \mathbf{V}^{(h)} = \mathbf{x} \cdot \mathbf{W}_V^{(h)}$$

kde $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{512 \times 64}$ a $\mathbf{W}_V \in \mathbb{R}^{512 \times 64}$ (protože $512/8 = 64$).

Klíčový detail: $\mathbf{K}$ a $\mathbf{V}$ se počítají pouze z trénovacích bodů. V kódu:

```
1 attention_src_x = x[:, :single_eval_pos].transpose(1, 2)
2 # first 20 positions
```

Attention matice je pak:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{64}}\right)$$

Pro trénovací body (pozice 0–19) je $\mathbf{Q}$ i $\mathbf{K}$ ze stejných 20 bodů, takže matice je 20×20 – trénovací body se dívají na sebe navzájem.

Pro testovací body (pozice 20–69) $\mathbf{Q}$ pochází z testovacích bodů, ale $\mathbf{K}$ pochází z trénovacích bodů. Matice je 50×20 – testovací body se dívají na trénovací body.

Tohle je přímá analogie ke GP predikci. Ve formuli $\mu_* = k(x_*, \mathbf{X})\mathbf{K}^{-1}\mathbf{y}$ se testovací body taky „dívají“ na trénovací body přes kernel $k(x_*, \mathbf{X})$. Rozdíl je, že GP používá explicitní RBF kernel, zatímco PFN se naučil vlastní implicitní „kernel“ zakódovaný ve váhách $\mathbf{W}_Q$ a $\mathbf{W}_K$.

Výstup *attention*:

$$\text{Output}^{(h)} = \mathbf{A} \cdot \mathbf{V}^{(h)}$$

Všech 8 výstupů (každý $\in \mathbb{R}^{64}$) se zřetězí a projde výstupní projekcí $\mathbf{W}_O$:

$$\text{MultiHead} = \text{Concat}(\text{Output}^{(1)}, \dots, \text{Output}^{(8)}) \cdot \mathbf{W}_O$$

Po *attention* následuje reziduální spojení (vstup + výstup *attention*) a LayerNorm – normalizace, která stabilizuje trénink. Reziduální spojení je technika, která přidává původní vstupní reprezentaci (třeba embedding datového bodu) k výstupu nelineární vrstvy (například MLP) těsně předtím, než tento výsledek pošle dál do sítě. Reziduální spojení zajišťuje, že model nezačíná optimalizaci chaotickým hádáním,

nýbrž bezpečným průtokem dat, ke kterému postupně přidává potřebnou komplexitu. Co se týče Layer-Norm, ten normalizuje embedding na nulový průměr a jednotkový rozptyl přes posledních d dimenzí (v našem případě 512). Pro vektor x délky 512:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma + \epsilon},$$

kde μ a σ jsou průměr a směrodatná odchylka přes 512 komponent vektoru a $\epsilon = 10^{-5}$ je malá konstanta pro numerickou stabilitu.

MLP – Multi-Layer Perceptron:

MLP je dvouvrstvá dopředná síť:

vstup (512) → Linear(512 → 1024) → GELU → Linear(1024 → 512) → výstup (512)

Vezme 512-dimenzionální vektor, rozšíří ho do 1024 dimenzí přes lineární vrstvu, aplikuje nelineární aktivaci GELU (hladká varianta ReLU), a zase zúží zpět na 512.

GELU (*Gaussian Error Linear Unit*) je funkce

$$\text{GELU}(x) = x \cdot \Phi(x),$$

kde Φ je distribuční funkce standardního normálního rozdělení. Pro velká kladná x se chová jako identita, pro velká záporná x vrací přibližně nulu.

Proč nakonec musíme přidávat MLP je pro zkušeného čtenáře v oblasti strojového učení poněkud triviální otázka, vysvětlíme i pro ostatní čtenáře. *Attention* je totiž lineární operace (ačkoliv se to nezdá), počítá pouze vážené průměry. MLP do tohoto procesu přidává nelinearitu, bez něj by transformer mohl dělat jenom lineární transformace s embeddingy. Avšak ve strojovém učení je to nedostačující, chceme zachytit a složitější transformace a operace, čímž zvyšujeme generalizační schopnosti modelu. A právě díky této vlastnosti může PFN aproximovat např. inverzi matice $\mathbf{K}^{-1}$ v GP. Váhy druhé lineární vrstvy jsou inicializovány na nulu. To znamená, že na začátku tréninku MLP vrací nulový vektor, a tak se tato vrstva na začátku chová jako identita. To se však mění v průběhu tréninku. Model začíná od jednoduchého stavu a postupně se učí komplexnější transformace. Po MLP opět následuje reziduální spojení a normalizace.

Toto se opakuje tolik, kolik vrstev obsahuje model, v našem případě 6 krát. Po každé vrstvě se embedding každého bodu obohacuje o informaci z ostatních bodů (přes *attention*) a transformuje (přes MLP). Po 6 vrstvách embedding testovacího bodu obsahuje dostatečnou informaci k tomu, aby *dekóder* z něj vyrobil prediktivní distribuci.

Po průchodu 6 vrstvami dostáváme opět nový tenzor. Ten obsahuje zpracované reprezentace jak trénovacích, tak i testovacích bodů. Naším úkolem teď by bylo extrahovat testovací hodnoty y (neboli *y-tokens*, jak jsme diskutovali výše), respektive jejich predikce, které model postupně vyrobil. Zbytek je pro účely inference nepodstatný. Platí totiž, že celá architektura PFN je postavena tak, že se právě v *y-tokenu* postupně akumulují informace přes všechny vrstvy transformeru přes *attention* z *x-tokenů* a z kontextu trénovacích bodů. Na konci celého procesu *y-token* pro testovací pozici obsahuje 512-dimenzionální vektor, který implicitně kóduje celou posteriorní prediktivní distribuci (PPD) pro daný trénovací bod. Uveďme opět pro zájemce, jak tento proces vypadá uvnitř, což by mělo pomoci k pochopení, co se v tomto kroku odehrálo:

```
1 # Split into train and test part
2 test_encoder_out = encoder_out[:, current_context_len:, :]
3 # (1, 50, 1, 512) -- test positions (20-69)
4
5 train_encoder_out = encoder_out[:, :current_context_len, :]
6 # (1, 20, 1, 512) -- train positions (0-19)
```

Proměnná `current_context_len` je rovna `single_eval_pos = 20`.

V dalším kroku se extrahuje *y-token* – poslední *token* v dimenzi *feature groups*:

```
1 test_encoder_out = test_encoder_out[:, :, -1, :]  
2 # (1, 50, 512) -- only y-token for test positions
```

2.3.4 Fáze 4 – Dekóder, převod embeddingu na logity.

Jsme si již řekli, že PFN je *encoder-only* architektura, tj. *decoder* blok, který je součástí obecné architektury transformer, v něm chybí. Decoder blok ale je napojen na hlavu, která v sobě obsahuje opět MLP vrstvu a softmax vrstvu. Tyto součástky však budeme potřebovat. Na konci předchozí fáze jsme získali tenzor `test_encoder_out` o tvaru (1, 50, 512), který v sobě obsahuje pro každý z 50 testovacích bodů jeden 512–dimenzionální embedding. Ted' potřebujeme z tohoto vektoru vytvořit prediktivní distribuci. To se opět udělá pomocí MLP (jednoduchá dopředná síť), která je doplněná o jednu novinku. Celkově schéma vypadá následovně:

vstup(512) → Linear(512 → 1024) → GELU → Linear(1024 → numBars) → výstup(numBars)

Jednotlivé prvky v schématu, který jsme uvedli na začátku sekce, plní následující funkci:

1. **Linear(512 → 1024)**: Vezme 512–dimenzionální embedding a promítne ho do 1024 dimenzí. To je lineární transformace – násobení maticí vah $W_1 \in \mathbb{R}^{512 \times 1024}$.
2. **GELU()**: Aplikuje nelineární aktivaci. Bez ní by dvě lineární vrstvy za sebou byly ekvivalentní jedné lineární vrstvě (součin matic je matice). GELU umožňuje *decoderu* zachytit nelineární vztahy mezi embeddingem a výstupní distribucí.
3. **Linear(1024 → numBars)**: Zúží z 1024 dimenzí na numBars (řekněme 1000). Výstupem je 1000 čísel, tzv. *logity*, a to jedno pro každý bucket.

Je vidět, že tady se objevuje nová proměnná `numBars`, jedná se totiž o analogii binu v histogramech, který se zavádí pro tzv. *BarDistribution*.

BarDistribution

BarDistribution je po–částech konstantní pravděpodobnostní rozdělení. V článku (Müller et al., 2022) se mu říká *Riemann distribution*, jedná se o diskretizaci spojité distribuce. Vizuálně to vypadá takto: Avšak je důležité odlišit tento koncept od klasického histogramu. Histogram odhaduje hustotu z dat. Hlavním principem je, že počítáme klik naměřených hodnot padlo do každého binu, a to určuje výšku jednotlivých sloupců. *BarDistribution* je naopak výstup modelu, tj. my neděláme měření a nepočítáme, kolik hodnot padlo do toho či oného binu. Právě samotný model určuje výšku sloupců dle nějakých svých pravidel. Neuronová síť na výstupu vrátí n čísel, každé číslo říká, jak vysoký má být jeden sloupec. To jest distribuce určuje síť, to umožňuje pak modelovat pestrou škálu různých rozdělení a nejsme omezení jenom normálním, exponenciálním rozdělením apod. Binům se v *BarDistribution* říká *bucket*. V implementaci je *BarDistribution* definován pomocí hranic (parametr `borders`), tím definujeme interval, na kterém je tato funkce definována. Může to vypadat například takto:
`borders = [-3.0, -2.994, ..., 0.0, ..., 2.994, 3.0]` Mezi sousedními hranicemi leží jeden bucket. Bucket č. i pokrývá interval $[b_i, b_{i+1})$ a má šířku $w_i = b_{i+1} - b_i$. V naší konfiguraci jsou `borders` typicky rovnoměrně rozložené, takže všechny buckety mají stejnou šířku. Implementace ale podporuje i nerovnoměrné dělení (hustší buckety v zajímavých oblastech, řidší na krajích).

Vraťme se zpátky k dekodování. Jednotlivé prvky v schématu, který jsme uvedli na začátku sekce, plní následující funkci:

1. `Linear(512 -> 1024)`: Vezme 512-dimenzionální embedding a promítne ho do 1024 dimenzí. To je lineární transformace – násobení maticí vah $W_1 \in \mathbb{R}^{512 \times 1024}$.
2. `GELU()`: Aplikuje nelineární aktivaci. Bez ní by dvě lineární vrstvy za sebou byly ekvivalentní jedné lineární vrstvě (součin matic je matice). GELU umožňuje *decoderu* zachytit nelineární vztahy mezi embeddingem a výstupní distribucí.
3. `Linear(1024 -> numBars)`: Zúží z 1024 dimenzí na `numBars` (řekněme 1000). Výstupem je 1000 čísel, říkáme jim *logity*, jeden pro každý bucket.

Logit v tomto případě je jenom surové číslo z poslední lineární vrstvy našeho *dekóderu*. Může to být jakékoliv reálné číslo. Důležité ale je, že nemají žádné jednotky, nesčítají se na jedničku a ne této etapě nemají žádnou přímou pravděpodobnostní interpretaci. Logit je pouze signál pro určitý bucket, do kterého spadne, jak moc si model myslí, že skutečná hodnota padne do daného intervalu. Čím vyšší je logit, tím více si je model jistý. Ale pro lepší interpretovatelnost musí tyto hodnoty nejdříve projít softmax vrstvou. Uvažujeme-li, že máme 1000 bucketů, softmax převede 1000 logitů na 1000 pravděpodobností:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

Logicky pak součet všech pravděpodobností je přesně 1 a vyšší logit implikuje vyšší pravděpodobnost. Hustotu pravděpodobnosti potom definujeme jako $f_i = \frac{p_i}{w_i}$, kde w_i je šířka bucketu. Širší bucket musí mít nižší hustotu při stejné pravděpodobnosti, aby distribuce dávala smysl jako aproximace spojitého rozdělení. Při rovnoměrných bucketech je tento rozdíl jen konstantní škálování. Evidentně pak můžeme dopočítat i všechny potřebné deskriptivní charakteristiky jako střední hodnotu, rozptyl atd. Jak jsme již minili dříve, neklademe na `BarDistribution` žádná omezení co se týče tvaru výsledné distribuce. Avšak v kontextu GP inference výstup typicky vypadá přibližně jako gaussovský zvon. To proto, že GP posteriorní prediktivní distribuce je přesně normální rozdělení. Pokud PFN dobře aproximuje GP posterior, buckety budou mít tvar zvonu. Ale model to nikdo nenutí, naučil se to sám, protože loss funkce, se kterou byl model trénován, ho odměňuje za to, že jeho distribuce co nejlépe odpovídá skutečné posteriorní distribuci.

Dále poskytneme krátký komentář, proč během trénování PFN volíme pro loss funkci právě Negative Log-Likelihood (NLL) tvar. NLL je ztrátová funkce, která měří kvalitu predikované distribuce. Její výpočet pro jeden testovací bod shrnuje algoritmus 4.

Algoritmus 4 Výpočet NLL ztráty přes `BarDistribution` pro jeden testovací bod

- 1: Vyprodukovat modelem logity $z_1, \dots, z_{1000}$
 - 2: Převed' logity softmaxem na pravděpodobnosti $p_1, \dots, p_{1000}$
 - 3: Převed' pravděpodobnosti na hustoty $f_i = p_i/w_i$, kde w_i je šířka bucketu
 - 4: Najdi bucket k , do kterého padne skutečné y_{test}
 - 5: Spočítej loss pro tento bod: $\mathcal{L} = -\log(f_k) = -\log(p_k) + \log(w_k)$
-

Člen $\log(w_k)$ je konstanta (borders se nemění), takže gradient teče jen přes $-\log(p_k)$. Minimalizace NLL je ekvivalentní maximalizaci likelihood, to znamená, že model se učí přiřadit co nejvyšší hustotu tam, kde skutečné y leží.

Otázkou je, proč bychom nemohli volit např. tvar Mean Square Error (MSE) jako loss funkci. MSE loss by měřil jen $(y_{\text{pred}} - y_{\text{true}})^2$, tj. chybu bodového odhadu. NLL přes `BarDistribution` měří kvalitu celé distribuce. Model se učí nejen správný střed, ale i správnou šířku a tvar nejistoty. To je přesně to, co potřebujeme pro aproximaci GP posterioru, chceme celou PPD, ne pouze průměr. Navíc, jak se ukáže

dále (Nagler, 2023), minimalizace NLL vede k optimálnímu řešení, které je Bayesovská posteriorní prediktivní distribuce, tedy přesně to, co GP analyticky počítá.

Na konci sekce zkusíme shrnout kompletní postup. Na úplném začátku jsme modelu předložili sadu trénovacích a testovacích bodů, přitom ale model nedostal `y_test` a účelem bylo ho naučit tyto hodnoty predikovat. PFN postupně zpracovalo a přetransformovalo skrz svoje vrstvy `x_train`, `y_train` a `x_test`. To znamená, že model zpracoval mu předložená informace jako celek naraz. Během toho se PFN naučilo pravidlo, jak se chovají různé GP posteriory pro různé parametry. Na konci jsme dostali pravděpodobnostní rozdělení pomocí převodu na logity a použití `BarDistribution`. Výsledek jsme vždy porovnali se skutečným GP a jako metriku, jak moc dobře PFN odhaduje skutečnost, jsme použili NLL, který bychom chtěli minimalizovat. Po dostatečném tréninku jsme pak dostali model, který na základě předložených dat ze skutečného GP prioru, které nikdy neviděl, je schopen odhadnout, odkud pochází a jak vypadá funkce, ze které pochází.

2.4 Teoretické statistické vlastnosti PFN

V této sekci uvedeme dosud známé statistické vlastnosti PFN, které publikoval Nagler v článku (Nagler, 2023). V okamžiku, kdy již máme natrénovaný model, dává smysl prozkoumat jeho statistické vlastnosti a ptát se, proč a jak fungují jeho prediktivní a aproximační schopnosti. Muller ve své práci ukázal empirická pozorování, která ale nepodpořil teoretickým zdůvodněním. To se avšak podařilo Naglerovi, který dokázal vybudovat teoretický základ pro PFN a řádně odůvodnit, proč a jak architektura PFN dává dobré výsledky při Bayesovské inference. Nagler ve své práci používá určitý formalismus pro zavedení celého problému, na kterém pak buduje samotnou teorii. My tento formalismus v této práci zachováme.

2.4.1 Statistický model

Uvažujme klasifikační úlohu, která se skládá ze tříd $Y \in \mathcal{Y}$ a příznaků $X \in \mathcal{X} \subseteq \mathbb{R}^d$. Předpokládejme dále, že máme iid data $D_n = (Y_i, X_i)_{i=1}^n$ pocházející z nějakého teoretického, avšak neznámého rozdělení p_0 . Naším cílem je predikovat toto podmíněné rozdělení na základě dostupných dat:

$$p_0(y | x) = P(Y = y | X = x).$$

Z hlediska Bayesovského neparametrického odhadování můžeme na p_0 nahlížet jako na realizaci náhodného parametru $p \in \mathcal{P}$, kde $\mathcal{P}$ je prostor podmíněných pravděpodobnostních rozdělení nad $\mathcal{Y} \times \mathcal{X}$. Tento parametr má rozdělení Π , které budeme říkat prior. Prior bude vyjadřovat naše přesvědčení o tom, jaké modely p jsou pravděpodobnější pro daný problém ještě předtím než uvidíme data. Formálně celý proces, ze kterého pocházejí data lze zapsat takto:

1. Vygeneruj $p \sim \Pi$.
2. Vygeneruj iid vzorky $D_n = (Y_i, X_i)_{i=1}^n$ a jeden dodatečný pár (Y, X) z modelu p .

Tento mechanismus definuje sdílené rozdělení nad $(D_n \cup (Y, X), p)$ pro každé n . Skutečné rozdělení p_0 je ta realizace p , ze které skutečně pocházejí naše data, zpravidla je p_0 neznámá.

Pro každé n poskytuje výše popsany statistický model dobře definované sdružené rozdělení n -tice $(D_n \cup (Y, X), p)$. Na základě tohoto modelu lze $p_0(y | x)$ aproximovat pomocí posterior predictive distribution (PPD):

$$\pi(y | x, D_n) = P(Y = y | X = x, D_n),$$

kterou jsme již zmiňovali v předchozí sekci, uved' me její definici ještě jednou i zde a se zařazením do kontextu. Tím vzniká celá rodina PPD indexovaná n . Pokud prior Π faktorizuje nezávisle pro marginální složky $p(y | x)$ a $p(x)$, tj. rozdělení příznaků a rozdělení tříd jsou v prioru nezávislé, pak lze PPD dále zapsat jako:

$$\pi(y | x, D_n) = \int p(y | x) d\Pi(p | D_n), \quad (2.1)$$

kde $\Pi(\cdot | D_n)$ je tzv. *posterior*, což je podmíněné rozdělení p dané pozorovanými daty D_n , získaná Bayesovým pravidlem. Tento integrál říká v zásadě relativně jednoduchou věc: průměrujeme

$$p(y | x)$$

přes všechna možná p vážená jejich posteriorní pravděpodobnostmi, pak modely p , které jsou konzistentní s D_n dostanou větší váhu. Podmínka faktorizace prioru je spíše technická. Platí-li podmínka, pak když jsme viděli data D_n , říkají nám něco o $p(y | x)$, ale x -ová souřadnice testovacího bodu sama o sobě

nic neříká o $p(y | x)$. Pokud by totiž ta podmínka neplatila, potom by i testovací souřadnice x byla informativní pro $p(y | x)$, což pro náš případ nedává smysl. PPD $\pi(y | x, D_n)$ je tedy *posteriorní střední hodnota* podmíněných rozdělání $p(y | x)$, vážená tím, jak jsou jednotlivé modely p konzistentní s D_n .

PFN je numerická aproximace celé rodiny PPD $\{\pi(\cdot | \cdot, D_n), n \in \mathbb{N}\}$. Základním teoretickým poznatkem je, že PPD sama o sobě je pro každé n optimálním prediktorem v následujícím smyslu. Definiujme třídu všech podmíněných pravděpodobnostních funkcí takto:

$$\mathcal{Q} = \left\{ q : (\mathcal{Y} \times \mathcal{X})^{n+1} \rightarrow [0, 1] \mid \sum_{y \in \mathcal{Y}} q(y | \cdot, \cdot) = 1 \right\}.$$

Každé $q \in \mathcal{Q}$ je tedy funkce, která dostane n trénovacích párů (Y_i, X_i) a jeden testovací vstup $\mathbf{X}$, a vrátí distribuci přes $\mathcal{Y}$.

Věta 2.4.1. π z rovnice 2.1 splňuje:

$$\pi = \arg \max_{q \in \mathcal{Q}} \mathbb{E}_{\Pi}[\log q(Y | X, D_n)], \quad (2.2)$$

kde $\mathbb{E}_{\Pi}$ je střední hodnota přes $(Y, X) \cup D_n$ generované dle mechanismu v sekci 2.1.

Jinými slovy PPD π maximalizuje očekávanou podmíněnou log-likelihood přes všechny možné prediktory z $\mathcal{Q}$. To znamená, že to je nejlepší možný prediktor za daného prioru Π .

Dále navíc maximalizaci výrazu $\mathbb{E}_{\Pi}[\log q(Y | X, D_n)]$ lze ekvivalentně chápat jako minimalizaci očekávané KL divergence

$$\mathbb{E} \left[\text{KL} \left(q(\cdot | X, D_n) \parallel \pi(\cdot | X, D_n) \right) \right].$$

PPD π je tedy KL-optimálním prediktorem v $\mathcal{Q}$.

Abychom PPD aproximovali v praxi, natrénujeme model q_{θ} , kde $\theta \in \Theta$ jsou parametry (např. váhy neuronové sítě). To znamená, že pro každou hodnotu θ existuje celá rodina funkcí:

$$\{q_{\theta} : (\mathcal{Y} \times \mathcal{X})^{n+1} \rightarrow [0, 1], n \in \mathbb{N}\},$$

ale závislost na n v notaci explicitně neuvádíme. KL-optimální parametry jsou definovány jako:

$$\theta^* = \arg \max_{\theta \in \Theta} \mathbb{E}_{\Pi_N} \mathbb{E}_{\Pi}[\log q_{\theta}(Y | X, D_N)], \quad (2.3)$$

kde Π_N je tzv. *size prior*, tj. rozdělání nad velikostí trénovací množiny N . Střední hodnota přes N zajišťuje, že q_{θ^*} umí napodobit celou rodinu* PPD pro různá n , ne pouze její n -tý prvek. Pro model q_{θ} zpravidla neexistuje θ takové, že $q_{\theta} = \pi$ přesně. V tom případě rovnice 2.3 definuje *KL-optimální* aproximaci π ve třídě $\{q_{\theta} : \theta \in \Theta\}$, tj. nejlepší aproximaci, které je daná architektura schopna.

Dále lze ukázat následující teoretickou vlastnost. Střední hodnota v (3) se v praxi aproximuje průměrováním přes m iid datasetů, např. pomocí metody Monte Carlo. Konkrétně generujeme datasety $(Y_j, X_j) \cup D^{(j)}$ velikosti $N_j + 1$ s $N_j \sim \Pi_N$ dle postupu 2. Empirická verze optimalizačního problému pak je:

$$\hat{\theta} = \arg \max_{\theta \in \Theta} \sum_{j=1}^m \log q_{\theta}(Y_j | X_j, D^{(j)}). \quad (2.4)$$

Přesnost $\hat{\theta}$ jako aproximace θ^* závisí na počtu Monte Carlo vzorků m . Ale důležité je, že se dá ukázat $\hat{\theta} = \theta^* + O_p(m^{-1/2})$. Z toho plyne, že konvergence $\hat{\theta} \rightarrow \theta^*$ je v praxi zaručena.

Nagler (2023) ukazuje, že pokud je trénovací loss NLL (negative log-likelihood, viz Fáze 8) a model má dostatečnou kapacitu, optimální řešení konverguje k Bayesovské posteriorní prediktivní distribuci. To znamená, že správně natrénovaný PFN je teoreticky ekvivalentní analytickému GP posterioru.

2.4.2 Od PPD k PFN

V této sekci si ukážeme podrobněji jak spolu souvisí pojmy PPD a PFN, řekneme si zajímavé vlastnosti ohledně učení PPD, která spojují tyto dva nástroje, a jako důsledek popíšeme i vlastnosti učení samotného PFN. Uvažujme definici PPD 2.1. Posterior $\Pi(\cdot | D_n)$ je plně určen priorem Π . Jde o standardní Bayesovo pravidlo aplikované na nekonečnědimenzionální parametr p . Pokud jsou data D_n generována z p_0 , doufáme, že se posterior $\Pi(p | D_n)$ s rostoucím n soustředí okolo p_0 , a tím pádem $\pi(y | x, D_n) \rightarrow p_0(y | x)$. Avšak v tomto okamžiku čelíme velmi netriviálnímu problému – jak zvolit správný prior, aby to konvergovalo. Obecně existují celé články a knížky o tom, jak se vypořádat s tímto problémem, který se vyskytuje v praxi tu a tam. Platí totiž, že je třeba, aby Π měl dostatečně velký support a tím pokrýval dostatečně bohatou třídu funkcí. To nám pak totiž umožní efektivní inferenci. Např. pro případ GP s RBF kernelem bychom chtěli pokryt celý prostor všemožných tvarů té funkce, tj. aby model uměl najít správné hyperparametry v RBF kernelu. Tím pádem, když PPD dostane sadu bodů, dokáže nám říct, ze které konkrétní konfigurace GP s RBF kernelem pochází. Ale ani velký suport nemusí stačit. Prior může sice p_0 obsahovat ve svém supportu, ale dávat příliš velkou hmotnost nepříznivým oblastem prostoru $\mathcal{P}$, čímž může konvergence posteriorů probíhat velmi pomalu nebo vůbec. Avšak Nagler ve svém článku ukazuje, že PPD se umí učit i v situaci, kdy p_0 leží mimo support prioru $\mathcal{P} = \{p : \Pi(p) > 0\}$. To platí, za určitých podmínek.

Věta 2.4.2. Uvažujme prior Π . Necht' dále platí:

1. Existuje jedinečné $p^* \in \mathcal{P}$ takové, že

$$p^* = \arg \min_{p \in \mathcal{P}} \text{KL}(p | p_0), \quad \text{KL}(p^* | p_0) < \infty.$$

2. Pro každé $\alpha \in (0, 1/2)$ existují množiny $B_1, \dots, B_{J(\alpha)}$ takové, že

$$\mathcal{P} \subseteq \bigcup_{j=1}^{J(\alpha)} B_j, \quad \sup_{p, p' \in B_j} H(p, p') \leq 4(\alpha^2/2)^{1/\alpha}, \quad \sum_{j=1}^{J(\alpha)} \Pi(B_j)^\alpha < \infty,$$

kde $H(p, p')$ je Hellingerova vzdálenost mezi p a p' .

Pak existuje $p^* \in \mathcal{P}$ takové, že

$$\pi(y | x, D_n) \xrightarrow{n \rightarrow \infty} p^*(y | x) \quad \text{s. j.,}$$

pro P_0 s.v. (y, x) . Navíc p^* je KL-optimální aproximace p_0 v $\mathcal{P}$:

$$p^* = \arg \min_{p \in \mathcal{P}} \text{KL}(p | p_0).$$

První podmínka v této větě požaduje, aby v supportu prioru existovala jednoznačná KL-optimální aproximace p^* pravého modelu p_0 a KL divergence p^* vůči p_0 je konečná. Druhá podmínka je technickým požadavkem na to, aby prior nepřiděloval příliš velkou hmotnost oblastem daleko od p^* , prior nesmí být příliš rozptýlený v nežádoucích oblastech supportu. Sama o sobě věta tvrdí, jak se PPD učí. Konkrétně, že se učí tím lépe, čím víc dat máme, a snaží se naučit co nejlepší aproximaci p_0 dosažitelnou v $\mathcal{P}$. Pokud $p_0 \in \mathcal{P}$ (prior obsahuje pravý model): $p^* = p_0$ a PPD konverguje přímo k p_0 . Pokud $p_0 \notin \mathcal{P}$ (prior pravý model neobsahuje): PPD stále konverguje, ale k nejbližšímu $p^* \in \mathcal{P}$ v KL smyslu. Logicky čím větší je $\mathcal{P}$, tím víc se přibližuje p^* ke skutečnému p_0 .

Toto tvrzení nám tedy úplně řeší problém hledání správného prioru, na který jsme upozorňovali na začátku sekce. My totiž nepotřebujeme perfektní prior, nýbrž aby byl "dostatečně rozumný" ve smyslu předpokladů věty 2.4.2, a objem dat postupně ten náš odhad prioru opraví. Avšak PPD je ideální teoretický případ. Pokud bychom chtěli tyto vlastnosti použít např. u PFN, je zde několik nuancí. PFN je totiž pouze aproximací PPD, jak ukážeme dále, trénovaná na omezených datasetech, a proto tvrzení 2.4.2 nelze přímo aplikovat.

Podívejme se podrobněji na to, jak PFN aproximuje PPD. Již jsme si definovali vztah 2.4 a teď na něj budeme nahlížet jako na optimalizační problém. Na konečný výsledek $\hat{\theta}$ mají vliv čtyři faktory:

1. datový prior Π ,
2. prior velikostí vzorků Π_N ,
3. model q_θ ,
4. počet vzorků m .

Jelikož PFN je předtrénovaný model, třídu modelů $\{q_\theta : \theta \in \Theta\}$ lze považovat za fixní vůči m . Přesnost $\hat{\theta}$ jako aproximace θ^* pak plyne ze standardních výsledků empirické minimalizace vztahu: $\hat{\theta} = \theta^* + O_p(m^{-1/2})$. Tím máme hned vyřešený jeden z faktorů. Ostatní tři jsou mnohem zajímavější.

Pokud Π obsahuje pouze jednoduché modely, bude i optimální PFN q_{θ^*} produkovat jednoduché funkce (y, x) . Naopak, jednoduchá architektura $\{q_\theta : \theta \in \Theta\}$ nemůže těžit ze složitého prioru Π . Aby PFN dobře fungoval na různorodých úlohách, musí mít dostatečnou kapacitu jak architektura q_θ jako samotná, tak i prior Π .

Prior Π_N velikostí vzorků lze chápat jako regularizátor. Při tréninku (odkaz) se vzorkují datasety $D^{(j)}$ s náhodnou velikostí $N_j \sim \Pi_N$. Definujme KL-optimální parametr pro danou velikost vzorku:

$$\theta_n^* = \arg \max_{\theta} \mathbb{E}_{\Pi}[\log q_\theta(Y | X, D_n)].$$

PPD $\pi(y | x, D_n)$, kterou se snažíme aproximovat, se mění s n . Jako funkce od (y, x) roste její složitost s n . Pro $n = 1$ je PPD blízká průměrnému modelu v prioru a typicky téměř konstantní. Pro velká n je stále víc a víc komplexní. Analogicky θ_n^* preferuje složitější modely s rostoucím n . Optimalizací přes náhodné velikosti N_j výsledný θ^* průměruje $\theta_{N_j}^*$. Distribuce Π_N tak určuje, které velikosti datasetů jsou více signifikantní. To lze chápat jako regularizaci complexity modelu. Malé N_j nutí model k jednoduchým predikcím, velké N_j k složitým.

2.4.3 Učení PFN In-Context

V předchozí sekci jsme ukázali podstatnou vlastnost PPD, a to, že se se zvětšujícím se datasetem PPD přibližuje ke skutečné distribuci. Následně jsme ukázali, že PFN je pouhou aproximací PPD, jejíž kvalita závisí na určitých faktorech, a nelze s jistotou tvrdit, že pro něj platí stejné vlastnosti jako pro PPD a nelze na něj přímo aplikovat větu 2.4.2. Ukazuje se, že PFN se pořád může chovat jako PPD a zachovat její některé klíčové vlastnosti, jako např. již zmíněné tvrzení. Avšak v praxi to není tak jednoduché a jednoznačné. PFN se dokáže přiblížit k $p^*(y | x)$, a to i při relativně malém množství dat ($n \leq 1000$). Navíc se zachovává i to, že ani nemusíme pro tuto vlastnost najít perfektní prior. Přesto ale prozatím nemáme žádné teoretické důvody nebo předpoklady, aby toto byla pravda. V této sekci se pokusíme je odvodit.

V tomto okamžiku je výhodné přepnout se z bayesovského pohledu na frekventistický. Pro libovolnou velikost datasetu n při inferenci nahlížíme na předtrénovaný model $q_{\hat{\theta}}(y | x, \cdot)$ jako na nenatrénovaný frekventistický prediktor pro $p_0(y | x)$, což je funkce $(\mathcal{Y} \times \mathcal{X})^n \rightarrow \mathcal{P}_{\mathcal{Y}|\mathcal{X}}$, která mapuje dataset D_n na

podmíněnou distribuci. Parametry θ jsou pak jen hyperparametry tohoto prediktoru, vyladěné před tréninkem. Prior Π a Π_N jsou jednoduše distribuce nad úlohami, na kterých chceme, aby prediktor fungoval dobře. Dále rozepíšeme chybu predikce na bias a rozptyl:

$$q_\theta(y | x, D_n) - p_0(y | x) = \underbrace{q_\theta(y | x, D_n) - \mathbb{E}_{D_n \sim p_0^n}[q_\theta(y | x, D_n)]}_{\text{rozptyl}} + \underbrace{\mathbb{E}_{D_n \sim p_0^n}[q_\theta(y | x, D_n)] - p_0(y | x)}_{\text{bias}}. \quad (2.5)$$

Již jsme řekli, že empiricky chyba predikce klesá spolu se zvětšujícím se n . Takže zkusme najít které strukturální aspekty PFN by to mohli vysvětlit.

Obecně ze struktury transformeru plyne, že jsou invariantní vůči permutacím vstupních dat (tj. symetrické), což je mimochodem jednou z výhod pro architekturu PFN dataset D_n je ze své podstaty množina, ne sekvence, takže permutační invariance je přesně ta vlastnost, kterou chceme pro i.i.d. vzorky.

Věta 2.4.3. Necht' $f : (\mathcal{Y} \times \mathcal{X})^n \rightarrow \mathcal{P}_{\mathcal{Y}|\mathcal{X}}$ je libovolný prediktor. Pak existuje jeho symetrizovaná verze $\tilde{f}$ taková, že pro každé pravděpodobnostní míry P :

$$\mathbb{E}_{D_n \sim P^n}[\tilde{f}(D_n)] = \mathbb{E}_{D_n \sim P^n}[f(D_n)], \quad \text{Var}_{D_n \sim P^n}[\tilde{f}(D_n)] \leq \text{Var}_{D_n \sim P^n}[f(D_n)]. \quad (2.6)$$

Tato věta říká, že takové symetrické prediktory jsou optimální z hlediska MSE. Toto je první opodstatnění faktu, proč chyba predikce pro PFN stále klesá.

Jsou tady i další strukturální vlastnosti prediktoru q_θ , které mají vliv na kvalitu predikce. Platí, že s větším D_n klesá vliv jednotlivých vzorků na výslednou predikci. Formálně předpokládáme, že existují $\alpha > 0$ a $L < \infty$ takové, že pro dostatečně velká n a s. v. datasety D_n, D'_n lišící se v jediném vzorku:

$$|q_\theta(y | x, D_n) - q_\theta(y | x, D'_n)| \leq Ln^{-\alpha}. \quad (2.7)$$

To nám umožňuje vyslovit tvrzení, které ukazuje, proč rozptyl během učení postupně mizí.

Věta 2.4.4. Necht' platí předpoklad 2.7, pak

$$|q_\theta(y | x, D_n) - \mathbb{E}[q_\theta(y | x, D_n)]| \lesssim n^{1/2-\alpha} \quad (2.8)$$

s vysokou pravděpodobností. Navíc pro $\alpha > 1/2$ platí $\lim_{n \rightarrow \infty} n^{1/2-\alpha} = 0$ a

$$q_\theta(y | x, D_n) - \mathbb{E}[q_\theta(y | x, D_n)] \xrightarrow{n \rightarrow \infty} 0 \quad \text{almost surely}. \quad (2.9)$$

Jinými slovy rozptyl postupně mizí s počtem dat n . Tak dostáváme, že hlavním pramenem chyby při inferenci není bias, kterého se už jen tak nezbavíme.

Ze vztahu 2.5 je vidět, že bias je určen chováním posloupností $\mathbb{E}[q_\theta(y | x, D_n)]$. Je logické, že bychom mohli předpokládat (nebo očekávat), že platí i tento vztah:

$$\mathbb{E}[q_\theta(y | x, D_n)] \xrightarrow{n \rightarrow \infty} \bar{q}_\theta(y | x),$$

kde $\bar{q}_\theta(\cdot | \cdot)$ je jakýsi zprůměrovaný prediktor, jehož konkrétní architekturu neznáme. Ale co můžeme říct s jistotou, tak že chování biasu hodně závisí na oné architektuře, jak ukazuje Nagler ve svém článku (Nagler, 2023), může růst, klesat, anebo být celou dobu konstantní. Co je avšak v našich silách – vyslovit nutnou podmínku pro mizení biasu. Má-li prediktor $q_\theta(\cdot | \cdot, D_n)$ mizející bias na dostatečně bohaté třídě funkcí, pak nutně musí být lokální. Lokální zde znamená, že např. k $q_\theta(y | x, D_n)$ by měly přispívat (asymptoticky) pouze vzorky $(Y_i, X_i) \in D_n$ s X_i blízko x . Tyto poznatky dává dohromady následující věta.

Věta 2.4.5. Necht' $\mathcal{P}$ je třída distribucí taková, že pro každé $p \in \mathcal{P}$:

$$\mathbb{E}_{D_n \sim p^n} [q_\theta(y \mid x, D_n)] \xrightarrow{n \rightarrow \infty} p(y \mid x). \quad (2.10)$$

Pokud platí 2.7, pak existuje posloupnost $\epsilon_n \rightarrow 0$ taková, že pro každé $\tilde{p} \in \mathcal{P}$ platí:

$$|q_\theta(y \mid x, D_n) - q_\theta(y \mid x, \tilde{D}_n)| \xrightarrow{n \rightarrow \infty} 0, \text{ s. j.} \quad (2.11)$$

kde $D_n = (Y_i, X_i)_{i=1}^n$ a $\tilde{D}_n = (Y'_i, X_i)_{i=1}^n$ s

$$Y'_i = \begin{cases} Y_i & \text{pokud } \|X_i - x\| \leq \epsilon_n, \\ \sim \tilde{p}(\cdot \mid X_i) & \text{pokud } \|X_i - x\| > \epsilon_n. \end{cases}$$

To znamená, že množina $\tilde{D}_n$ je poskládaná tak, že místo vzdálených od x bodů obsahuje náhodný šum, ale podstatné je, že nakonec ten šum (odlehle body) vůbec nemají vliv na výslednou predikci. To ovšem platí pro dostatečně bohatou množinu $\mathcal{P}$. Výsledek postrádá smysl, je-li $\mathcal{P}$ příliš malá. I pro bohatou $\mathcal{P}$ jde jen o nutnou, nikoliv postačující podmínku, konstantní prediktor $q_\theta = 1$ je lokální, ale jeho bias se s n nemění. Důležitý je ale důsledek pro bias-variance tradeoff: pokud bias mizí pro bohatou $\mathcal{P}$, prediktor může efektivně využít jen $\epsilon_n n = o(n)$ vzorků, takže podmínka 2.7 pravděpodobně neplatí pro $\alpha = 1$. Dále se ukáže, že lokalizace je kamenem úrazu pro transformer architekturu PFN. *Attention váhy* $a_j^{(h)}$ jsou výstupem SoftMax, tedy jsou vždy kladné pro všechny vzorky j , nikdy přesně nulové. Transformer tedy vždy váží nenulově i vzdálené vzorky. Přepis příznaků vzdálených bodů proto vždy predikci ovlivní, a lokalita ve smyslu věty 2.4.5 není splněna. Transformer tak garantuje mizení rozptylu, ale nikoli mizení biasu. Tato pozorování popíšeme podrobněji dále.

2.4.4 PFN transformer

Jelikož tato práce je věnovaná pohledu na PFN skrz architekturu Transformer (Vaswani et al., 2017), dává smysl podívat se na statistické vlastnosti právě této architektury. Uvažujme tedy opět pro jednodušost transformer s jednou vrstvou. Dataset $D_n = \{V_i\}_{i=1}^n$, kde $V_i = (Y_i, X_i) \in \{0, 1\} \times \mathbb{R}^d$, a testovací vektor $v = (0, x)$. Sít' je definována následujícími operacemi:

$$a_j^{(h)} = \text{SoftMax}\left(v^\top W_q^{(h)} V_1, \dots, v^\top W_q^{(h)} V_n\right)_j,$$

$$u' = \sum_{h=1}^H \sum_{j=1}^n a_j^{(h)} W_v^{(h)} V_j,$$

$$u = \text{LayerNorm}(v + u'; \gamma),$$

$$z' = W_{r,2} \text{ReLU}(W_{r,1} u; \gamma),$$

$$z = \text{LayerNorm}(u + z'; \gamma),$$

$$q_\theta(\cdot \mid x, D_n) = \text{SoftMax}(W_o z),$$

kde matice vah jsou $W_q^{(h)}, W_v^{(h)} \in \mathbb{R}^{(d+1) \times (d+1)}$, $W_{r,1}, W_{r,2}^\top \in \mathbb{R}^{m \times (d+1)}$, $W_o \in \mathbb{R}^{|\mathcal{Y}| \times (d+1)}$. V tomto případě parametr θ , který jsme všude uváděli v kontextu PFN v předchozích sekcích, sbírá všechny tyto matice.

To znamená, že $\theta = \{W_q^{(1)}, \dots, W_q^{(H)}, W_v^{(1)}, \dots, W_v^{(H)}, W_{r,1}, W_{r,2}, W_o, \gamma\}$ Operace SoftMax, LayerNorm a ReLU jsou definovány jako:

$$\text{SoftMax}(v) = \frac{\exp(v)}{\sum_j \exp(v_j)}, \quad \text{LayerNorm}(v; \gamma) = \gamma_1 \frac{v - \text{avg}(v)}{\|v - \text{avg}(v)\|_2 + |\gamma_2|} + \gamma_3, \quad \text{ReLU}(v) = \max(0, v),$$

kde $\exp$ a $\max$ působí po složkách. První dvě rovnice definují *attention mechanismus* s H hlavami (Vaswani et al., 2017). Ze součtu $a_1^{(h)} + \dots + a_n^{(h)} = 1$ plyne, že *attention váhy* tvoří pravděpodobnostní distribuci přes trénovací vzorky. Myšlenka je, že v každé hlavě *attention váhy* $a_j^{(h)}$ zdůrazňují konkrétní vzorky $V_j \in D_n$, konkrétně ty, které jsou "podobné" testovacímu vektoru v ve smyslu měřeném skalárním součinem $v^\top W_q^{(h)} V_j$. Každá *attention hlava* dovoluje jinou definici podobnosti prostřednictvím matice $W_q^{(h)}$. Intuitivně to funguje takto: transformer si při predikci pro x vyhledá v trénovacích datech vzorky, které považuje za relevantní, a jejich labely agreguje. Různé hlavy mohou sledovat různé aspekty podobnosti, jedna hlava třeba srovnává hodnoty X_i s x , jiná sleduje jiný rys vstupních vektorů.

Nás by také zajímalo, jaké vlastnosti má rozptyl a bias pro tuto architekturu, protože v předchozí kapitole jsme poznamenávali, že konkrétní vlastnosti těchto charakteristik se mění v závislosti na volbě konkrétní architektury.

Věta 2.4.6. Pro $\mathcal{X} = \{x : \|x\|_2 \leq K\}$ a omezené matice vah ($\|W_q^{(h)}\|_2, \|W_v^{(h)}\|_2, \|W_{r,1}\|_2, \|W_{r,2}\|_2, \|W_o\|_2 < \infty$) platí

$$|q_\theta(y | x, D_n) - \mathbb{E}[q_\theta(y | x, D_n)]| \lesssim n^{-1/2} \quad (2.12)$$

s vysokou pravděpodobností.

Velmi podobnou větu jsme již viděli v předchozí sekci. Dokonce tvrdí skoro stejné: rozptyl mizí nezávisle na parametru θ . Jde o strukturální vlastnost architektury, nikoliv výsledek tréninku. Důvodem je, že *attention mechanismus* nutně přiděluje každému vzorku váhu řádu $1/n$, takže vliv libovolného jednotlivého vzorku klesá s n . V případě biasu jsme již říkali, že ta situace je horší. Bias v případě transformeru totiž silně závisí na volbě θ . Označme $\bar{q}_\theta(y | x)$ limitní hodnotu predikce pro $n \rightarrow \infty$.

Věta 2.4.7. Za předpokladů platnosti věty 2.4.6 plyne

$$\mathbb{E}[q_\theta(\cdot | x, D_n)] \xrightarrow{n \rightarrow \infty} \bar{q}_\theta(\cdot | x), \quad (2.13)$$

kde $\bar{q}_\theta(\cdot | x)$ je definováno stejně jako $q_\theta(\cdot | x, D_n)$, ale s u' nahrazeným výrazem $\bar{u}' = \sum_{h=1}^H W_v^{(h)} \mathbb{E}_{V \sim g_h}[V]$,

kde $g_h(s) = \frac{\exp(v^\top W_q^{(h)} s)}{\mathbb{E}_{V \sim p_0}[\exp(v^\top W_q^{(h)} V)]} \cdot p_0(s)$.

To znamená, že v definici transformeru výše jsme nahradili výraz u' tím novým $\hat{u}'$. Navíc v definici $\hat{u}'$ se objevuje tzv. *exponentially tilted function* (česky exponenciálně vychýlená funkce). V tomto kontextu se zavádí jako asymptotická variace *attention mechanismu*. Tato funkce $g_h(s)$ zvyšuje pravděpodobnost těm hodnotám s , které se nejvíce podobají vektoru v ve smyslu $v^\top W_q^{(h)} s$, a snižuje pro zbylé body. Jinými slovy ten složitý vzoreček na obrázku ukazuje teoretickou situaci, co by se stalo, kdyby měl Transformer k dispozici nekonečně mnoho vzorků. Už nerozděluje váhy mezi jednotlivými datovými body, ale vezme celou spojitou distribuci dat a "vychýlí" ji celou směrem k té informaci, na kterou zrovna potřebuje upřít pozornost. Více formálně to znamená, že každá *attention hlava* h tak hodnotí určitý aspekt neznámé distribuce p_0 (charakterizovaný maticí $W_q^{(h)}$). Pokud jsou matice $(W_q^{(h)})_{h=1}^H$ dobře nastaveny, tyto jednotlivé pohledy rozlišují různé hodnoty příznaků.

Avšak g_h lokalizuje prediktor pouze do jisté míry. Ačkoli zvyšuje váhu vzorků podobných v , ale ne ve smyslu věty 2.4.5. Vzorky vzdálené od x stále nenulově přispívají, protože SoftMax nikdy nevytváří

přesně nulové váhy. Přepsání příznaků vzdálených vzorků proto vždy predikci změní, a tak lokalita podle 2.4.5 není splněna. Proto plyne, že bias transformeru obecně nemůže úplně zmizet. Limitní bias $\bar{q}_\theta(y | x) - p_0(y | x)$ přesto může být malý, pokud síť za *attention vrstvou* dokáže ze součtu aspektových souhrnů $W_v^{(h)} \mathbb{E}_{V \sim g_n}[V]$ zkonstruovat dobrou aproximaci p_0 . Relevance jednotlivých aspektů závisí na pravé distribuci p_0 . Méně relevantní aspekty mohou přispívat méně. Na malých vzorcích ($n = 1$) jsou všechny *attention váhy* $a_j^{(h)} = 1$, takže všechny aspekty přispívají stejnou měrou. To naznačuje, že bias může klesat v rozsahu velikostí, na které byl θ natrénován, nikoliv však za jejich hranicí, kde není důvod očekávat pokles biasu. Klíčovou roli hraje přítomnost více *attention hlav* ($H > 1$), spolupráce více hlav může efektivně napodobit mechanismus průměrování modelů. Dalším klíčovým nástrojem je větší počet *attention vrstev*. Navíc empirické testy ukazují, že na výslednou kvalitu predikce (a zmenšování biasu) hraje neposlední roli samotný trénink a jeho nastavení. Všechny tyto manipulační součástky tak dávají velký prostor pro manévr a různé jejich kombinace či nastavení mohou vést k velmi signifikantnímu poklesu biasu.

Dále lze různě upravovat strukturu PFN během inference či tréninku pro zlepšení lokality. Naším účelem je tedy nějakým způsobem donutit transformer se dívat na jednotlivé body co nejvíc lokálně a neuvažovat pro bod x příliš vzdálené hodnoty. Jedním z přímočarých způsobů je *post-hoc lokalizace*, kterou navrhuje Nagler. Pro predikci labelu v testovacím bodě x :

1. Sestrojí se redukovaná trénovací množina $D_n(x)$ odstraněním všech vzorků kromě k_n nejbližších sousedů bodu x z D_n .
2. Predikuje se label maximalizující $q_\theta(\cdot | x, D_n(x))$.

Omezení na okolí x je ekvivalentní "roztažení" cílové funkce $p(y | \cdot)$ v okolí x , lokálně hladká funkce se chová přibližně jako konstantní. Konstantní funkce jsou pro q_θ snazší k aproximaci. Lokalizace tedy zlepšuje bias za cenu mírného zvýšení rozptylu, prediktor pracuje s menší efektivní velikostí datasetu $k_n \ll n$.

Kapitola 3

Experimentální průzkum vlastností PFN

V této kapitole provedeme řadu experimentu, ve kterých budeme zkoumat a ověřovat různé hypotézy o tom, jak funguje PFN. Ačkoli Nagler se snaží ve své práci matematicky popsat principy PFN a naznačit jeho principy fungování, ale uvažuje model s jednou vrstvou a navíc popisuje, co PFN transformer ve výsledku vůbec dělá a jak fungují jeho vnitřní mechanismy, jenom částečně. Proto se pokusíme tyto otevřené otázky probrat jestli ne skrz matematický aparát, tak alespoň pomocí empirických pozorování. Všechny experimenty budou provdny jenom pro architekturu typu transformer. Bude nás většinou zajímat jak vnitřní struktura PFN, tak i jeho některé vlastnosti jakožto nástroje pro Bayesovskou inferenci. Rovněž prozkoumáme mechanismy pomocí kterých PFN vyhodnocuje vstupní data a čím se tyto mechanismy liší od regrese pomocí Gaussovských procesů.

3.1 Vnitřní struktura PFN transformeru a jeho základní principy

Nejdříve uvedeme základní analýzu PFN transformeru. Zaměříme se zde na to, co přesně dělá se vstupními daty, podíváme se, zde se snaží nakopírovat stejné mechanismy jako GP, anebo zda dělá něco jiného. Experimenty jsme prováděli na dvou typech modelů. Oba sdílí identickou architekturu transformeru: 6 vrstev, 8 *attention hlav*, dimenze embeddingu 512, dimenze skrytých vrstev dopředné sítě 1024 a výstupní FullSupportBarDistribution s 1000 biny, celkem 14,14 milionu parametrů. Architektura je nastavena s parametry `features_per_group=1` a `attention_between_features=False`, tedy přesně tak, jak jsme ji popsali v předchozí kapitole pro případ 1D regrese. Oba modely byly trénovány na syntetických datech vygenerovaných z Gaussovského procesu s RBF kernelem, vstupní prostor $x \in [0, 1]$. Liší se však tím, z jakého prioru nad hyperparametry trénovací data pocházela, a tedy i délkou tréninku, neboť pro nefixní hyperparametry model potřebuje více času na učení.

Model se fixními hyperparametry:

Tento model byl trénován na GP s pevně nastavenými hyperparametry RBF kernelu: $\ell = 0,3$, `outputscale = 1,0`, $\sigma^2 = 10^{-4}$. Model se tedy učil aproximovat posterior jednoho konkrétního GP a trénink konvergoval za 100 epoch.

Model s náhodně vzorkovanými hyperparametry:

Model byl trénován na GP, jehož hyperparametry byly při každé dávce vzorkovány nezávisle z předem dané distribuce: $\ell \sim \mathcal{U}(0,05, 1,0)$, `outputscale` $\sim \text{LogNormal}(0, 0,5)$, $\sigma^2 \sim 10^{\mathcal{U}(-3, -1)}$. Tento model se tedy musí naučit generalizovat přes celou rodinu GP funkcí s různými korelačními délkami a amplitudami – jde o plnohodnotný meta-learning nad distribucí hyperparametrů. Právě proto vyžaduje podstatně delší trénink: model viděl data z nespočetně různých tvarů funkcí a musel se naučit rozpoznávat hyperparametry přímo z kontextových dat. Trénink trval 500 epoch.

Výsledky obou modelů porovnáváme v průběhu jednotlivých experimentů s referenčním analytickým GP posteriorem, který pro dané hyperparametry a kontextová data vrací přesné řešení. Začneme ovšem s případem fixních hyperparametrů. Pro model to je značné zjednodušení, přesně ví, co se má naučit, a jak se ukáže dále, velmi efektivně. Na tomto jednoduchém příkladě ukážeme hlavní principy inference této metody, abychom následně mohli přejít na těžší příklady. To znamená, že model s s náhodně vzorkovanými hyperparametry budeme zkoumat v další sekci.

3.1.1 Struktura attention matic

Začneme však s tím, že vykreslíme si jednotlivé vrstvy a podíváme se na vnitřnosti transformeru a jak se jeho vrstvami prochází tok dat. *Attention matic* je přirozené okno do toho, jak transformer interně organizuje informaci. Připomínáme, že v naší konfiguraci má model 6 vrstev a v každé vrstvě 8 *attention hlav*, přičemž každá hlava může sledovat jiný aspekt vstupních dat. Vizualizaci provedeme pro 100-epoch model na sekvenci délky 120 bodů: 20 trénovacích bodů a 100 testovacích bodů. Výsledkem je *attention matic* tvaru 120×120 , přičemž každý prvek a_{ij} říká, jak moc se bod i (jako *query*) dívá na bod j (jako *key*). Každá vrstva přitom neobsahuje jen jednu takovou matici, ale 8, tj. jednu za každou *attention hlavu*, protože každá hlava má vlastní naučené váhy. Abychom nemuseli sledovat všech 8 matic zvlášť (pro 6 vrstev bychom obdrželi 48 matic), sečteme je po prvcích a vydělíme osmi, čímž z nich uděláme jedinou *průměrnou matici* pro každou vrstvu. Prvek a_{ij} v ní pak vyjadřuje, jak moc se bod i dívá na bod j v průměru přes všech 8 hlav.

Protože víme, které pozice odpovídají trénovacím bodům (0–19) a které testovacím (20–119), lze matici přirozeně rozdělit na čtyři kvadranty:

	Key: Train	Key: Test
Query: Train	Train→Train	Train→Test
Query: Test	Test→Train	Test→Test

Kvadrant *Test→Train* je pro nás klíčový, ten nám totiž říká, jak moc testovací body „čerpají“ informaci z trénovacích dat při budování své predikce. Z pohledu GP inference bychom očekávali, že právě tento kvadrant bude dominantní – predikce v novém bodě x^* závisí na pozorovaných párech (X, y) , nikoli naopak.

Na obrázku 3.1 je vidět, jak se struktura *attention matic* vyvíjí přes vrstvy:

Vrstva 0:

vykazuje relativně rovnoměrně rozložené váhy. Žádný kvadrant výrazně nedominuje a váhy jsou rozloženy plošně přes všechny pozice. Tuto vrstvu můžeme interpretovat jako *explorativní* fázi, tzn. model sbírá globální informaci o rozložení všech bodů v sekvenci a podrobněji jejich vzájemné vazby nezkoumá.

Vrstvy 1–4:

attention postupně řídne, váhy se soustředí na méně pozic a zbytek klesá k nule. Lze tady pozorovat tvorbu určitých struktur ve vzorech: kvadrant *Test→Train* začíná nabývat na intenzitě, zatímco *Train→Test* slábne. Tyto prostřední vrstvy zřejmě slouží k internímu zpracování informace, tj. postupnému sestavování reprezentace, ze které bude moci poslední vrstva provést samotnou inferenci.

Vrstva 5:

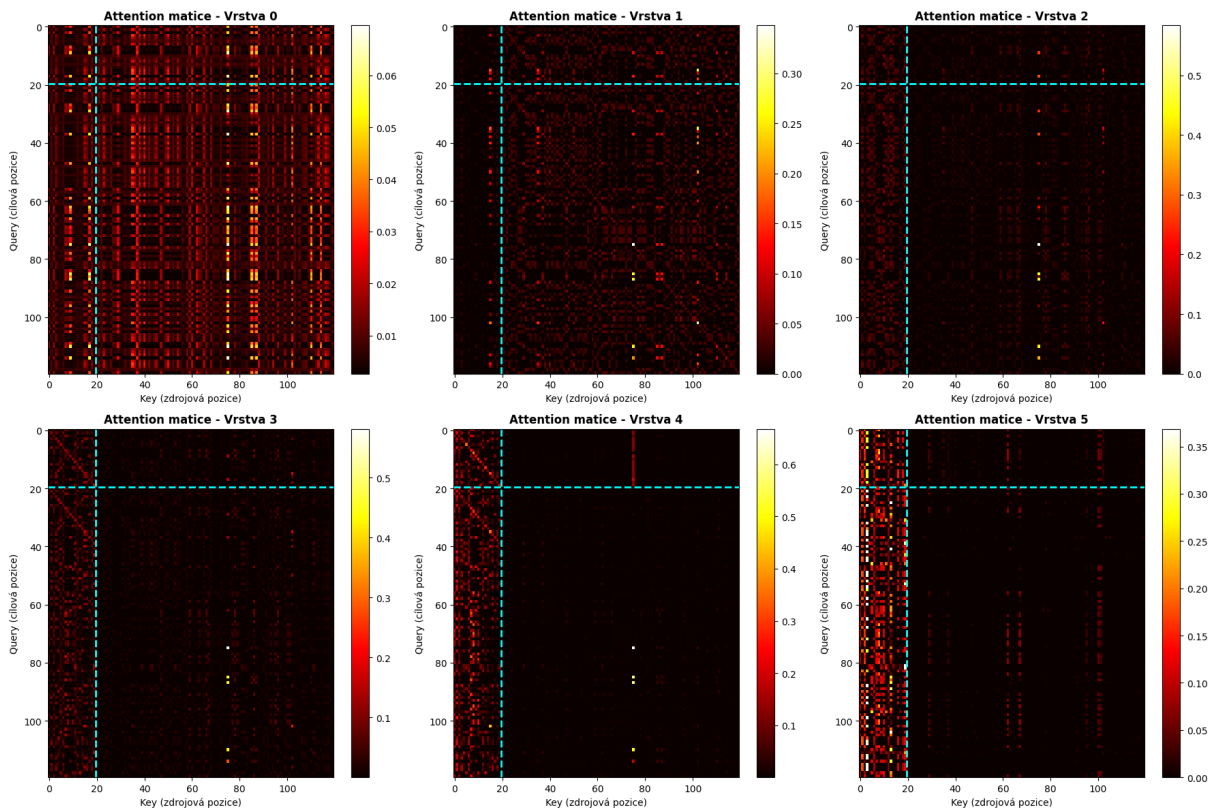
tato poslední vrstva již vykazuje velmi zřetelný vzor. Kvadrant *Test→Train* je jasně dominantní a strukturovaný, zatímco kvadrant *Train→Test* je skoro nulový. Tato asymetrie je přesně to, co bychom čekali od GP inference: testovací body jsou podmíněny trénovacími daty, nikoli naopak.

Podstatné je, že tuto kauzální asymetrii model neměl explicitně zakódovanou a tudíž vyplynula přirozeně z optimalizace na syntetických GP datech. Kdybychom totiž model trénovali na datech bez této

struktury, *attention mechanismus* by zůstal symetrický. Samotná existence asymetrie v poslední vrstvě je tak prvním experimentálním důkazem, že PFN provádí Bayesovskou inferenci a nikoliv pouhý *pattern matching*, neboli učení příkladu nazpaměť.

Na maticích na obrázku 3.1 si lze všimnout ještě jednoho nápadného jevu, konkrétně svislých pruhů. Některé *key* pozice (sloupce) dostávají vysokou *attention* od téměř všech *query* pozic, nezávisle na tom, kde daný *query* bod leží. Ve vrstvě 0 jsou nejostřejší, v hlubších vrstvách obvykle přetrvává jeden reziduální (v naší realizaci nápadně kolem *key* pozice 75). Jde o tzv. *attention sink*, mechanismus dobře popsany u velkých jazykových modelů (Xiao et al., 2023). Vzniká ze dvou důvodů. Za prvé, softmax nutí každý řádek matice sečíst na jedničku – každý *query* bod tedy *musí* svou *attention* někam rozdělit, i když pro něj žádný klíč není opravdu relevantní (typicky v raných vrstvách, než se vybudují informativní reprezentace, nebo u bodů daleko od dat). Přbytek *attention* se pak „vylije“ na několik stále stejných pozic, které tak vytvoří svislé pruhy. Za druhé, body s netypickou hodnotou (extrémní Y_j nebo okrajové X_j) dostanou po zakódování větší embedding, a tím i vyšší skóre od všech ostatních bodů – přitahují proto *attention* bez ohledu na to, odkud se *query* dívá.

Pro naši otázku je podstatné, že tyto pruhy jsou *query-nezávislé*: Nadaraya-Watsonův ani RBF estimator by je nikdy nevytvořil, protože jejich váhy jsou lokální a závisejí na poloze query bodu x^* . Svislý pruh naopak znamená „všechny *query* se dívají na tentýž bod bez ohledu na svou polohu“, část *attention rozpočtu* tedy jde na *normalizační sink*, nikoli na kernelové porovnávání. Tímto se dá říct, že existence pruhů je tak dalším nepřímým dokladem, že PFN *attention* není prostý kernel smoothing.



Obrázek 3.1: *Attention matice* pro všech 6 vrstev 100-epoch modelu (průměr přes 8 hlav) pro jednu konkrétní GP realizaci. Osa x : *key* pozice (na které body se díváme), osa y : *query* pozice (které body se dívají). Cyánová čára odděluje trénovací (0–19) a testovací (20–119) pozice. Světlejší barva odpovídá vyšší *attention váze*. Přesné hodnoty vah závisí na konkrétních vstupních datech, kvalitativní vzor (vrstva 0 distribuovaná → vrstva 5 s dominantním *Test*→*Train* blokem) je však stabilní.

3.1.2 Matematický rámec pro srovnávací experimenty

Než přistoupíme k samotným experimentům, je užitečné shrnout matematický aparát, který budeme v dalších sekcích potřebovat. Jde konkrétně o tři formule, jejichž vzájemný vztah je jádrem naší analýzy.

Posteriorní střední hodnota GP:

Po pozorování kontextových párů (X, y) je posteriorní střední hodnota Gaussovského procesu v novém bodě x^* dána vztahem:

$$\mu(x^*) = k(x^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}]^{-1} \mathbf{y},$$

kde $\mathbf{K}(\mathbf{X}, \mathbf{X})_{ij} = k(x_i, x_j)$ je kernelová matice a σ^2 je rozptyl šumu. Klíčovou roli hraje inverze $[\mathbf{K} + \sigma^2 \mathbf{I}]^{-1}$, která *dekoreluje* blízké trénovací body. Body, které si jsou podobné, si vzájemně „konkurují“ a jejich kumulativní vliv je potlačen.

Nadaraya-Watsonův estimátor:

Nejjednodušší alternativou, jak odhadnout GP, je Nadaraya-Watsonův (NW) estimátor, tj. normalizovaný váhovaný průměr trénovacích hodnot:

$$\hat{y}_{\text{NW}}(x^*) = \frac{\sum_i k(x^*, x_i) y_i}{\sum_i k(x^*, x_i)}.$$

NW se od GP liší právě absencí inverze kernelové matice. Tento model je až moc velkým zjednodušením, pokud bychom ho chtěli použít pro odhad GP. Ten implicitně předpokládá, že trénovací body jsou vzájemně nekorelované. Blízké body proto dostávají neúměrně velkou kumulativní váhu a predikce jsou pak příliš hladké. Současně je ale velmi přirozenou alternativou, jak hledat GP. Právě proto dále budeme zkoumat, zda se transformer nenaučil jenom tuto jednoduchou cestu.

Attention mechanismus transformeru:

Tento mechanismus jsme již zmiňovali v 2.3, kde jsme popisovali vnitřní strukturu PFN transformeru. *Attention* v PFN počítá pro každý testovací bod x^* váhovanou kombinaci hodnot z trénovacích bodů:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad \mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V.$$

Výsledná kernelová matice *závisí na datech*:

$$\text{Kernel}_{\text{attn}}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}}\right).$$

Na rozdíl od fixního RBF kernelu se tato matice plně adaptuje na konkrétní vstupní sekvenci. Matice vah $\mathbf{W}_Q, \mathbf{W}_K$ jsou naučeny z dat a mohou kódovat libovolnou strategii podobnosti.

Klíčová otázka, kterou budeme v následujících experimentech testovat, pak zní: *odpovídají attention váhy PFN normalizovaným RBF vahám, nebo se model naučil odlišnou strategii?* A pokud odlišnou, jak moc se jeho výstup blíží skutečnému GP posterioru oproti triviálnímu NW estimátoru?

3.1.3 Dělá PFN něco podobného jako GP s RBF kernelem?

V této sekci se zaměříme, zda se PFN transformer snaží napodobit chování RBF kernelu, a proto ho tak dobře aproximuje, anebo dělá něco jiného. Konkrétně pro každý testovací bod x^* porovnáme *attention váhy* a_i z poslední vrstvy (průměr přes 8 hlav) s normalizovanými RBF vahami:

$$w_i = \frac{k(x^*, x_i)}{\sum_j k(x^*, x_j)}.$$

Experiment provedeme pro fixní model na $n_{\text{ctx}} = 20$ trénovacích bodech a 100 testovacích bodech, průměrem přes 5 nezávislých *seedů*.

Každá metrika je průměrována přes 5 nezávislých *seedů*, přičemž pro každý *seed* je vygenerována jedna GP sekvence délky 100 (prvních 20 bodů jako kontext, všech 100 jako testovací body). MSE je počítáno jako střední kvadratická odchylka mezi střední hodnotou predikce PFN a analytickým GP posteriorem přes všech 100 testovacích bodů. Korelace *attention* vs. RBF je pro každý testovací bod x^* spočítána jako Pearsonova korelace mezi *attention* vahami poslední vrstvy (průměr přes 8 hlav) a normalizovanými RBF vahami $\exp(-d^2/2\ell^2)$, zprůměrovaná přes všechny testovací body. Jako referenční baseline zahrnujeme rovněž Nadaraya-Watsonův estimátor: MSE(NW, GP) měří jeho odchylku od analytického GP posterioru a slouží jako dolní mez kvality nejjednoduššího *kernel smoothingu*. MSE(PFN, NW) pak kvantifikuje, o kolik se predikce PFN od NW liší, pokud $\text{MSE(PFN, NW)} \approx \text{MSE(NW, GP)}$, znamená to, že PFN a NW predikují zcela odlišné hodnoty, a tedy že PFN neprovádí prostý *kernel smoothing*. Naměřené hodnoty jsou shrnuty v tabulce 3.1.

Metrika	100-epoch model
Průměrná korelace Attn vs RBF	$0,374 \pm 0,270$
MSE(PFN, GP)	$0,000167 \pm 0,000096$
MSE(NW, GP)	$0,2099 \pm 0,1462$
MSE(PFN, NW)	$0,2092 \pm 0,1439$
Poměr MSE(NW)/MSE(PFN)	$\sim 1\,256\times$

Tabulka 3.1: Výsledky korelační analýzy *attention* vah vs. RBF kernelu a srovnání přesnosti predikcí. Průměr přes 5 *seedů*.

Z tabulky 3.1 vyplývají dvě zásadní pozorování. Za prvé, korelace *attention* vah s RBF kernelem je nízká ($0,374 \pm 0,270$) a s vysokou variabilitou, což naznačuje, že model *neprovádí* prostý RBF *kernel smoothing*. Také je zjevné, že *attention mechanismus* se naučil jinou strategii. Jak je vidět na obrázku 3.2a, *attention váhy* jsou výrazně ostřejší než RBF, soustředí ují téměř veškerou váhu na nejbližší bod, zatímco RBF ji distribuuje plynule.

Za druhé, $\text{MSE(PFN, NW)} \approx \text{MSE(NW, GP)} \approx 0,21$, vzdálenost mezi PFN a NW je přibližně stejně velká jako vzdálenost NW od GP. To znamená, že PFN sedí blízko GP posterioru, zatímco NW stojí daleko od obou. Tyto obě metody predikují zcela odlišné hodnoty. Dále je vidět, že je PFN o $1\,256\times$ přesnější než NW (poměr $\text{MSE(NW, GP)}/\text{MSE(PFN, GP)} = 0,2099/0,000167$).

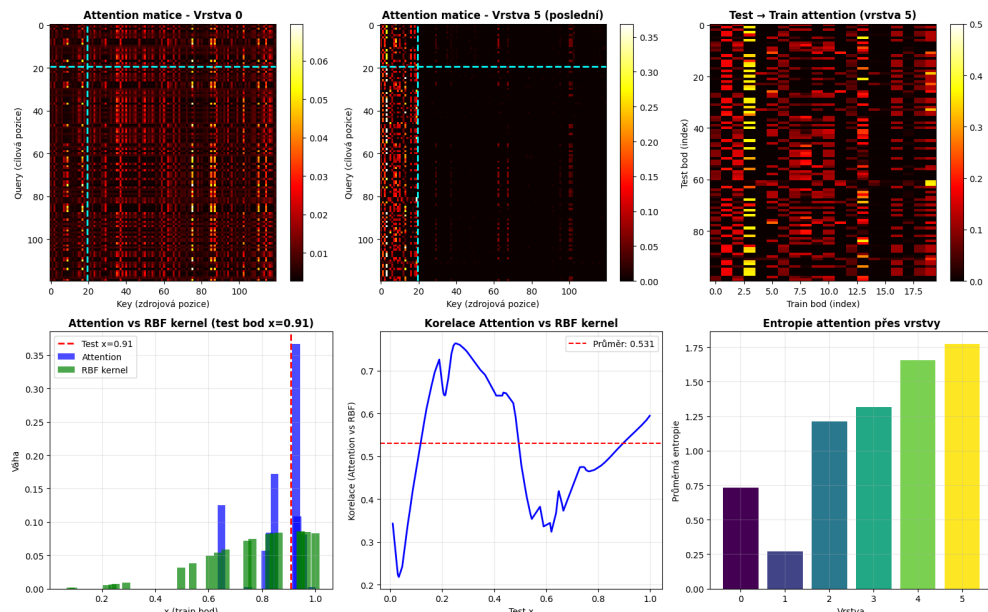
Nás by ale mohlo zajímat, proč je *attention* ostřejší než RBF, tj. proč ty *attention váhy* jsou rozloženy ne tak hladce a plynule, jako u RBF na obrázku 3.2a. *Attention hlavy* totiž nepotřebují počítat čistou kernelovou podobnost $k(x^*, x_i)$. Potřebují vypočítat to, co v kombinaci s ostatními hlavami a dopřednou sítí (*feed-forward network*, kterou budeme v celé práci označovat zkratkou FFN) dá nejlepší aproximaci GP posterioru. A pro ten může být výhodnější jiná strategie, kde se bod dívá na sousedy s jinou silou, dokonce se ukazuje, že se může dívat na vzdálenější body víc než na svoje blízké sousedy. Na konci pak FFN snáze provede korekci odpovídající efektu $\mathbf{K}^{-1}$.

Metodologická poznámka k této analýze:

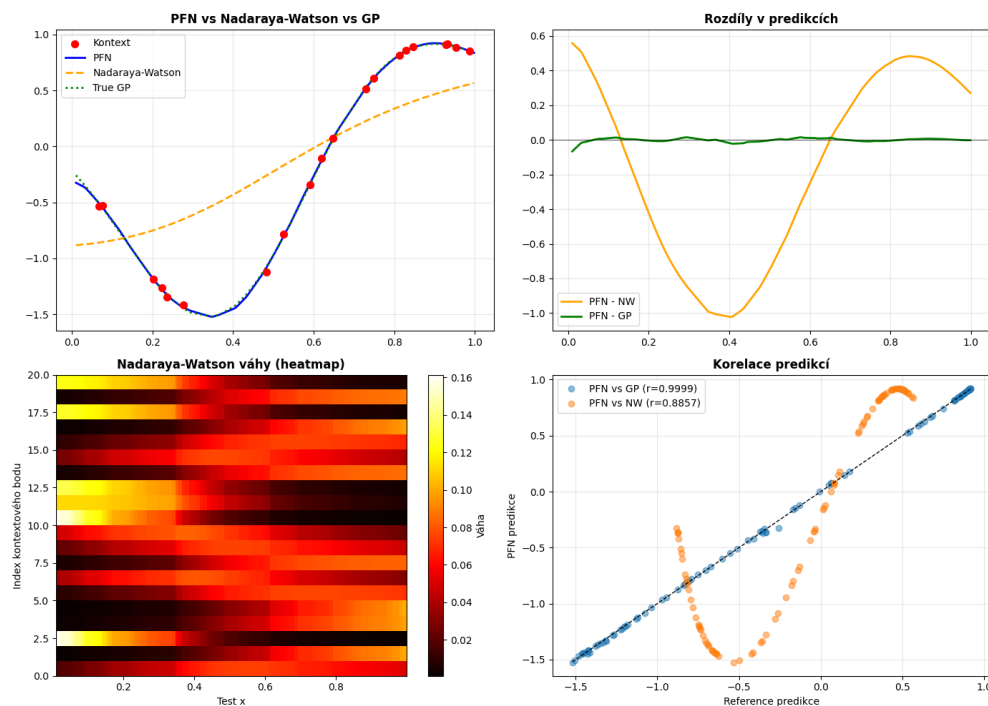
Nízká korelace *attention* vah s RBF kernelem je platný a hodnotný závěr o chování *jedné komponenty* sítě. Existují však důvody, proč z ní nelze přímo usuzovat na celkové chování modelu.

Za prvé, *attention* není výstupní mechanismus, po něm následuje FFN, residuální spojení a v případě vícevrstvého modelu dalších pět bloků. I kdyby *attention váhy* v poslední vrstvě přesně odpovídaly normalizovaným RBF vahám, výstup PFN by se od NW stále lišil díky korekcím v FFN.

Za druhé, GP na rozdíl od RBF neváží trénovací body jen podle jejich podobnosti s x^* , ale přes inverzi $\mathbf{K}^{-1}$ zohledňuje i to, jak jsou body korelované mezi sebou (blízké body si „konkurují“, vzdálené



(a) Detail *attention vah* poslední vrstvy (průměr přes 8 hlav) pro vybraný testovací bod v porovnání s normalizovanými RBF vahami. *Attention* je výrazně ostřejší, koncentruje téměř veškerou váhu na nejbližší bod, zatímco RBF distribuuje váhu plynule. Korelace $0,374 \pm 0,270$.



(b) Srovnání predikcí PFN, Nadaraya-Watsonova estimátoru a analytického GP posterioru ($n_{\text{ctx}} = 20$). NW systematicky přehlazuje, PFN sleduje GP posterior s $\text{MSE } 167 \times 10^{-6}$ oproti MSE 0,21 u NW – rozdíl $\sim 1256\times$.

Obrázek 3.2: Porovnání *attention vah* s RBF kernelem (nahore) a predikcí PFN, NW a GP (dole).

mohou dostat i zápornou váhu). Nízká korelace *attention* s čistým RBF je tedy očekávaná, i perfektní GP by ji při tomto srovnání vykazoval.

3.1.4 Jak přesný je PFN na skutečné GP realizaci?

Předchozí experiment porovnával PFN se syntetickým GP posteriorem, tj. modelu jsme předložili trénovací body a srovnávali jsme jeho predikci s analytickým řešením. V následující části přistoupíme k přísnějšímu testu. Oba modely (PFN i GP oracle) budou predikovat *skutečné šumové hodnoty* GP realizace, kterou neviděly. Cílem je zjistit, jak se přesnost PFN vyvíjí s velikostí kontextu $n_{\text{ctx}} \in \{5, 10, 15, 20\}$.

Nejprve vygenerujeme kompletní GP realizaci délky 100 bodů, tedy jednu konkrétní náhodnou funkci vzorkovanou z GP, u níž pro každý z 100 vstupů x známe jeho skutečnou hodnotu y s šumem. Z těchto 100 bodů náhodně vybereme n_{ctx} jako *trénovací kontext*, tj. dvojice (x, y) , které oběma metodám ukážeme jako pozorovaná data. Zbývajících $100 - n_{\text{ctx}}$ bodů necháme skryté a slouží jako *testovací cíl*, jejich x předložíme modelu k predikci, ale jejich pravé y -hodnoty mu neprozradíme.

Obě metody nyní pro každý testovací bod vrátí svou predikci $\hat{y}(x)$. PFN ji spočítá jedním dopředným průchodem z kontextu. GP oracle je referenční, „ideální“ Gaussovský proces, který má oproti PFN výhodu, že *zná* pravé hyperparametry, ze kterých byla realizace vygenerována ($\ell = 0,3$, $\sigma^2 = 10^{-4}$), a z kontextu spočítá přesný analytický posterior podle vzorce $\mu(x^*) = k(x^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}]^{-1} \mathbf{y}$. Slouží tedy jako horní hranice toho, čeho lze na daných datech dosáhnout, s níž PFN porovnááme.

Kvalitu predikce měříme pomocí MSE zprůměrované přes všechny testovací body dané realizace. Abychom výsledek nezáviseli na jedné konkrétní realizaci, celý postup zopakujeme a metriky nakonec zprůměrujeme přes $5 \text{ seedů} \times 20 \text{ realizací} = 100 \text{ instancí}$ pro každou hodnotu n_{ctx} .

n_{ctx}	PFN	GP oracle
5	0,0408 $\pm$ 0,0698	0,0340 $\pm$ 0,0635
10	0,0123 $\pm$ 0,0349	0,0112 $\pm$ 0,0360
15	0,0064 $\pm$ 0,0271	0,0028 $\pm$ 0,0036
20	0,0021 $\pm$ 0,0025	0,0017 $\pm$ 0,0011

Tabulka 3.2: MSE vůči skutečným hodnotám GP realizace jako funkce velikosti kontextu. Průměr přes 100 instancí ($5 \text{ seedů} \times 20 \text{ realizací}$).

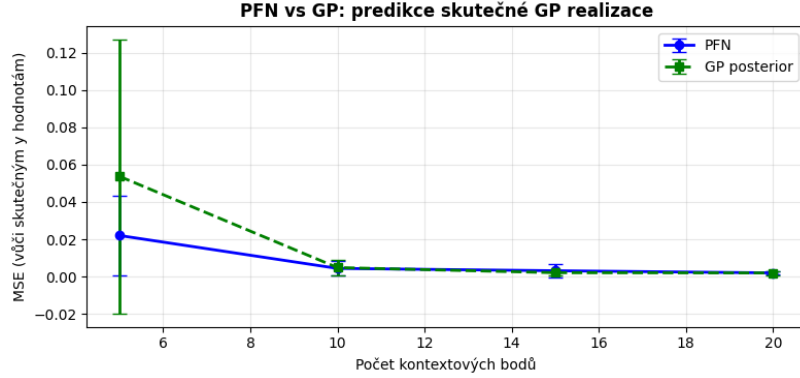
Diskuze:

Z tabulky 3.2 a obrázku 3.3 vyplývají tři pozorování.

Za prvé, GP oracle konzistentně dosahuje nižšího nebo srovnatelného MSE oproti PFN ve všech hodnotách n_{ctx} . To je očekávané: oracle zná přesné hyperparametry a provádí exaktní Bayesovskou inferenci, zatímco PFN je pouze aproximace.

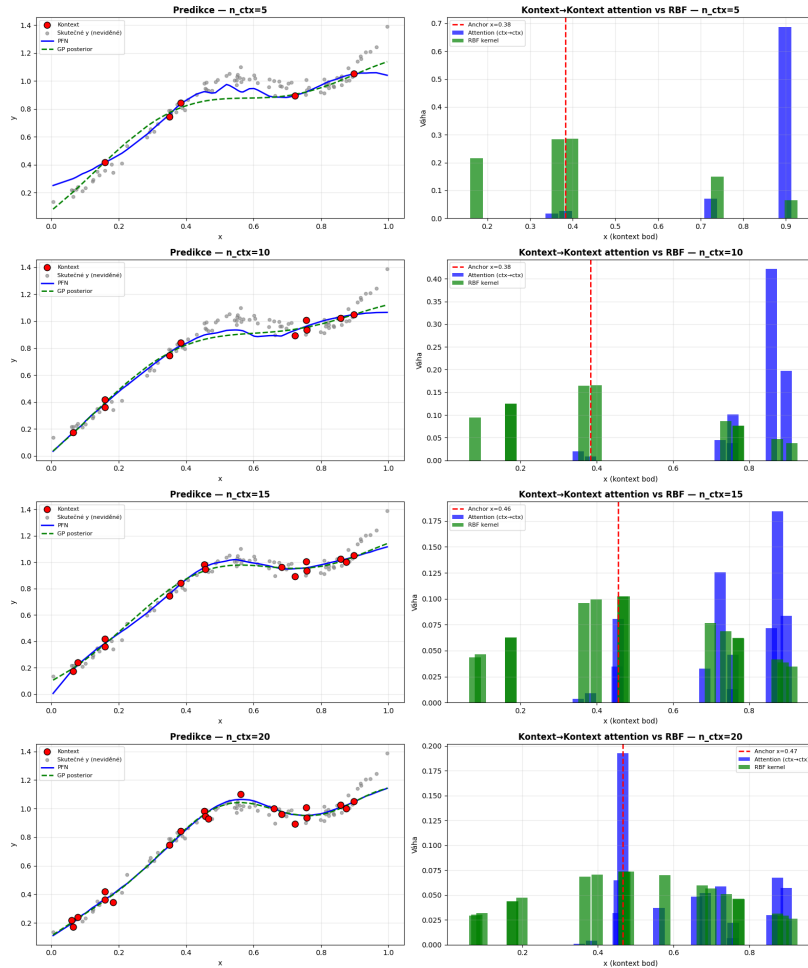
Za druhé, rozdíl se zvětšuje se středním n_{ctx} . Při $n = 5$ je převaha GP oracle malá (0,034 vs 0,041); při $n = 15$ již GP dosahuje 0,003 oproti 0,006 u PFN. Příčinou je, že s více kontextovými body disponuje GP oracle přesnou inverzí kernelové matice $\mathbf{K}^{-1}$, zatímco PFN tuto operaci pouze aproximuje. Při malém kontextu je $\mathbf{K}^{-1}$ málo informativní a obě metody jsou si blízko.

Za třetí, od $n = 20$ se obě metody přibližují: MSE(0,0021 vs 0,0017) jsou si prakticky rovnocenné. PFN tedy s dostatečným kontextem konverguje k chování GP oracle, aniž by explicitně invertoval kernelovou matici. Toto je jedním z klíčových empirických potvrzení toho, že PFN provádí plnou GP inferenci namísto pouhého kernel smoothingu.



(a) MSE PFN (modrá) a GP oracle (zelená přerušovaná) jako funkce n_{ctx} . Chybové úsečky $= \pm 1 \text{ std}$ přes 20 realizací. GP oracle mírně vede od $n = 10$, protože zná správné ℓ .

Experiment 6: vliv velikosti kontextu (stejná GP realizace)



(b) Vizualizace predikcí pro $n \in \{5, 10, 15, 20\}$. Červené body: kontextová data dostupná modelu; šedé body: skutečné neviděné hodnoty; modrá: PFN; zelená přerušovaná: GP posterior se správným ℓ . Všechny řádky vychází ze stejné GP realizace, liší se pouze velikostí kontextu. Od $n = 15$ obě křivky téměř splývají.

Obrázek 3.3: 100-epoch model: MSE a predikce jako funkce velikosti kontextu.

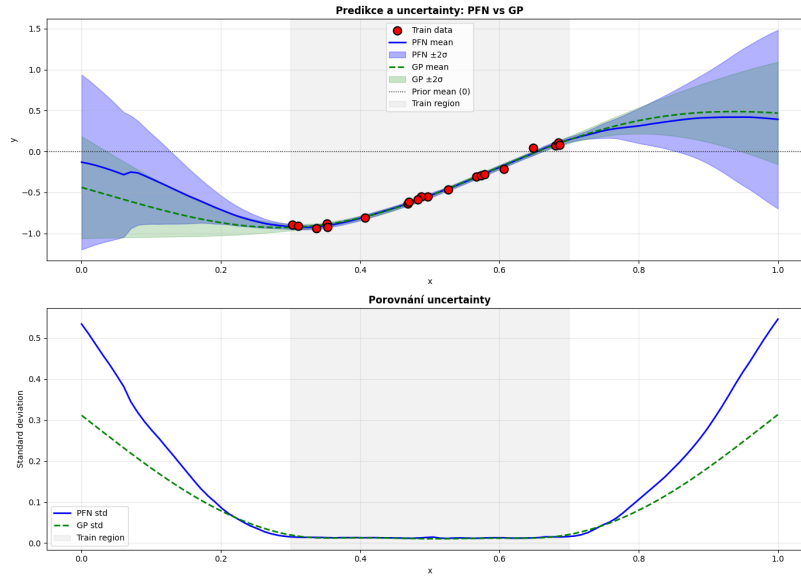
3.1.5 Konvergence střední hodnoty a kalibrace rozptylu

V tomto experimentu ověřujeme, zda PFN správně zachytí nejen střední hodnotu predikce, ale také tvar profilu nejistoty (tzv. *uncertainty*), tedy kde si byl model jistější a kde méně. Trénovací data jsou omezena na oblast $x \in [0,3, 0,7]$ ($n_{\text{ctx}} = 20$), testovací body pokrývají celý interval $[0, 1]$ ($n_{\text{test}} = 100$). Výsledek experimentu jsme opět dostali tímto způsobem – průměrování přes 5 realizací $\times 5$ *seedů* = 25 instancí celkem. Metriky kalibrace: $\text{Corr}(\hat{\sigma}_{\text{PFN}}, \sigma_{\text{GP}})$ (korelace směrodatných odchylek) a $\text{MSE}(\hat{\sigma}_{\text{PFN}}, \sigma_{\text{GP}})$ (absolutní chyba kalibrace). Dále uvádíme *ratio far/near*, což je poměr průměrné směrodatné odchylky (*std*) v oblasti extrapolace ($x \in [0, 0,2] \cup [0,8, 1]$) ke směrodatné odchylce v centru dat ($x \in [0,4, 0,6]$), který měří, jak výrazně model zvyšuje nejistotu mimo oblast trénovacích dat.

Střední hodnota PFN sleduje GP posterior s vysokou přesností v oblasti s daty. Mimo trénovací sadu ($x < 0,3$, $x > 0,8$) PFN správně konverguje k prioru (hodnota 0), stejně jako GP. Avšak poznamenejme, že pro realizace s vysokou amplitudou ($|m_{\text{train}}| > 1,5$), dochází k neúplné konvergenci, GP se v takových případech chová identicky.

Metrika	Hodnota
$\text{Corr}(\text{PFN std}, \text{GP std})$	$0,9812 \pm 0,0252$
$\text{MSE}(\text{PFN std}, \text{GP std})$	$0,00376 \pm 0,00911$
Ratio far/near – PFN	$21,3 \times \pm 5,2 \times$
Ratio far/near – GP	$25,9 \times \pm 3,8 \times$

Tabulka 3.3: Experiment 1: kalibrace rozptylu pro 100-epoch model (průměr přes 25 instancí).



(a) Srovnání střední hodnoty a rozptylu PFN (modrá) a GP (zelená). Trénovací body v $x \in [0,3, 0,7]$, predikce v celém $[0, 1]$. PFN správně rozpoznává region extrapolace zvýšeným rozptylem.

Obrázek 3.4: Experiment 1 – 100-epoch model: konvergence střední hodnoty a kalibrace rozptylu.

Diskuze:

Korelace $r = 0,98$ potvrzuje, že PFN správně identifikuje *tvar* profilu uncertainty tam, kde je nejistota větší a kde menší. $\text{MSE}(\hat{\sigma}) = 0,0038$ signalizuje, že absolutní kalibrace je přesná. *Ratio far/near* u PFN

(21,3×) dosahuje přibližně 82 % dynamického rozsahu GP (25,9×). Z toho usuzujeme, že PFN mírně podhodnocuje míru zvýšení *uncertainty* v oblasti extrapolace, avšak můžeme tvrdit, že charakter chování je zachován. Toto je zásadní vlastnost. Model bez explicitní reprezentace GP posterioru přesto generuje *uncertainty*, která odpovídá bayesovskému výpočtu.

3.2 PFN nad rodinou Gaussovských procesů: náhodné hyperparametry

V předchozí sekci jsme zkoumali model, který jsme natrénovali na *fixních* hyperparametrech, a který se snažil aproximovat posterior jednoho konkrétního Gaussovského procesu s pevně zvoleným $\ell = 0,3$, $\sigma^2 = 10^{-4}$. Šlo o výrazné zjednodušení. Model přesně věděl, jaký tvar funkcí má predikovat, a jediné, co se musel naučit, bylo efektivně invertovat kernelovou matici jednoho jediného kernelu. V této sekci přejdeme k obtížnějšímu a zajímavějšímu případu. Ted' model natrénovujeme na *celé rodině* Gaussovských procesů, jehož hyperparametry jsou při každé dávce vzorkovány náhodně z předem daného rozdělení.

Tento posun mění úlohu zásadním způsobem. Zatímco fixní model aproximuje jeden GP posterior, náhodný model se musí naučit něco podstatně obecnějšího, a to rozpoznat hyperparametry přímo z kontextových dat a teprve poté provést odpovídající inferenci. Z jednoho dopředného průchodu se tak stává nejen aproximace inverze kernelové matice, ale i *implicitní identifikace* korelační délky, amplitudy a šumu. V důsledku model musí provést i amortizovanou Bayesovská marginalizaci přes nejistotu v těchto hyperparametrech. Právě tato vlastnost otevírá řadu otázek, které u fixního modelu nemají smysl:

- Jak přesně model identifikuje ℓ z kontextu?
- Kolik bodů k tomu potřebuje?
- A jak dobře si poradí s celým rozpětím korelačních délek, na kterém byl trénován?

A řadu dalších

3.2.1 Trénink nad rozdělením hyperparametrů

Architektura modelu je identická s fixním případem: 6 vrstev, 8 *attention hlav*, dimenze embeddingu 512, dimenze skryté vrstvy dopředné sítě 1024, výstupní FullSupportBarDistribution s 1000 biny, nastavení `features_per_group=1`. Liší se pouze trénovací režim a délka tréninku oproti fixnímu modelu, který běžel 100 epoch, zatímco tento náš náhodný model běžel 500 epoch. Zatímco fixní model viděl při každé dávce data z téhož GP, náhodný model dostával při každé dávce data z GP s nezávisle vzorkovanými hyperparametry:

$$\ell \sim \mathcal{U}(0,05, 1,0), \quad \text{outputscale} \sim \text{LogNormal}(0, 0,5), \quad \sigma^2 \sim 10^{\mathcal{U}(-3,-1)}.$$

Vstupní prostor zůstává $x \in [0, 1]$, kernel je opět RBF. Šum tak během tréninku pokrývá rozmezí $\sigma^2 \in [10^{-3}, 10^{-1}]$. V experimentech níže, kde náhodný model testujeme samostatně (kalibrace rozptylu, přesnost napříč ℓ , identifikace ℓ), přitom volíme fixní šum $\sigma^2 = 10^{-3}$ na spodní hranici tohoto rozsahu, aby testovací data zůstala uvnitř trénovací distribuce.

Jak jsme již řekli v úvodu této sekci, tato drobná změna trénovacího protokolu má velký důsledek. Model totiž musí generalizovat přes nespočetně mnoho tvarů funkcí, a to od rychle oscilujících realizací s krátkou korelační délkou po hladké, téměř lineární průběhy. Nemůže si proto „zapamatovat“ jeden kernel, musí se ho naučit najít z dat. Právě proto vyžaduje podstatně delší trénink oproti fixnímu modelu.

Dále postupujeme ve dvou krocích. Nejprve ověříme, zda si náhodný model zachovává stejné vnitřní mechanismy, které jsme identifikovali u fixního modelu (*attention asymetrie*, ne-RBF strategie, kalibrace rozptylu), a za jakou cenu. Poté se zaměříme na schopnost, kterou fixní model **nemá** a mít nemůže, konkrétně na identifikaci hyperparametrů přímo z kontextových dat.

Přenositelnost vnitřních mechanismů.

Náhodný model řeší těžší úlohu než fixní. Přirozená otázka tedy zní, zda si zachovává stejné kvalitativní mechanismy, které jsme odhalili v předchozí sekci, nebo zda ho nutnost generalizace vede k odlišné

vnitřní organizaci. V této části projdeme tři klíčové experimenty z předchozí sekce: strukturu *attention matic*, srovnání *attention* s RBF kernelem a kalibraci rozptylu, a v každém z nich porovnáme oba modely přímo.

3.2.2 Struktura attention matic

Stejně jako u fixního modelu vykreslíme *attention matic* pro všech 6 vrstev (průměr přes 8 hlav) a sledujeme jejich vývoj napříč hloubkou sítě. Připomínáme kvadrantovou interpretaci: matici lze rozdělit podle toho, zda *query* i *key* pozice patří trénovacím, nebo testovacím bodům, přičemž pro nás klíčový je kvadrant *Test*→*Train*, který říká, jak testovací body čerpají informaci z kontextu.

Výsledek je na obrázku 3.5. Kvalitativní vzor se podobá fixnímu modelu. Vrstva 0 je pořad spíše poznávací, model se dívá na celou situaci a zkoumá její zákonitosti. Prostřední vrstvy postupně řádnou a specializují se. Poslední vrstva vykazuje jasně dominantní kvadrant *Test*→*Train*, zatímco *Train*→*Test* je téměř nulový. Model tímto způsobem sám z dat objevil základní princip GP inference: testovací body jsou podmíněny trénovacími, ale ne naopak.

Jediný rozdíl je kvantitativní: vzory jsou u náhodného modelu se zdají zašuměné a méně čisté než u fixního. To dává dobrý smysl. Fixní model se v poslední vrstvě soustředí uje na inverzi jediného, předem známého kernelu, a jeho *attention* se proto může vyladit velmi ostře. Náhodný model naproti tomu musí ve stejných vahách zakódovat i identifikaci hyperparametrů, proto část kapacity *attention* jde na hledání korelační délky z kontextu, nikoli jen na samotné vážení bodů. Výsledná matice je proto „rozmazanější“, aniž by však ztratila základní strukturu.

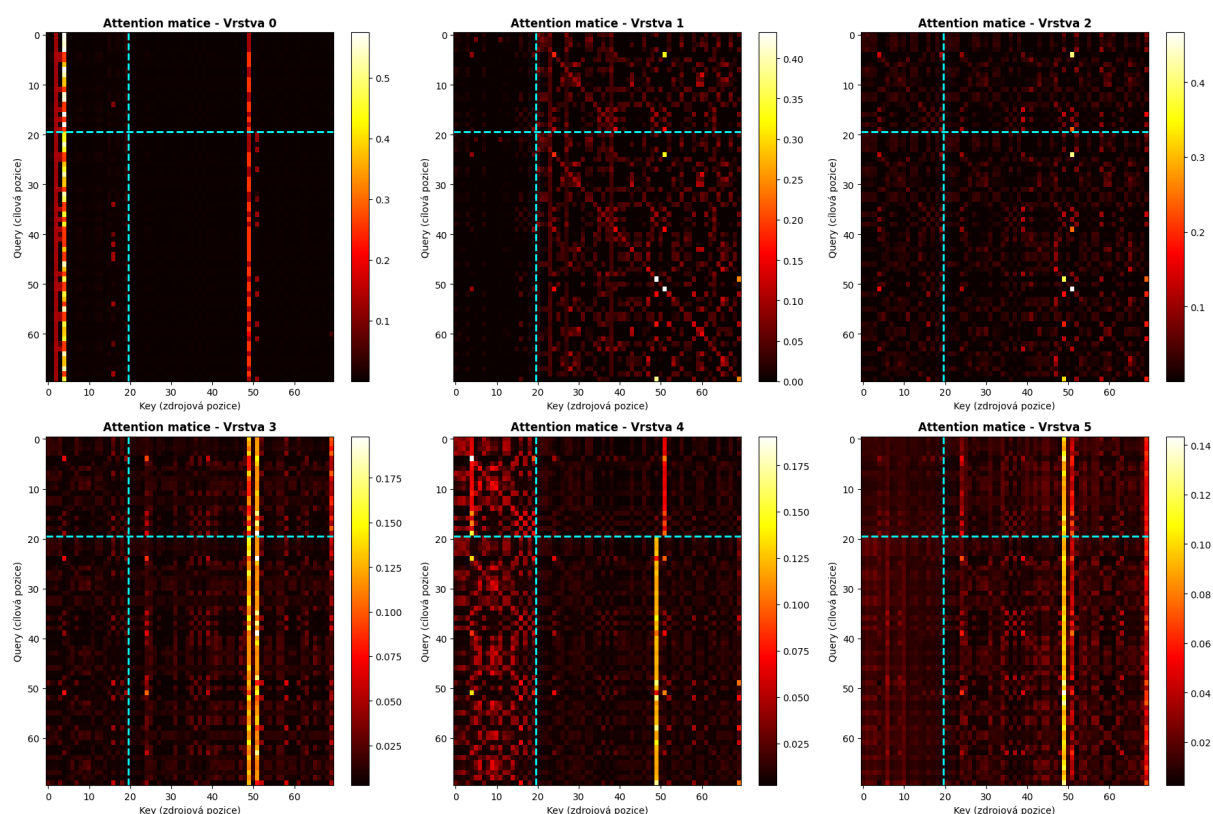
3.2.3 Attention vs RBF kernel

V předchozí sekci jsme ukázali, že *attention* fixního modelu *nekoreluje* s normalizovaným RBF kernelem. Model se naučil ostřejší a nelokální strategii, nikoli prostý kernel-smoothing. Zopakujeme stejný experiment i pro náhodný model. Pro každý testovací bod x^* porovnáme *attention váhy* poslední vrstvy s normalizovanými RBF vahami $w_i = k(x^*, x_i) / \sum_j k(x^*, x_j)$. Výsledky obou modelů shrnuje následující tabulka 3.4.

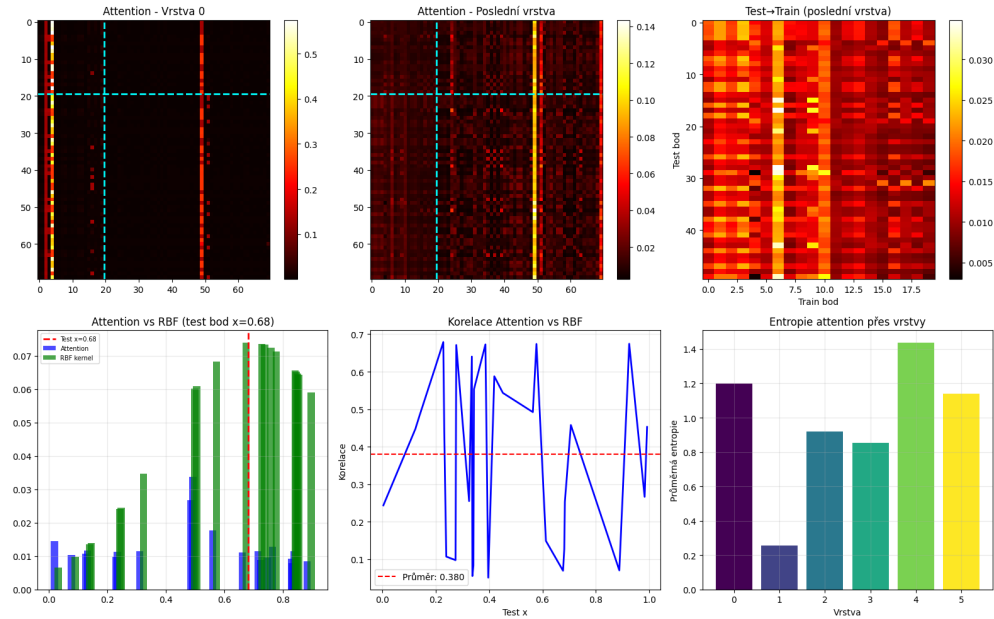
Metrika	Fixní model	Náhodný model
Průměrná korelace Attn vs RBF	$0,374 \pm 0,270$	$0,316 \pm 0,311$

Tabulka 3.4: Korelace *attention vah* poslední vrstvy s normalizovaným RBF kernelem. Průměr přes 5 *seedů*. U náhodného modelu je korelace ještě nižší než u fixního.

Korelace náhodného modelu ($0,316 \pm 0,311$) je dokonce *nižší* než u fixního ($0,374 \pm 0,270$). Na první pohled by se to mohlo jevit jako zhoršení, ale je tomu naopak. Jak jsme již podrobně rozebrali v předchozí sekci, čistý RBF kernel $k(x^*, x_i)$ není správnou referenční hodnotou, tou jsou GP váhy $w_i^{\text{GP}} = \sum_j k(x^*, x_j) (\mathbf{K}^{-1})_{ji}$, které jsou nelokální a mohou být i záporné. Nižší korelace tedy neznamená, že se model vzdaluje od GP. Spíše znamená, že se vzdaluje od prosté *kernel-averaging* strategie a nachází lepší a více efektivní reprezentaci. U náhodného modelu je tento efekt výraznější, protože model se navíc naučil, že fixní RBF s jedním konkrétním ℓ vůbec není univerzálně správný. Korelační délku teprve odhaduje z dat, takže se na žádný pevný kernel nemůže spoléhat. Obrázek 3.6 potvrzuje podobný kvalitativní obraz jako u fixního modelu: *attention* je ostrý a koncentruje váhu na nejbližší body, zatímco RBF ji rozprostírá více plynule.



Obrázek 3.5: *Attention matice* pro všech 6 vrstev náhodného modelu (průměr přes 8 hlav). Cyánová čára odděluje trénovací a testovací pozice. Vzor je kvalitativně identický s fixním modelem – od distribuované vrstvy 0 po dominantní blok *Test*→*Train* v poslední vrstvě – pouze méně vyostřený, protože část *attention rozpočtu* jde na identifikaci hyperparametrů.



Obrázek 3.6: Detail *attention vah* poslední vrstvy náhodného modelu (modrá) pro vybraný testovací bod v porovnání s normalizovanými RBF vahami (červená). Stejně jako u fixního modelu je *attention* výrazně ostřejší a s RBF nekoreluje (korelace $0,316 \pm 0,311$).

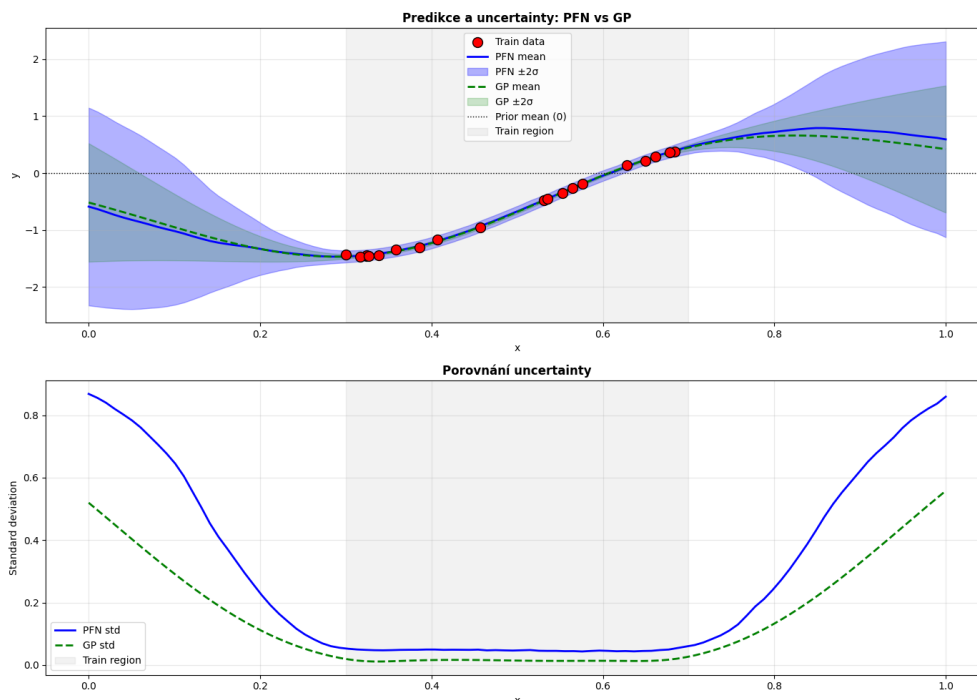
3.2.4 Kalibrace rozptylu

Zatímco střední hodnota vypovídá o kvalitě bodové predikce, rozptyl vypovídá o tom, zda model správně kvantifikuje svou nejistotu. Zopakujeme tedy experiment kalibrace z předchozí sekce i pro náhodný model. Trénovací data jsou omezena na oblast $x \in [0,3, 0,7]$ ($n_{\text{ctx}} = 20$), testovací body pokrývají celý interval $[0, 1]$, a měříme, jak moc směrodatná odchylka PFN odpovídá GP. Model je přitom testován na datech z GP ($\ell = 0,3$), tedy na jediné korelační délce, i když byl trénován na celém rozpětí.

Zde budeme zkoumat tzv. **profil nejistoty** – tedy to, jak se předpovídaná směrodatná odchylka modelu mění napříč vstupním prostorem. U dobře kalibrovaného modelu by měla být nejmenší tam, kde leží trénovací data (uvnitř $[0,3, 0,7]$), a plynule růst směrem k okrajům intervalu, kam už žádná data nedosahují – tam, kde model „neví“, má dávat najevo větší nejistotu. Sledujeme přitom dvě odlišné věci: zda má profil správný *tvar* (tzn. roste nejistota na správných místech?) a zda má i správnou *velikost* (tzn. roste dostatečně, nebo si model svou nejistotou příliš věří?). Výsledky obou modelů srovnává tabulka 3.5.

Metrika	Fixní model	Náhodný model
Corr(PFN std, GP std)	$0,9812 \pm 0,0252$	$0,9734 \pm 0,0292$
MSE(PFN std, GP std)	$0,00376 \pm 0,00911$	$0,03478 \pm 0,03461$
Ratio far/near – PFN	$21,3 \times \pm 5,2 \times$	$7,49 \times \pm 2,20 \times$
Ratio far/near – GP	$25,9 \times \pm 3,8 \times$	$25,9 \times \pm 3,8 \times$

Tabulka 3.5: Kalibrace rozptylu obou modelů (průměr přes 25 instancí). *Ratio far/near* je poměr směrodatné odchylky v oblasti extrapolace ke směrodatné odchylce v centru dat. Náhodný model správně poznává, *kde* je nejistota větší a kde menší (vysoká korelace), ale její *velikost* odhaduje výrazně hůř, v okrajích ji zvyšuje méně, než by měl.



Obrázek 3.7: Náhodný model: srovnání střední hodnoty a rozptylu PFN (modrá) a GP (zelená). Trénovací body v $x \in [0,3, 0,7]$, predikce v celém $[0, 1]$. Model správně zvyšuje nejistotu v oblasti extrapolace, ale nárůst je mírnější než u GP – širší predikce v centru dat odráží trénink na distribuci hyperparametrů.

Zde nacházíme první výraznou nevýhodu náhodného modelu. Korelace profilu nejistoty zůstává vysoká ($r = 0,973$ oproti $0,981$ u fixního modelu). Obě sítě tedy správně rozpoznávají kde je nejistota větší a kde je menší. Absolutní kalibrace (tj. skutečná *velikost* předpovídané nejistoty, nikoli jen to, kde je větší a kde menší) je však u náhodného modelu podstatně horší. MSE směrodatné odchylky vzroste zhruba desetinásobně ($0,0348$ vs $0,0038$) a *ratio far/near* klesne z $21,3\times$ na pouhých $7,49\times$, tj. z 82% na zhruba 29% dynamického rozsahu GP. Jinými slovy, náhodný model zvyšuje nejistotu v oblastech extrapolace znatelně méně výrazně.

Diskuze:

Tento výsledek se může zdát na první pohled paradoxní. Model trénovaný na větším množství epoch (500) má horší kalibraci rozptylu než model trénovaný na menším množství epoch (100). Vysvětlení se schovává právě v širších trénovacích podmínkách. Náhodný model se učil kalibrovat nejistotu pro celé rozpětí $\ell \in [0,05, 1,0]$, přičemž profil rozptylu se s korelační délkou dramaticky mění, a to takovým způsobem, že krátká korelační délka znamená rychlý nárůst nejistoty mimo data, dlouhá naopak volnější. Model tedy nemůže být současně dokonale kalibrován pro všechny ℓ , jeho odhad rozptylu je jistým průměrem přes možné korelační délky, což profil nutně vyhlazuje. Testujeme-li ho na jednom konkrétním $\ell = 0,3$, projeví se toto vyhlazení jako širší predikce v centru a mírnější nárůst na okrajích (obrázek 3.7). Fixní model tímto kompromisem netrpí, protože zná jediné správné ℓ . Tímto usuzujeme, že jde tedy o cenu za generalizaci, nikoli o vadu tréninku.

Nové schopnosti: identifikace hyperparametrů.

Dostáváme se k jádru toho, proč náhodný model vůbec studujeme. Experimenty této sekce nemají u fixního modelu obdobu, testují totiž přesně tu schopnost, kterou fixní model postrádá, a tou je identifikace hyperparametrů Gaussovského procesu z kontextových dat a přizpůsobit jim predikci.

3.2.5 Přesnost napříč korelačními délkami

Nejprve se ptáme, jak dobře model zvládá různé korelační délky. Pro každou hodnotu $\ell \in \{0,05, 0,1, 0,2, 0,3, 0,5, 0,7, 0,9\}$ vygenerujeme GP realizace, předložíme modelu $n_{\text{ctx}} = 30$ kontextových bodů a měříme MSE mezi jeho střední hodnotou a analytickým GP posteriorem se správným ℓ . Výsledky (medián, průměr a směrodatná odchylka přes 50 instancí) shrnuje tabulka 3.6.

ℓ	Medián MSE	Průměr MSE	Std MSE
0,05	0,01317	0,01918	0,01767
0,10	0,000604	0,002773	0,009616
0,20	0,000102	0,002304	0,010984
0,30	$4,57 \times 10^{-5}$	$2,18 \times 10^{-4}$	$7,40 \times 10^{-4}$
0,50	$1,99 \times 10^{-5}$	$2,86 \times 10^{-5}$	$2,42 \times 10^{-5}$
0,70	$1,03 \times 10^{-5}$	$1,52 \times 10^{-5}$	$1,90 \times 10^{-5}$
0,90	$9,18 \times 10^{-6}$	$1,23 \times 10^{-5}$	$9,99 \times 10^{-6}$

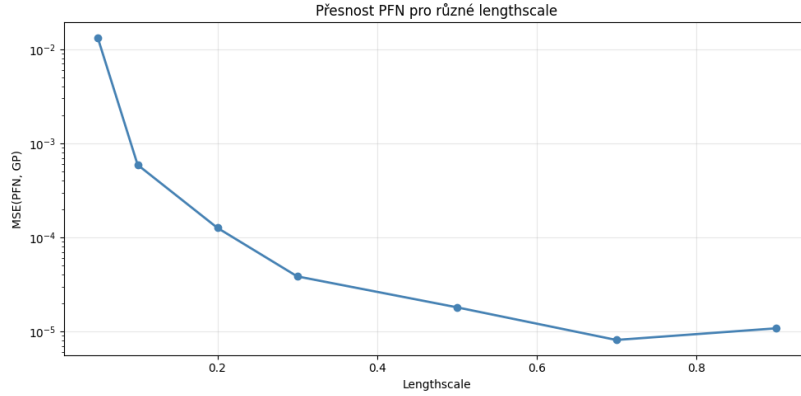
Tabulka 3.6: MSE(PFN, GP) náhodného modelu jako funkce korelační délky ℓ ($n_{\text{ctx}} = 30$, průměr přes 50 instancí). Medián klesá monotónně s rostoucím ℓ – o dva řády z $\ell = 0,05$ na $\ell = 0,9$.

Medián MSE klesá monotónně s rostoucím ℓ , z 0,013 při $\ell = 0,05$ na $9,2 \times 10^{-6}$ při $\ell = 0,9$, tedy skoro o více než tři řády. Avšak nejtěžší je pro model rychle oscilující funkce s krátkou korelační délkou. Důvod je dvojitý. Za prvé, funkce s $\ell = 0,05$ vyžaduje hustou vzorkovací mřížku, 30 bodů na intervalu $[0, 1]$ prostě nestačí k jednoznačné identifikaci tak krátké korelační délky, mezi body zůstává příliš mnoho neznámé informace. Dalším vysvětlením může být to, že trénovací rozdělení $\ell \sim \mathcal{U}(0,05, 1,0)$ přiřazuje hodnotám $\ell < 0,1$ jen 5 % pravděpodobnostní hmoty, takže model viděl extrémně krátké korelační délky během tréninku jen zřídka. Poznamenejme i rozdíl mezi mediánem a průměrem pro malá ℓ : průměr je výrazně vyšší (např. 0,0028 vs medián 0,0006 při $\ell = 0,1$), což může prozrazovat existenci ojedinělých, výrazně špatných instancí, které táhnou průměr nahoru. Pro $\ell \geq 0,5$ jsou medián i průměr téměř totožné. Model je v *hladkém režimu* spolehlivý a bez outlierů. Obrázek 3.8b tento efekt ilustruje názorně: pro krátká ℓ PFN „přehlazuje“ rychlé oscilace, protože je z řídkého kontextu nedokáže rozlišit.

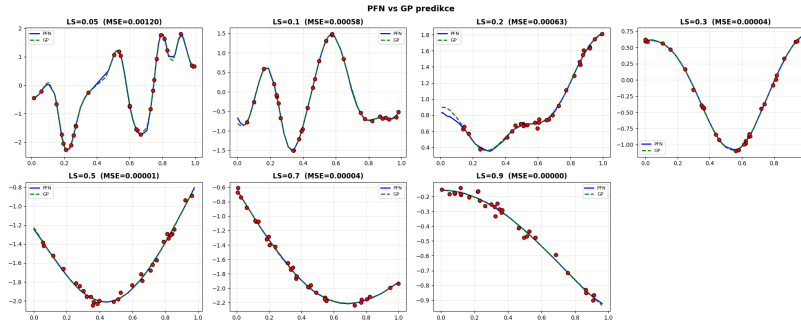
3.2.6 Identifikace korelační délky z kontextu

Předchozí experiment ukázal, že přesnost roste s ℓ , ale ještě přímo nedokázal, že model korelační délku skutečně umí *identifikovat*. K tomu slouží následující test. Data pocházejí z GP se známým ℓ . Modelu předložíme $n_{\text{ctx}} \in \{3, 5, 10, 20, 40, 60\}$ bodů a jeho predikci porovnáme se dvěma referenčními posteriory: se *správným* GP (zná pravé ℓ) a s *špatným* GP (má výrazně odlišné ℓ_{wrong}). Klíčová je následující úvaha: pokud PFN identifikoval korelační délku z dat, jeho predikce bude blízko správnému GP a daleko od špatného, tedy poměr $\text{MSE}(\text{wrong})/\text{MSE}(\text{correct})$ bude velký. Pokud ji neidentifikoval, obě chyby budou podobné a poměr bude blízký jedné. Provedeme tento test pro krátké ($\ell = 0,1$, $\ell_{\text{wrong}} = 0,7$) i střední ($\ell = 0,2$, $\ell_{\text{wrong}} = 0,05$) korelační délky. Výsledky jsou v tabulkách 3.7 a 3.8.

Výsledek je jednoznačný a představuje jeden z nejsilnějších důkazů, že náhodný model skutečně provádí identifikaci hyperparametrů. Při $n_{\text{ctx}} = 3$ je poměr *wrong/correct* pouze 9–15×. S tak malým kontextem model nedokáže korelační délku spolehlivě určit a musí hledat kompromis mezi možnými ℓ , takže od obou GP je zhruba stejně daleko. Od $n_{\text{ctx}} = 20$ však poměr roste rychle lineárně a při $n_{\text{ctx}} = 60$ dosahuje $1\,691\times$ pro $\ell = 0,1$. Zásadní je, že MSE vůči *správnému* GP klesá exponenciálně (při $n = 60$ je 0,000134, tedy prakticky nulová), zatímco MSE vůči *špatnému* GP zůstává konstantně vysoký ($\sim 0,23$). To znamená, že od $n_{\text{ctx}} \approx 20$ se náhodný model chová skoro identicky jako GP se správnou korelační délkou bez toho, aby mu ji kdokoli sdělil. Identifikace $\ell = 0,2$ je o něco snazší (výraznější korelační



(a) MSE(PFN, GP) jako funkce ℓ . Krabicové grafy ukazují distribuci přes 50 instancí. Výrazný nárůst chyby pro krátkou korelační délku $\ell = 0,05$.



(b) Vizualizace predikcí pro různé ℓ . Pro malé ℓ (rychlé oscilace) PFN zatelně přehlazuje, pro velké ℓ sleduje GP posterior téměř dokonale.

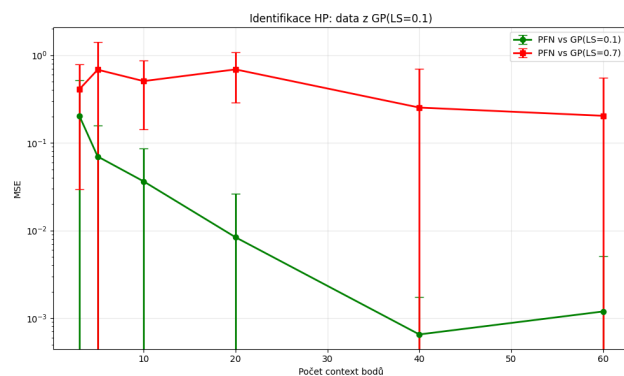
Obrázek 3.8: Náhodný model: přesnost napříč korelačními délkami.

n_{ctx}	MSE(PFN, $\text{GP}_{\text{correct}}$)	MSE(PFN, GP_{wrong})	Poměr
3	$0,1239 \pm 0,1047$	$1,846 \pm 4,437$	$14,9\times$
5	$0,1043 \pm 0,1226$	$1,403 \pm 6,431$	$13,4\times$
10	$0,0483 \pm 0,0579$	$0,812 \pm 2,033$	$16,8\times$
20	$0,00855 \pm 0,01269$	$0,2665 \pm 0,1725$	$31,2\times$
40	$0,000629 \pm 0,00149$	$0,2799 \pm 0,1707$	$444,7\times$
60	$0,000134 \pm 0,000111$	$0,2266 \pm 0,1447$	$1\,691\times$

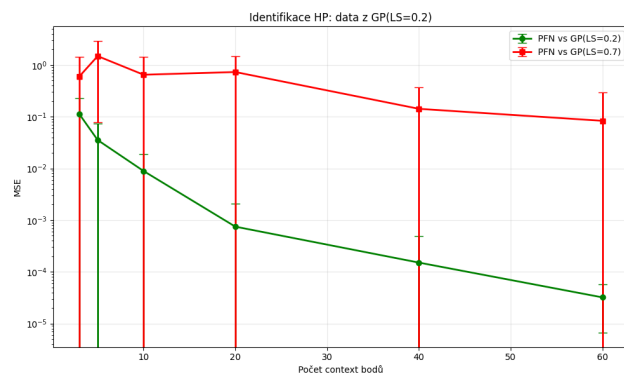
Tabulka 3.7: Identifikace korelační délky pro $\ell = 0,1$ (proti $\ell_{\text{wrong}} = 0,7$). Průměr přes 50 instancí. Poměr wrong/correct roste lineárně s n_{ctx} .

n_{ctx}	MSE(PFN, $\text{GP}_{\text{correct}}$)	MSE(PFN, GP_{wrong})	Poměr
3	$0,0610 \pm 0,0563$	$0,5499 \pm 0,9273$	$9,0\times$
5	$0,0613 \pm 0,1150$	$0,4576 \pm 2,020$	$7,5\times$
10	$0,0200 \pm 0,0415$	$0,2319 \pm 0,4873$	$11,6\times$
20	$0,00170 \pm 0,00409$	$0,04528 \pm 0,03577$	$26,6\times$
40	$8,43 \times 10^{-5}$	$0,04148 \pm 0,04236$	$491,8\times$
60	$3,78 \times 10^{-5}$	$0,03028 \pm 0,02328$	$801,4\times$

Tabulka 3.8: Identifikace korelační délky pro $\ell = 0,2$ (proti $\ell_{\text{wrong}} = 0,05$). Průměr přes 50 instancí.



(a) $\ell = 0,1$ proti $\ell_{\text{wrong}} = 0,7$. Od $n_{\text{ctx}} = 20$ se MSE vůči správnému GP a špatnému GP rozevírají, poměr dosahuje $1\,691\times$ při $n = 60$.



(b) $\ell = 0,2$ proti $\ell_{\text{wrong}} = 0,05$. Tentýž trend, poměr $801\times$ při $n = 60$.

Obrázek 3.9: Náhodný model: identifikace korelační délky z kontextu. MSE(PFN, $\text{GP}_{\text{correct}}$) klesá exponenciálně s n_{ctx} , zatímco MSE vůči špatnému GP zůstává vysoký.

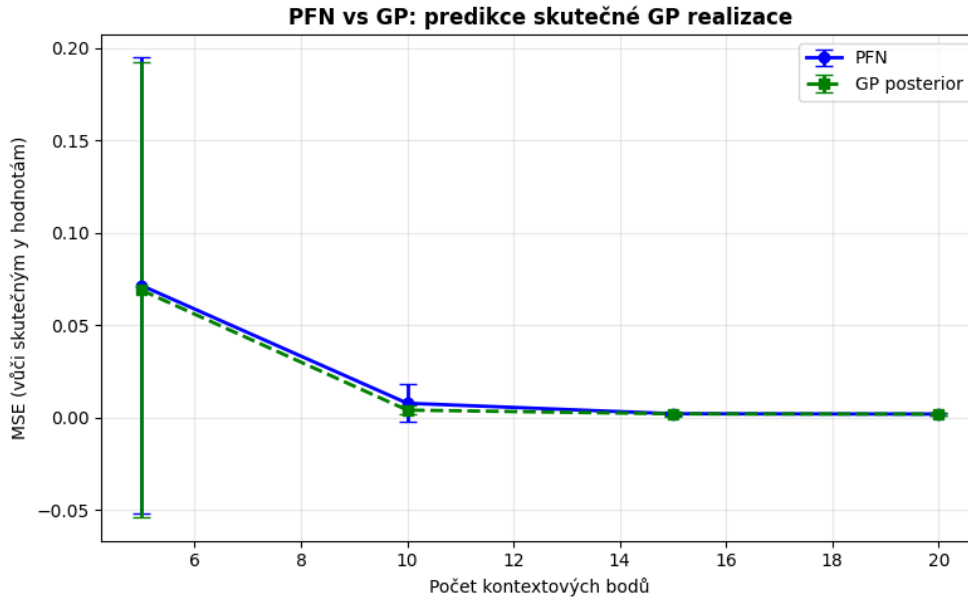
vzor), ale rozdíl mezi oběma režimy není dramatický. Právě zde se ukazuje kvalitativní propast mezi oběma modely. Dá se říct, že fixní model má korelační délku „zabudovanou“ do vah, náhodný model ji čte z kontextu.

3.2.7 Predikce na skutečné realizaci a role kontextu

Zopakujeme nakonec i experiment z předchozí sekce, ve kterém oba modely predikují *skutečné* (šumové) hodnoty neviděné GP realizace, s kontextovými body $n_{\text{ctx}} \in \{5, 10, 15, 20\}$. Připomeňme, že *fixní PFN* je model natrénovaný na jediném GP s pevně danou korelační délkou $\ell = 0,3$. Tu jsme si řekli, že má „zabudovanou“ a nemusí ji z dat zjišťovat. Naproti tomu *náhodný PFN* byl trénován na celé rodině GP s různými ℓ , takže správnou korelační délku musí u každého vstupu nejprve rozpoznat z kontextu, a teprve pak predikovat. Tentokrát nás zajímá přímé srovnání obou těchto modelů a *GP oracle*, který zná správné $\ell = 0,3$. Výsledky jsou uvedeny v tabulce 3.9.

n_{ctx}	PFN (náhodný)	PFN (fixní)	GP oracle
5	$0,0598 \pm 0,0958$	$0,0408 \pm 0,0698$	$0,0340 \pm 0,0635$
10	$0,0188 \pm 0,0751$	$0,0123 \pm 0,0349$	$0,0112 \pm 0,0360$
15	$0,0063 \pm 0,0193$	$0,0064 \pm 0,0271$	$0,0028 \pm 0,0036$
20	$0,0020 \pm 0,0013$	$0,0021 \pm 0,0025$	$0,0017 \pm 0,0011$

Tabulka 3.9: MSE vůči skutečným hodnotám GP realizace ($\ell = 0,3$) jako funkce velikosti kontextu. Průměr přes 100 instancí. Náhodný model zaostává za fixním pro střední n , ale při $n = 20$ je s ním srovnatelný.

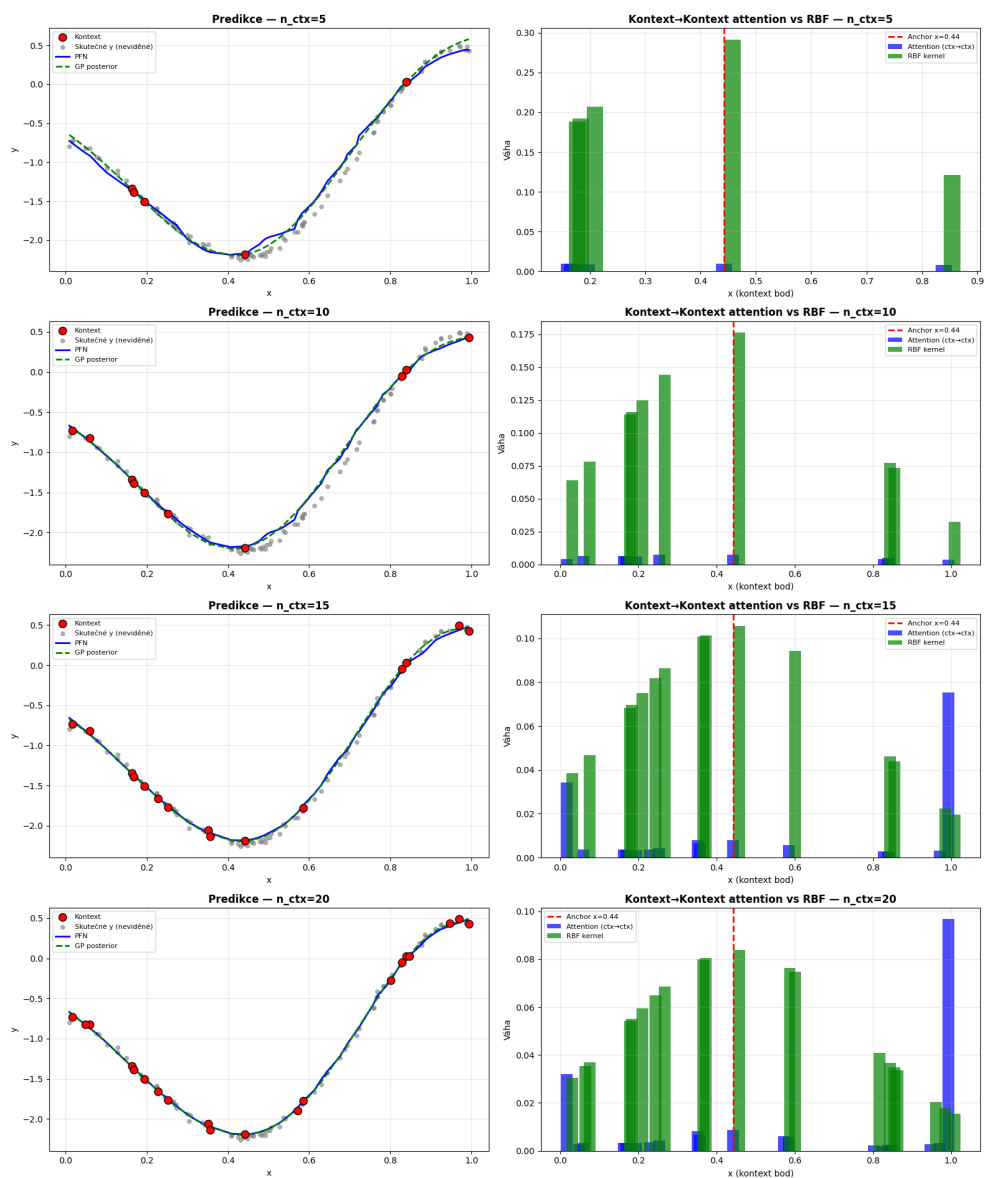


Obrázek 3.10: Náhodný model: MSE PFN (modrá) a GP oracle (zelená přerušovaná) jako funkce n_{ctx} . Rozptyl je větší než u fixního modelu, zejména pro malé n , náhodný model musí ℓ identifikovat při každé inferenci.

Srovnání s tabulkou 3.2 z předchozí sekce opět odhaluje jakousi daň za generalizací. Pro střední velikost kontextu ($n = 5$ až 10) je náhodný model viditelně horší než fixní ($0,060$ vs $0,041$ při $n = 5$),

protože fixní model zná $\ell = 0,3$, kdežto náhodný ho musí z několika málo bodů teprve odhadnout a při řídkém kontextu je tento odhad zatížen chybou, což se projeví i větším rozptylem na obrázku 3.10. Od $n = 15$ se však oba modely začínají vyrovnávat a při $n = 20$ jsou prakticky nerozlišitelné (0,0020 vs 0,0021), oba těsně za *GP oracle*. Náhodný model tedy s dostatečným kontextem dožene fixní verzi, aniž by kdy znal správné ℓ předem.

Experiment 4: vliv velikosti kontextu (stejná GP realizace)



Obrázek 3.11: Náhodný model: predikce (levý sloupec) a *attention* vs RBF (pravý sloupec) pro $n \in \{5, 10, 15, 20\}$. Červené body: kontext; šedé: skutečné neviděné hodnoty; modrá: PFN; zelená přerušovaná: GP se správným ℓ .

3.2.8 Shrnutí: co přináší náhodné hyperparametry

Přechod od fixního k náhodnému modelu odhalil dvě tváře této architektury. Na jedné straně si náhodný model zachovává všechny kvalitativní mechanismy fixního, a to jsou: kauzální asymetrie *attention* (poslední vrstva dominantní *Test*→*Train*), model nedělá RBF strategii (korelace 0,32 vs 0,37), zachovává se správný *tvar* profilu nejistoty (korelace std > 0,97). Vnitřní organizace transformera je tedy robustní vůči šíři trénovací distribuce.

Na druhé straně za tuto obecnost platí jistou daň. Absolutní kalibrace rozptylu se zhorší (*ratio far/near* klesne z 21,3× na 7,49×), protože jeden model nemůže být dokonale kalibrován pro všechny korelační délky současně. A pro střední velikosti kontextu je náhodný model méně přesný než fixní, neboť korelační délku teprve odhaduje z dat.

Avšak rozhodující je to, co náhodný model umí navíc. Dokáže identifikovat korelační délku z kontextu, a to spolehlivě. Poměr $MSE(wrong)/MSE(correct)$ roste exponenciálně s n a dosahuje 1 691× při $n = 60$. Od $n_{ctx} \approx 20$ se chová téměř identicky jako GP se správným ℓ . Náhodný model tedy je amortizovaný Bayesovský inferenční nástroj, který korelační délku a amplitudu odečítá přímo z pozorovaných dat, aniž by mu je kdokoli sdělil předem.

3.3 Specializace vrstev: čtení labelů a kernelové porovnávání

V předchozích dvou sekcích jsme se pohybovali výhradně mezi šestivrstevnými modely. Nejprve šlo o fixní model ze sekce 3.1, poté o náhodný model ze sekce 3.2. Ukázali jsme, že PFN neprovádí prostý *kernel smoothing* a že náhodný model dokáže identifikovat korelační délku z kontextu. Zůstává však otevřená otázka, jak tento výpočet uvnitř transformeru vlastně probíhá. Jinými slovy, co dělá která vrstva a proč jich model vůbec potřebuje víc než jednu. V této sekci proto poprvé budeme systematicky měnit hloubku sítě a zkoumat, zda se mezi vrstvami vytváří jakási dělba práce, závisí-li výsledky na počtu vrstev apod.

Analýzu provedeme na rodině pěti modelů, které se liší pouze počtem vrstev: 1, 2, 4, 6 a 8 vrstev. Všechny sdílejí identickou architekturu s modely z předchozích sekcí (8 *attention* hlav, dimenze embeddingu 512, dimenze skryté vrstvy dopředné sítě 1024, `features_per_group=1` a výstupní `FullSupportBarDistribution` s 1000 biny). Všechny byly rovněž trénovány na stejném rozdělení hyperparametrů jako náhodný model:

$$\ell \sim \mathcal{U}(0,05, 1,0), \quad \text{outputscale} \sim \text{LogNormal}(0, 0,5), \quad \sigma^2 \sim 10^{\mathcal{U}(-3, -1)}.$$

Jednovrstvý model konvergoval již za 100 epoch, dvou a čtyřvrstvý model byly trénovány 300 epoch a nejhlubší šesti a osmivrstvý 500 epoch. Tato řada modelů nám umožní sledovat, jak se vnitřní strategie transformeru mění s přidáváním vrstev.

Motivace pro tuto analýzu vychází přímo z Naglerovy teoretické práce, kterou jsme rozebírali v sekci 2.4.4. Připomeňme, že v PFN je každý kontextový bod reprezentován *tokenem* $V_j = (Y_j, X_j)^\top$, který v sobě slučuje jak vstupní souřadnici X_j , tak její pozorovanou hodnotu Y_j . *Attention skóre* jednovrstvého modelu je proto lineární kombinací obojího, zhruba tvaru $\alpha X_j + \beta Y_j$. GP váha $k(x^*, X_j)$ naproti tomu závisí výhradně na vstupní souřadnici. Nagler odtud odvozuje, že jediná vrstva nemůže reprodukovat GP posterior přesně, tzv. *tilted-measure limit* se totiž nikdy nelokalizuje kolem x^* , a v predikci proto zůstává nenulový bias. To přirozeně vede k hypotéze o rozdělování práce mezi vrstvy. Pokud jedna vrstva nestačí, mohly by se hlubší modely naučit tento úkol rozdělit, a to tak, že rané vrstvy budou číst hodnoty Y_j (tzv. *label reading*) a prostřední vrstvy porovnávat vzdálenosti ve vstupním prostoru X (tzv. *kernel-like chování*). Právě tuto hypotézu budeme v následujících experimentech testovat.

3.3.1 Korelační analýza attention napříč vrstvami

Abychom mohli změřit, na co se která vrstva dívá, spočítáme pro každou vrstvu a každou z jejích osmi hlav dvě korelace *attention vah*:

- korelaci s *RBF/NW kernelem* $k(x^*, X_j)$, která měří, jak moc se hlava chová jako *kernelové porovnání* vstupních vzdáleností,
- korelaci s *labely* Y_j , která měří, jak moc hlava místo toho čte pozorované hodnoty.

První z nich budeme dále stručně nazývat **kernelová korelace**, druhou **labelová korelace**. Vysoká *kernelová korelace* tedy znamená, že hlava váží body podle jejich vzdálenosti od x^* , vysoká *labelová korelace* naopak, že je váží podle jejich hodnot Y_j . Experiment provedeme na 200 instancích se sdílenými daty pro všechny modely, s kontextem $n_{\text{ctx}} = 40$, $n_{\text{test}} = 20$ a referenčním kernelem s $\ell_{\text{NW}} = 0,5$.

Proč Spearman a ne Pearson:

Ukazuje se totiž, že *attention váhy* jsou silně zešikmené. Naprostá většina váhy leží na několika málo bodech a zbytek je skoro nulový. Pearsonova korelace je citlivá na absolutní hodnoty, a proto monotónní vztah mezi *attention* a kernelem systematicky podhodnocuje. Přechodem na *Spearmanovu* korelaci se

kernelové korelace ve středních vrstvách zvedly z přibližně 0,35 na $\sim 0,8$. Trend zůstává stejný, ale je mnohem výraznější. V celé této sekci proto uvádíme Spearmanovy korelace.

Přehled *kernelových korelací* zprůměrovaných přes hlavy shrnuje tabulka 3.10 a graficky jej spolu s *labelovými korelacemi* zachycuje obrázek 3.12.

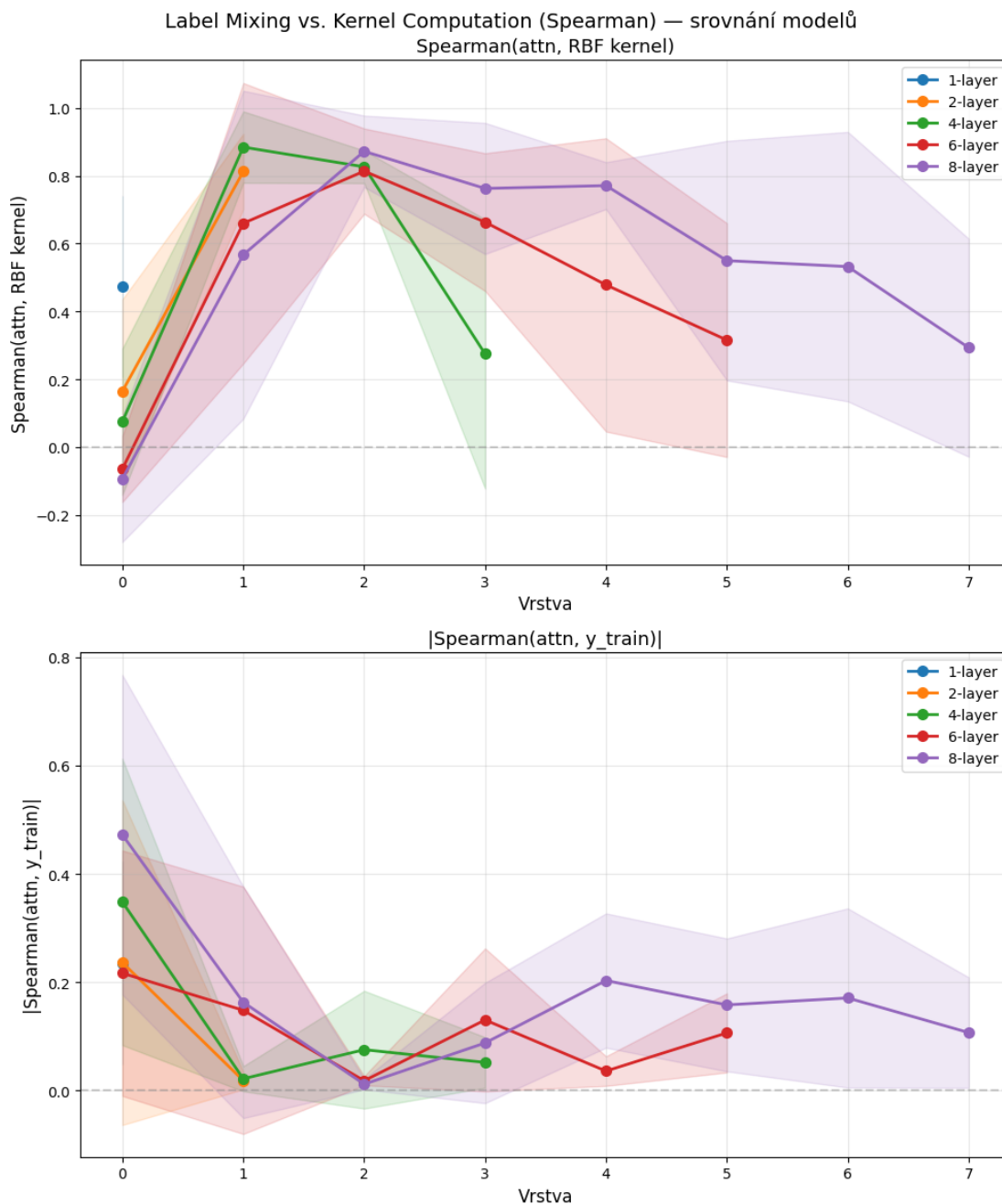
Vrstva	1 vrstva	2 vrstvy	4 vrstvy	6 vrstev	8 vrstev
0	0,473	0,165	0,075	−0,065	−0,094
1	—	0,813	0,885	0,660	0,567
2	—	—	0,826	0,814	0,872
3	—	—	0,275	0,663	0,763
4	—	—	—	0,478	0,771
5	—	—	—	0,316	0,550
6	—	—	—	—	0,532
7	—	—	—	—	0,293

Tabulka 3.10: Spearmanova korelace *attention vah* s RBF/NW kernelem, zprůměrovaná přes 8 hlav dané vrstvy a přes 200 instancí. Pomlčka značí, že daná vrstva u mělčího modelu neexistuje. Ve vrstvě 0 je *kernelová korelace* vždy nízká (u 6 i 8 vrstev dokonce mírně záporná), zatímco ve středních vrstvách skáče přes 0,8.

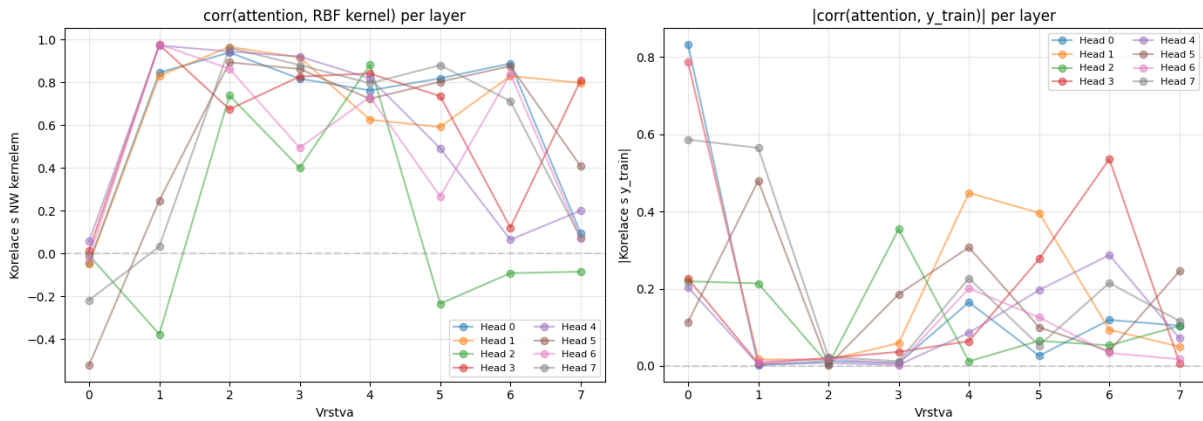
Z tabulky i obrázku vystupuje jasný a opakující se napříč modely vzor. Vrstva 0 má u všech vícevrstvých modelů nejvyšší *labelovou korelaci* ($|\text{corr}| = 0,22$ až $0,47$) a zároveň nejnižší *kernelovou korelaci* ($-0,10$ až $0,17$). U šesti- i osmivrstvého modelu je *kernelová korelace* první vrstvy dokonce mírně záporná. První vrstva tedy primárně čte *labels*, tj. shromažďuje informaci o pozorovaných hodnotách, nikoli o geometrii vstupů. Střední vrstvy tento obraz obracejí. *Kernelová korelace* vyskočí na 0,61 až 0,89 a *labelová* klesne na 0,01 až 0,08. To je právě to *kernel-like chování*, které bychom u GP inference čekali, hlava porovnává vzdálenosti ve vstupním prostoru. Poslední vrstva pak *kernelovou korelaci* opět snižuje (na 0,27 až 0,44), zřejmě proto, že už neslouží k porovnávání, ale k sestavení konečné predikce.

Zvláštní pozornost si v tomto případě zaslouží jednovrstvý model. Jeho jediná vrstva má *kernelovou korelaci* 0,473, což je nejvyšší hodnota v celém řádku vrstvy 0 napříč modely. Má to dobrý smysl. Když má model k dispozici jen jednu vrstvu, nemůže si dovolit ji „utratit“ na pouhé čtení *labelů* a musí v ní stihnout jak čtení *labelů*, tak *kernelové porovnání* současně. Teprve s tím, jak se hloubka modelu zvětšuje, si model může dovolit specializaci, kdy první vrstva jen posbírá *labels* a *kernelovou práci* nechá vrstvám pozdějším. Tuto specializaci na úrovni jednotlivých hlav ilustruje obrázek 3.13. Hlavy uvnitř jedné vrstvy nejsou zaměnitelné, každá pokrývá trochu jiný aspekt vstupu, což je mimo jiné známá vlastnost *multihead-attention* mechanismu v transformerech.

Stejnou heterogenitu lze nahlédnout i prostorově. Obrázek 3.14 zobrazuje, kam se dívá každá z 8 hlav jednovrstvého modelu. V tomto případě usuzujeme, že je dobré pro čtenáře popsat, co tento obrázek znázorňuje a co znamená. Vysvětleme tedy nejprve, jak takový graf vzniká a jak ho číst. Zafixujeme jeden testovací bod, v naší realizaci $x^* = 0,59$, a ptáme se, mezi které kontextové body daná hlava rozdělí svou *attention váhu* při predikci v tomto bodě. Každý kontextový bod vyneseme do roviny jako jednu tečku, kde vodorovná souřadnice je jeho poloha X_j a svislá souřadnice jeho pozorovaná hodnota Y_j . Tečku pak obarvíme podle *attention váhy*, kterou tomuto bodu hlava přidělila, tj. čím světlejší tečka, tím větší váha. Graf tedy čteme tak, že sledujeme, kde se světlé tečky hromadí. Pokud se hromadí ve svislém pruhu kolem x^* , hlava vybírá body podle blízkosti ve vstupním prostoru a hodnota Y_j ji nezajímá. Pokud se naopak hromadí nahoře nebo dole, hlava vybírá body podle jejich hodnoty Y_j , tedy čte *labels*. Kdyby tedy hlava prováděla čisté *kernelové porovnání*, viděli bychom svislý pruh kolem $x^* = 0,59$ nezávisle na Y_j . Skutečnost podle tohoto obrázku je ale jiná. Hlavy míří na různé oblasti roviny. *Head 2* se dívá



Obrázek 3.12: Srovnání všech pěti modelů. Nahoře: Spearmanova korelace *attention vah* s RBF kernelem jako funkce indexu vrstvy. Dole: absolutní hodnota korelace *attention vah* s *labels* Y_j . Barevné pásy značí ± 1 směrodatnou odchylku přes hlavy. Vzor je u všech modelů stejný: vrstva 0 má nejvyšší *labelovou* a nejnižší *kernelovou* korelaci, střední vrstvy se chovají *kernel-like* (vysoká *kernelová*, nízká *labelová* korelace) a v poslední vrstvě *kernelová* korelace opět klesá.



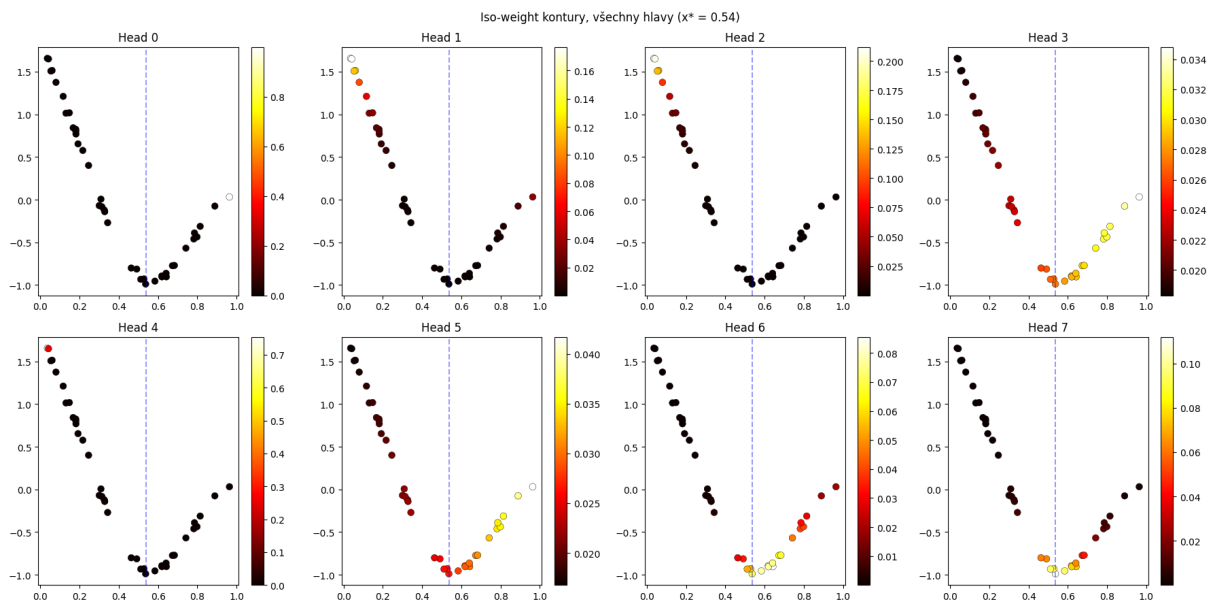
Obrázek 3.13: Osmivrstvý model, rozklad na jednotlivé hlavy. Vlevo: *kernelová korelace*, vpravo: *labelová korelace*, obojí pro každou z 8 hlav zvlášť napříč všemi 8 vrstvami. Ve vrstvě 0 se hlavy rozcházejí, některé silně čtou *labely* (vpravo nahoře), jiné ne, zatímco ve středních vrstvách se většina hlav sbíhá k vysoké *kernelové korelaci*. Heterogenita hlav uvnitř vrstvy je motivací pro test důležitosti hlav v sekci 3.3.2.

na body s nízkým X a vysokým Y (vlevo nahoře), *Head* 1, 3, 5 a 7 naopak na body s vysokým X a nízkým Y (vpravo dole) a několik hlav (0, 4, 6) degeneruje na jediný bod, může to být právě již zmíněný *attention sink*, který jsme popsali v sekci 3.1.1. Připomeňme, že naše kontextové body pocházejí z jedné konkrétní GP realizace, která na tomto intervalu klesá. Body vlevo tak mají vysoké hodnoty Y_j a body vpravo nízké. Když se tedy hlava dívá do levého horního rohu, vybírá si body s vysokým Y_j , a když do pravého dolního, vybírá si body s nízkým Y_j . Jednotlivé hlavy si vybírají body podle jejich hodnot Y_j . *Attention* tedy závisí i na *labelu*, nikoli jen na vzdálenosti ve vstupním prostoru, což je prostorová obdoba nenulové *labelové korelace* z tabulky 3.10.

Klíčový závěr korelační analýzy bychom mohli zformulovat takto: *nelze tvrdit, že PFN provádí čistý kernel smoothing*. Což je mimo jiné i jeden z výsledků předchozí analýzy v sekci 3.1.3, kde jsme ukázali, že *attention váhy* nekorelují s RBF kernelem a že se predikce PFN zásadně liší od Nadaraya-Watsonova estimátoru. Zde jsme ke stejnému závěru dospěli z opačné strany, tedy pohledem dovnitř jednotlivých vrstev. V žádné vrstvě není *labelová korelace* nulová, model v každém kroku míchá informaci o vstupu X i o hodnotě Y . Avšak střední vrstvy *kernel-like chování* výrazně preferují, ale nikdy ho nedosahují v čisté podobě. Toto pozorování je plně v souladu s Naglerovým teoretickým výsledkem, ke kterému se vrátíme v následující části.

3.3.2 Které hlavy model skutečně potřebuje: head-knockout

Korelační analýza z předchozí části má jedno zásadní omezení, *attention vzor* totiž pouze popisuje. Ignoruje *value vektory*, FFN i reziduální tok, a proto z ní nelze usuzovat na skutečnou funkční roli hlavy v celém modelu. Navíc Nagler ve své práci dokázal (Věta 6.3), že *softmax attention* vždy míchá X_j i Y_j . „Čistý kernel“ je tedy principiálně nedosažitelný a samotná korelace hlavy nám nemusí nutně říct, zda je pro predikci důležitá. Potřebujeme proto metriku, která místo popisu měří skutečný dopad hlavy na výstup. Takovou vlastnost budeme nazývat **kauzální důležitost**. Neptáme se, s čím *attention hlavy* souvisí, ale co se stane, když hlavu z modelu skutečně odebereme. Jde o princip tzv. *ablace*, tedy o cílený zásah do modelu a změření jeho následku. Teprve dopad tohoto zásahu na predikci prozradí, zda byla komponenta pro výpočet nutná, nebo s ním jen náhodně korelovala.



Obrázek 3.14: *Iso-weight rozklad* jednovrstvého modelu na 8 hlav. Pro pevný testovací bod $x^* = 0,59$ (modrá přerušovaná čára) je každý kontextový bod vyneseno do roviny (X_j, Y_j) a obarven *attention vahou* dané hlavy (světlejší = vyšší). *Kernelové porovnání* by koncentrovalo váhu kolem x^* nezávisle na Y_j . Místo toho různé hlavy mívají do různých rohů roviny (X, Y) , tedy na různé hodnoty Y_j , což je přímý vizuální doklad, že *attention* závisí i na *labelu*. Hlavy 0, 4, 6 navíc degenerují na jediný bod (*attention sink*).

Tuto myšlenku realizuje metoda **head-knockout** (doslova „vyřazení hlavy“). Připomeňme, že každá z 8 *attention* hlav vrstvy pracuje samostatně. Každá hlava si spočítá vlastní výstup, tedy svou vlastní váženou kombinaci kontextových bodů. Těchto 8 dílčích výstupů se pak musí složit dohromady do jedné společné reprezentace, se kterou dál pracuje zbytek sítě. Právě toto zajišťuje naučená výstupní projekce $\mathbf{W}_{\text{out}}^{(h)}$, tj. matice, přes kterou se přičítá příspěvek h -té hlavy do společné reprezentace. Podstatné je, že jiná cesta neexistuje. Cokoli hlava spočítala, dostane se ke zbytku modelu jedinečně vynásobením touto maticí a přičtením ke společné reprezentaci. Konkrétní hlavu proto můžeme odpojit velmi jednoduše, stačí její výstupní projekci vynulovat ($\mathbf{W}_{\text{out}}^{(h)} = \mathbf{0}$). Hlava pak sice dále počítá, ale její výsledek se nikam nepřičte. Ostatní hlavy, dopředné síť i reziduální spoje přitom zůstanou beze změn. Poté modelem znovu proženeme stejný vstup a měříme, o kolik se zhorší shoda predikce s GP posteriorem. Rozdíl oproti původnímu kompletnímu modelu je pak čistým příspěvkem právě té jedné hlavy.

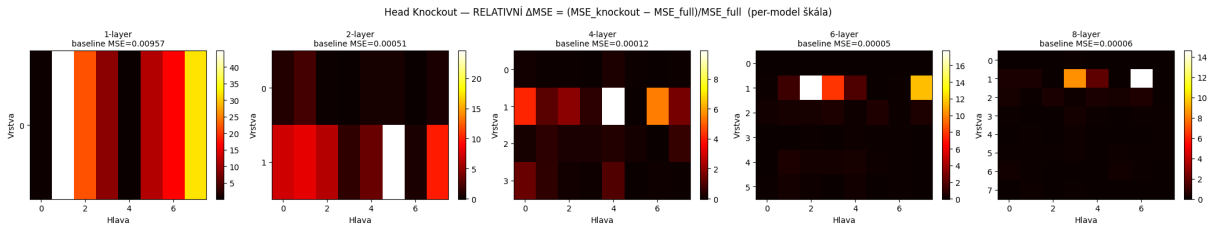
Klíčové je použít *relativní* metriku:

$$\text{rel} = \frac{\text{MSE}(\text{knockout}, \mu_{\text{GP}}) - \text{MSE}(\text{full}, \mu_{\text{GP}})}{\text{MSE}(\text{full}, \mu_{\text{GP}})}.$$

Baseline chybou zde rozumíme MSE kompletního modelu vůči GP posteriору, tj. jmenovatel tohoto zlomku. Používáme tuto metriku, jelikož absolutní rozdíl MSE by mohl být zavádějící. Hluboké modely totiž aproximují GP posterior s velmi malou *baseline chybou* (řádově 10^{-4}), takže jejich absolutní zhoršení není výrazné pouze kvůli této architektonické vlastnosti, ať je hlava důležitá, nebo ne. Relativní metrika naproti tomu měří zhoršení v násobcích vlastní *baseline chyby* modelu, a proto je mezi modely srovnatelná. Velká hodnota *rel* znamená, že hlava je důležitá pro aproximaci GP posteriору. Hodnota, která blízko nuly naopak značí, že model bez ní funguje prakticky stejně. Test provedeme na 50 instancích s kontextem $n_{\text{ctx}} = 40$. Výsledky shrnuje tabulka 3.11 a obrázek 3.15.

Model	Nejdůležitější (vrstva, hlava)	baseline MSE	max rel Δ MSE
1 vrstva	(0, 1)	0,00957	44,9×
2 vrstvy	(1, 5)	0,00051	24,6×
4 vrstvy	(1, 4)	0,00012	9,9×
6 vrstev	(1, 2)	0,00005	17,7×
8 vrstev	(1, 6)	0,00006	14,6×

Tabulka 3.11: Kauzální důležitost hlav podle *head-knockout*, měřená relativně k *baseline chybě* modelu. Uvedena je nejdůležitější hlava každého modelu, jeho *baseline MSE* vůči GP a maximální relativní zhoršení po vynulování této hlavy (např. 44,9× znamená 44,9násobek *baseline chyby*). Agregováno mediánem přes 50 instancí, $n_{\text{ctx}} = 40$.



Obrázek 3.15: *Head-knockout* pro všech pět modelů. Každá teplotní mapa zobrazuje relativní zhoršení $\text{rel} = (\text{MSE}_{\text{knockout}} - \text{MSE}_{\text{full}}) / \text{MSE}_{\text{full}}$ (světlejší = důležitější) po vynulování hlavy na dané pozici (vrstva $\times$ hlava). Každý model má vlastní barevnou škálu, jinak by hluboké modely s malou *baseline chybou* byly celé tmavé. Kauzálně důležité hlavy se koncentrují do raných vrstev (vrstva 0–1) napříč všemi modely. Jednovrstvý model je nejkřehčí (až 44,9× *baseline*), ale i hluboké modely nesou odebrání jediné hlavy 10 až 25× vlastní *baseline chyby*.

Obrázek ilustruje jasnou strukturu, kauzálně důležité hlavy se koncentrují v raných vrstvách. Je vidět, že dominantní hlava je u každého modelu ve vrstvě 0 nebo 1. To potvrzuje hypotézu o rozdělení práce z korelační analýzy. Pro hluboké modely platí, že *knockout* je nepohorší a tak v absolutních číslech zůstávají blízko GP posterioru. Avšak odebrání jedné hlavy zhorší predikci relativně k vlastní *baseline chybě*, a to napříč všemi hloubkami. Z toho tedy plyne, že žádná hlava není bezvýznamná. Jinými slovy, každá hlava má na výsledné predikci svůj podíl. Odebrání kterékoli z nich zvedne chybu modelu, jak je vidět v tabulce 3.11, ale model se bez ní nezhroutí a jeho predikce zůstává použitelná. Jednovrstvý model je nejkřehčí, což nás nepřekvapuje. Jelikož model má pouze jednu vrstvu, má pouze 8 hlav a každá je pro výpočty a predikce důležitá.

Poznamenejme, že z výsledků přes všech 50 instancí bereme *medián*, nikoli průměr. Na většině instancí je totiž chyba PFN vůči GP posterioru nepatrná (řádově zhruba 10^{-4}). Na několika málo instancích ale může být o řády vyšší. Průměr by tak byl zcela ovlivněn těmito *outliery* a mezi modely by kolísal o dva řády už při pouhé změně *seedu*. Medián tuto křehkost odstraňuje a dává stabilní a mezi modely srovnatelný obraz. Což mimochodem je známý jev ze statistiky, že medián je robustnější charakteristika než průměr (viz. např. proč se vždy uvádí median mezd místo průměru)

Podstatné je, že tento obraz nám korelační analýza ukázat nemohla. Hlava může s kernelem korelovat silně, a přesto být nahraditelná. Jiná může korelovat slabě, a přesto být pro predikci důležitá. *Head-knockout* tak korelační obraz nejen doplňuje, ale v jistém smyslu ho i koriguje. Tímto usuzujeme, že funkční roli hlavy určuje až její skutečný dopad na predikci, nikoli tvar jejího *attention vzoru*.

3.3.3 Rozklad chyby a role hloubky

Celou tuto sekci jsme zkoumali modely s různým počtem vrstev. Předchozí dvě části této sekce zkoumaly, *co* vrstvy dělají. Nyní bychom se mohli ptát, *kolik* jich vlastně potřebujeme, tedy jak hloubka ovlivňuje samotnou přesnost aproximace GP posterioru. K tomu použijeme rozklad střední kvadratické chyby na dvě složky, který přímo navazuje na *bias-variance* rozklad (2.5) z teoretické kapitoly. Pro každý model měříme MSE vůči GP posterioru při rostoucím počtu kontextových bodů $n \in \{5, 10, 20, 40, 80, 120\}$ (200 instancí pro každé n) a naměřené hodnoty proložíme modelem

$$\text{MSE}(n) = \text{bias}^2 + \frac{c}{n}.$$

Člen bias^2 zde představuje tzv. *ireducibilní chybu*, tj. tu část chyby, která nezmizí ani při nekonečném kontextu ($n \rightarrow \infty$). Připomeňme si, že toto je jedna z vlastností, kterou Nagler popisuje u transformeru, jak jsme již zmiňovali v sekci 2.4.4, *bias* transformeru totiž obecně nemůže úplně zmizet, zatímco jeho rozptyl s počtem dat klesá. Člen c/n je naopak rozptyl, který s přibývajícím daty klesá. Odhadnuté koeficienty jsou v tabulce 3.12, proložení pak na obrázku 3.16.

Grafy a tabulka nám ukazuje následující situaci. Za prvé, jednovrstvý model má jako jediný signifikantně nenulový $\text{bias}^2 = 0,00476$. To je přímé empirické potvrzení Naglerova teoretického výsledku ze sekce 2.4.4. Jedna vrstva nestačí k asymptotickému přiblížení GP posterioru, a proto v ní zůstává ireducibilní chyba, kterou nelze odstranit žádným množstvím kontextových dat. Od dvou vrstev výše je $\text{bias}^2 \approx 0$. Dvě vrstvy už tedy stačí k tomu, aby se model při dostatku dat přiblížil GP posterioru libovolně přesně.

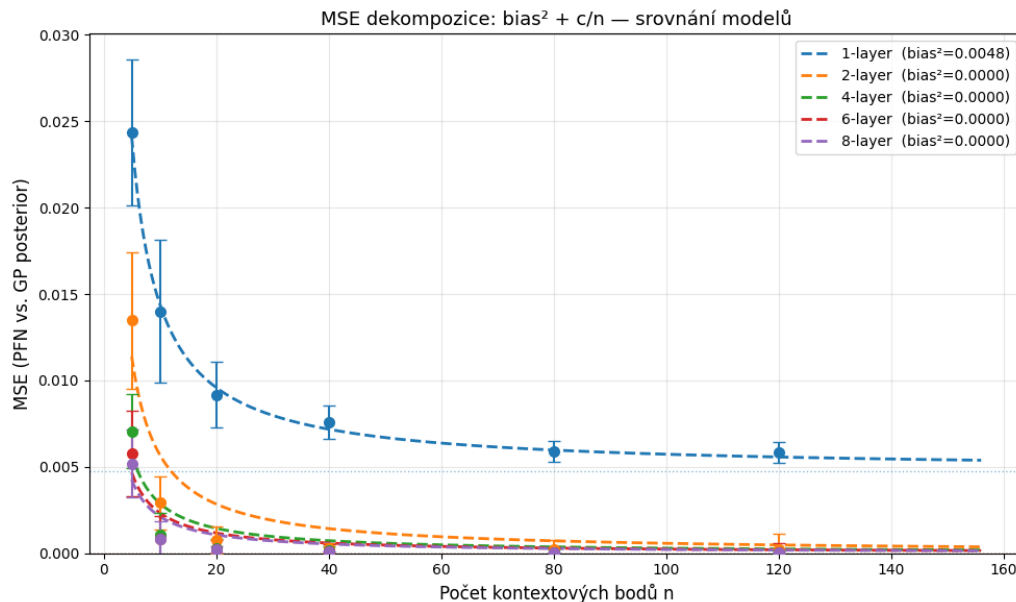
Za druhé, hloubka působí především na rozptylový člen c , který monotónně klesá:

$$c_{1L} = 0,097 > c_{2L} = 0,068 > c_{4L} = 0,035 > c_{6L} = 0,029 > c_{8L} = 0,026.$$

Přidání vrstev tedy zrychluje konvergenci, hlubší model dosáhne stejné přesnosti z menšího kontextu. Zlepšení se však postupně sytí, mezi šesti a osmi vrstvami se c už prakticky nemění. Z praktického hlediska to znamená, že přibližně šest vrstev představuje bod, za kterým další prohlubování sítě nepřináší téměř žádný zisk v přesnosti.

Model	bias^2	c (rozptyl)
1 vrstva	0,00476	0,097
2 vrstvy	-0,00152	0,068
4 vrstvy	-0,00089	0,035
6 vrstev	-0,00076	0,029
8 vrstev	-0,00065	0,026

Tabulka 3.12: Rozklad MSE vůči GP posterioru na $\text{bias}^2 + c/n$. Jednovrstvý model má jako jediný signifikantně nenulový bias^2 . Od dvou vrstev výše je ireducibilní chyba statisticky neodlišitelná od nuly a hloubka už jen snižuje rozptylový člen c . Poznamenejme dále, že mírně záporné odhady bias^2 u hlubších modelů nemají věcný význam. Proložení totiž nevynucuje, aby bias^2 vyšel nezáporný, a když je skutečná hodnota blízká nule, může odhad vlivem šumu klesnout těsně pod nulu. Tyto hodnoty tedy prakticky znamenají nulový ireducibilní bias.



Obrázek 3.16: Rozklad $\text{MSE} = \text{bias}^2 + c/n$ pro všech pět modelů. Body jsou naměřené MSE (chybové úsečky $\pm 1 \text{ std}$), přerušované křivky proložení. Jednovrstvý model (modrá) se s rostoucím n vyrovnává na nenulové asymptotě $\text{bias}^2 = 0,0048$ (tečkovaná čára), zatímco hlubší modely klesají prakticky k nule. Rozdíly mezi 4, 6 a 8 vrstvami jsou už zanedbatelné.

3.3.4 Shrnutí: co se která vrstva naučila

Systematické měnění hloubky odhalilo, že PFN organizuje svůj výpočet takovým způsobem, že každá komponenta se zaměří na něco jiného, tj. jistým způsobem si komponenty rozdělují práci během výpočtů.

Korelační analýza ukázala specializaci vrstev. Rané vrstvy čtou *labels* (nejvyšší *labelová* a nejnižší *kernelová korelace*), střední vrstvy se chovají *kernel-like* a poslední vrstva opouští *kernelové porovnávání* ve prospěch sestavení predikce. Současně však platí, že žádná vrstva nedělá čistý *kernel smoothing*. V každém kroku se míchá informace o X i o Y , přesně jak předpovídá Naglerova věta o *softmax attention*.

Head-knockout tento popisný obraz doplnil z jiné perspektivy. Důležité hlavy se koncentrují do raných vrstev a jednovrstvý model je nejkřehčí. Avšak i v hlubokých modelech odebrání jediné hlavy znatelně zvedne chybu modelu. Žádná hlava tedy není bezvýznamná.

Rozklad chyby nakonec vysvětlil, proč modelu jedna vrstva nestačí. Jednovrstvý model má jako jediný nenulový $\text{bias}^2 = 0,0048$, zatímco od dvou vrstev a výše je ireducibilní chyba nulová a hloubka už jen snižuje rozptylový člen c .

Dohromady nám tato pozorování dávají jasný obraz o tom, jak PFN uvnitř funguje. Model si rozděluje práci mezi vrstvy: rané vrstvy sbírají *labels*, střední vrstvy provádějí *kernelové porovnávání* a poslední vrstva sestavuje konečnou predikci. Žádná vrstva přitom nedělá prostý *kernel smoothing*. Čím je model hlubší, tím je toto rozdělení práce zřetelnější.

3.4 Hloubka jako iterace řešiče: PFN versus gradientní sestup

Předchozí sekce ukázala, co jednotlivé vrstvy dělají. Rané vrstvy čtou *labels*, střední vrstvy porovnávají vstupní vzdálenosti a hloubka odstraňuje ireducibilní bias jednovrstvého modelu. Zůstává však hlubší otázka, proč vlastně přidávání vrstev pomáhá a jaký výpočet odpovídá jedné vrstvě. Připomeňme, že GP posterior mean není nic jiného než řešení lineární soustavy

$$\mu(x^*) = k(x^*, \mathbf{X}) \boldsymbol{\alpha}, \quad \boldsymbol{\alpha} = (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y},$$

tedy inverze matice $\mathbf{K} + \sigma^2 \mathbf{I}$. Tato inverze se dá řešit iterativně. Přirozeně se proto nabízí hypotéza, že hloubka transformeru odpovídá počtu iterací takového řešiče. V této sekci tuto hypotézu kvantitativně otestujeme.

Nejjednodušším iterativním řešičem je gradientní sestup (dále jen GD) na kvadratické ztrátě $\frac{1}{2} \boldsymbol{\alpha}^\top (\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$, jehož iterace zní

$$\boldsymbol{\alpha}^{(0)} = \mathbf{0}, \quad \boldsymbol{\alpha}^{(t+1)} = \boldsymbol{\alpha}^{(t)} + \eta (\mathbf{y} - (\mathbf{K} + \sigma^2 \mathbf{I}) \boldsymbol{\alpha}^{(t)}).$$

Tyto iterace mají i druhý výklad. Inverzi $(\mathbf{K} + \sigma^2 \mathbf{I})^{-1}$ lze rozvinout do geometrické řady, které se v tomto kontextu říká **Neumannova řada**, a částečné součty této řady odpovídají přesně jednotlivým krokům GD. Jeden krok GD tedy znamená jeden přičtený člen řady, a proto budeme oba pohledy volně zaměňovat. Rychlost konvergence řídí **podmíněnost** soustavy

$$\kappa = \frac{\lambda_{\max}(\mathbf{K}) + \sigma^2}{\lambda_{\min}(\mathbf{K}) + \sigma^2},$$

kde $\lambda_{\max}, \lambda_{\min}$ jsou krajní vlastní čísla kernelové matice $\mathbf{K}$. Chyba GD klesá v nejhorším případě jako ρ^t , kde $\rho = (\kappa - 1)/(\kappa + 1)$ a t je počet kroků. Obyčejný GD tak potřebuje k dané přesnosti řádově $O(\kappa)$ kroků, tj. čím hůře je soustava podmíněná, tím víc iterací. Odtud plyne testovatelná predikce. *Pokud každá vrstva PFN odpovídá jednomu kroku iterativního řešiče, měly by hůře podmíněné úlohy (větší κ) vyžadovat více vrstev.*

K ověření této predikce potřebujeme kontrolovaně měnit κ , aniž bychom měnili cokoli jiného. Toho dosáhneme přes šum σ^2 . Numericky totiž platí $\lambda_{\min}(\mathbf{K}) \approx 0$, takže $\kappa \approx 1 + \lambda_{\max}(\mathbf{K})/\sigma^2$ a samotný šum nastavuje podmíněnost prakticky přímo. Pracujeme proto se dvěma sadami pěti modelů (1, 2, 4, 6 a 8 vrstev), které se liší výhradně šumem, na kterém byly trénovány i testovány: režim **Easy** ($\ell = 0,3, \sigma^2 = 0,5$) a režim **Hard** ($\ell = 0,3, \sigma^2 = 0,1$). Na rozdíl od předchozích sekcí, kde jsme používali jediný model trénovaný na náhodných hyperparametrech, je zde každá sada trénována přesně na svých testovacích hyperparametrech. Měříme tedy výkon na trénovacím rozdělení, nikoli generalizaci. Všechny pět hloubek v každé sadě bylo trénováno stejných 300 epoch, takže výsledky nejsou zkreslené nevyrovnaným tréninkem. Výslednou podmíněnost obou režimů shrnuje tabulka 3.13.

Režim	ℓ	σ^2	$\kappa (n=40)$	$\rho = \frac{\kappa-1}{\kappa+1}$
Easy	0,3	0,5	≈ 50	$\approx 0,96$
Hard	0,3	0,1	≈ 245	$\approx 0,99$

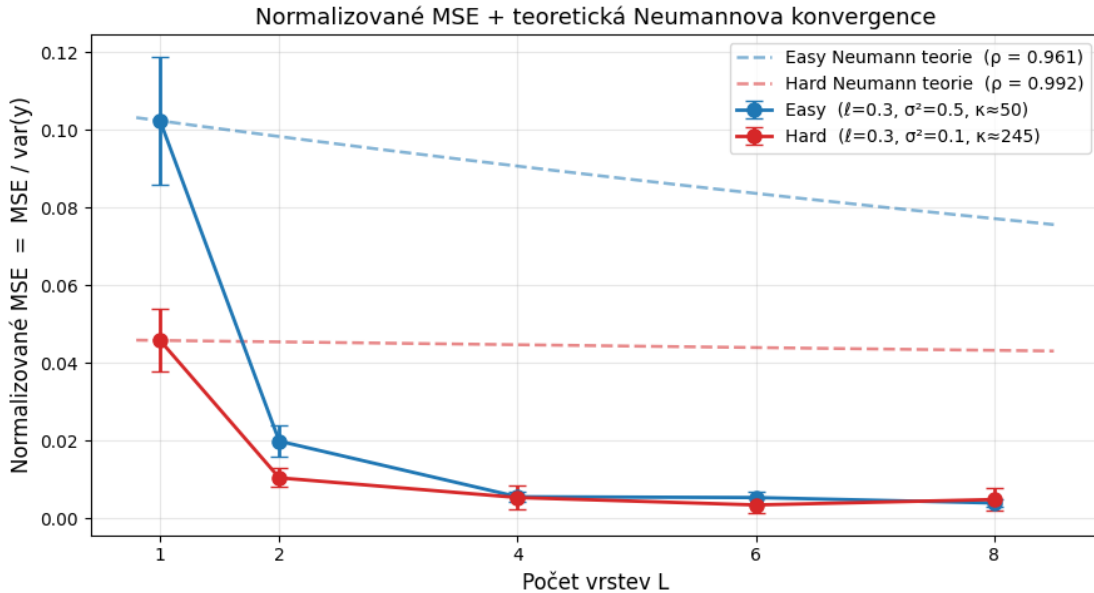
Tabulka 3.13: Podmíněnost obou režimů, průměr přes 50 instancí při $n = 40$. Snížení šumu z 0,5 na 0,1 zhoršuje podmíněnost přibližně pětinašobně. Veličina ρ udává, o kolik se v nejhorším případě zmenší chyba gradientního sestupu za jeden krok.

3.4.1 Hloubka snižuje chybu, ale ne jako obyčejná Neumannova řada

Nejprve ověříme základní předpoklad, tedy že hloubka vůbec zlepšuje aproximaci. Empirický pokles chyby zároveň porovnáme s teoretickou křivkou ρ^L obyčejného iterativního řešiče. Pro každý model a každou hloubku měříme na 200 sdílených instancích ($n_{\text{ctx}} = 40$, $n_{\text{test}} = 10$) tzv. **normalizované MSE**

$$\text{nMSE} = \frac{\text{MSE}(\hat{\mu}_{\text{PFN}}, \mu_{\text{GP}})}{\text{var}(\mu_{\text{GP}})},$$

tedy chybu predikce PFN vůči přesnému GP posterioru, dělenou rozptylem cíle (GP posterior mean přes testovací body). Za normalizační jmenovatel bereme záměrně $\text{var}(\mu_{\text{GP}})$, a nikoli $\text{var}(y)$. Pozorované hodnoty y totiž obsahují šum, který posterior mean neobsahuje, a normalizace šumem by uměle obracela pořadí obou režimů. Empirické hodnoty spolu s teoretickou křivkou ρ^L zachycuje obrázek 3.17. Plná čára v něm ukazuje naměřenou chybu PFN jako funkci hloubky L , přerušovaná čára pak hranici ρ^L , tedy jak by chyba klesala, kdyby každá vrstva provedla přesně jeden krok obyčejného řešiče.



Obrázek 3.17: Normalizované MSE jako funkce hloubky L pro oba režimy (plné čáry, chybové úsečky ± 1 směrodatná chyba) proti teoretické *worst-case* Neumannově konvergenci ρ^L (přerušované čáry). Chyba prudce klesá s hloubkou, u *Easy* z 0,10 na 0,004, u *Hard* z 0,046 na 0,005, a hlavní zisk se odehrává mezi jednou a čtyřmi vrstvami. Empirický pokles je ovšem mnohem rychlejší než teoretická křivka, už 2–4 vrstvy dosáhnou přesnosti, kterou by Neumannova řada nedala ani po desítkách kroků.

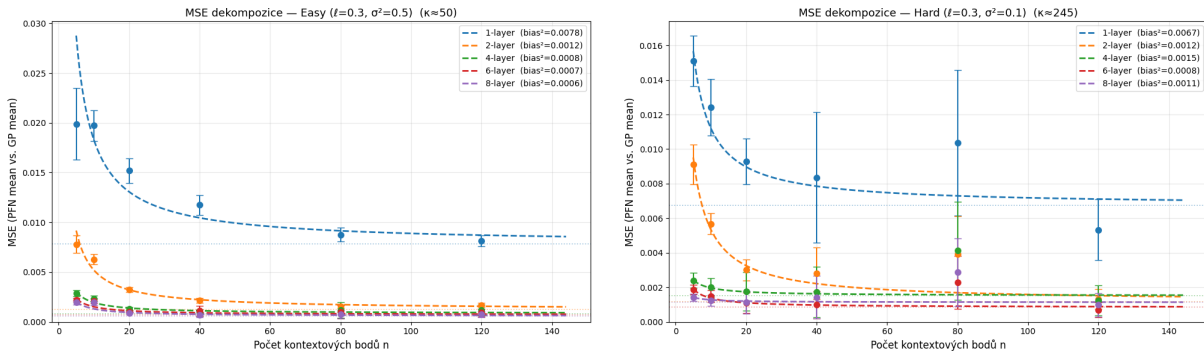
Z obrázku plynou dvě pozorování. Za prvé, hloubka skutečně a výrazně zlepšuje aproximaci. Normalizované MSE u obou režimů prudce klesá, přičemž naprostá většina zisku připadá na přechod z jedné na čtyři vrstvy, pak se pokles sytí. Za druhé, empirický pokles je mnohem rychlejší než teoretická křivka ρ^L . Zatímco *worst-case* teorie předpovídá pro obě hodnoty ρ z tabulky 3.13 jen velmi pozvolný pokles, PFN se přiblíží GP posterioru už za dvě až čtyři vrstvy. Naivní srovnání „vrstva = jeden krok Neumannovy řady“ tedy neplatí. Buď dělá jedna vrstva mnohem víc než jeden krok, anebo řešič, který PFN implementuje, není obyčejný gradientní sestup. Abychom mezi těmito možnostmi rozlišili, potřebujeme férovější srovnání, které zavedeme v sekci 3.4.3.

3.4.2 Rozklad chyby: hloubka odbourává bias i rozptyl

Než přejdeme ke gradientnímu sestupu, rozložíme chybu stejně jako v sekci 3.3.3. Uvidíme tak, kterou složku chyby hloubka snižuje. Pro každý model měříme MSE vůči GP posterioru při rostoucím kontextu $n \in \{5, 10, 20, 40, 80, 120\}$ a naměřené hodnoty proložíme modelem $\text{MSE}(n) = \text{bias}^2 + c/n$. Připomeňme, že bias^2 je ireducibilní chyba, která nezmizí ani při nekonečném kontextu, a člen c/n je rozptyl klesající s daty. Odhadnuté koeficienty pro oba režimy shrnuje tabulka 3.14, proložení je na obrázku 3.18.

Model	Easy ($\sigma^2 = 0,5$)		Hard ($\sigma^2 = 0,1$)	
	bias^2	c	bias^2	c
1 vrstva	0,00784	0,105	0,00674	0,045
2 vrstvy	0,00124	0,040	0,00117	0,042
4 vrstvy	0,00085	0,011	0,00152	0,005
6 vrstev	0,00071	0,008	0,00084	0,005
8 vrstev	0,00059	0,007	0,00113	0,001

Tabulka 3.14: Rozklad $\text{MSE} = \text{bias}^2 + c/n$ vůči GP posterioru pro oba režimy. V obou režimech klesají s hloubkou obě složky, ireducibilní bias jednovrstvého modelu zhruba o řád, rozptylový člen c ještě výrazněji. Zlepšení se sytí kolem čtyř vrstev.



Obrázek 3.18: Rozklad $\text{MSE} = \text{bias}^2 + c/n$ pro režim *Easy* (vlevo) a *Hard* (vpravo). Body jsou naměřené MSE (chybové úsečky $\pm 1 \text{ std}$), přerušované křivky proložení. S rostoucí hloubkou se křivky posouvají dolů. Jednovrstvý model má nejvyšší asymptotu i nejpomalejší pokles, hluboké modely klesají prakticky k nule.

Obraz je konzistentní s náhodným modelem ze sekce 3.3.3. Největší skok nastává mezi jednou a dvěma vrstvami, jak je vidět v tabulce 3.14, pak se zlepšování sytí. Hloubka tedy neodstraňuje jen ireducibilní bias, ale zrychluje i konvergenci vzhledem k množství dat. Kolika krokům obyčejného řešiče to odpovídá, z tohoto rozkladu ale nevyčteme. K tomu potřebujeme řešič skutečně spustit.

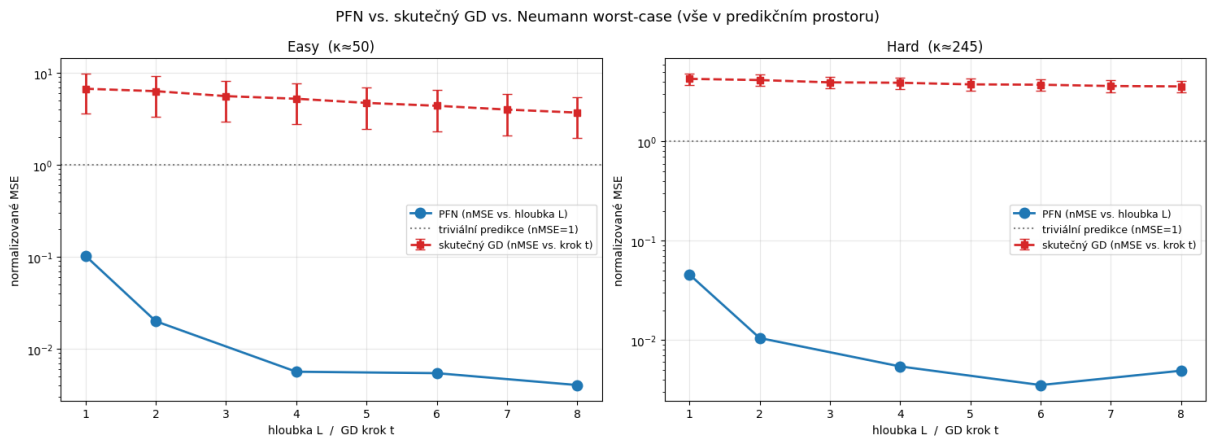
3.4.3 Férové srovnání: skutečný gradientní sestup

Zdánlivý rozpor ze sekce 3.4.1, tedy že PFN konverguje rychleji, než Neumannova řada dovoluje, má jednoduchou příčinu. Srovnávali jsme dvě různé veličiny. Míra $\rho = (\kappa - 1)/(\kappa + 1)$ popisuje, jak rychle konverguje samotný vektor koeficientů α , tedy chybu v tzv. α -prostoru. My ale měříme chybu v predikčním prostoru, tj. pro $\mu(x^*) = k(x^*, \mathbf{X}) \alpha$. Tyto dva prostory nejsou totéž. Predikce je kernelově

vážená kombinace koeficientů α a toto vážení tlumí právě ty složky, ve kterých gradientní sestup konverguje nejpomaleji. Srovnávat empirickou predikční chybu s křivkou ρ^L je proto nefér, penalizuje totiž řešič za chyby, které se do predikce skoro nepromítnou.

Férové srovnání vyžaduje spustit skutečný gradientní sestup na stejných datech a měřit jeho chybu ve stejném predikčním prostoru jako u PFN. Pro každou instanci proto iterujeme výše uvedenou rovnici s optimálním fixním krokem $\eta = 2/(\lambda_{\max} + \lambda_{\min})$. Po každém kroku t zrekonstruujeme predikci $\mu_{\text{GD}}^{(t)}(x^*) = k(x^*, \mathbf{X}) \alpha^{(t)}$ a spočítáme její normalizované MSE vůči GP posterioru, tedy přesně tu metriku, kterou u PFN vynášíme proti hloubce. Výsledek srovnání je na obrázku 3.19.

I v tomto případě usuzujeme, že je dobré pro čtenáře popsat, co obrázek znázorňuje a jak ho číst. Modrá křivka ukazuje chybu PFN jako funkci počtu vrstev L . Červená křivka ukazuje chybu gradientního sestupu jako funkci počtu kroků t , vnesenou na stejné ose. Porovnáváme tedy jednu vrstvu PFN s jedním krokem GD přímo proti sobě. Tečkovaná vodorovná čára značí triviální predikci konstantním průměrem, které odpovídá $\text{nMSE} = 1$. Cokoli nad touto čarou je horší, než kdybychom vůbec nepredikovali.



Obrázek 3.19: PFN versus skutečný gradientní sestup, obojí v predikčním prostoru, pro *Easy* (vlevo) a *Hard* (vpravo). Modrá: normalizované MSE PFN proti hloubce L . Červená: normalizované MSE gradientního sestupu proti počtu kroků t (stejně osy). Tečkovaná čára značí triviální predikci konstantním průměrem ($\text{nMSE} = 1$). PFN je hluboko pod triviální úrovní už při jedné vrstvě, zatímco skutečný gradientní sestup zůstává nad triviální predikcí i po osmi krocích.

Obrázek dává jednoznačnou odpověď, reálný gradientní sestup je pomalý i v predikčním prostoru. Jeho normalizované MSE zůstává po prvních osm krocích nad triviální hranicí, tj. hůř než predikce konstantním průměrem. PFN je naproti tomu pod touto hranicí už s jedinou vrstvou. Rozpor z předchozí části se tím vysvětluje a zároveň obrací. Nefér zvýhodněný nebyl řešič, naopak PFN skutečný gradientní sestup výrazně předčí. Abychom rozdíl vyčíslili, dopočítáme pro každou hloubku L , kolik kroků gradientního sestupu dá stejnou chybu jako L -vrstvý PFN. Výsledky jsou v tabulce 3.15.

PFN hloubka L	kroky GD, Easy ($\kappa \approx 50$)	kroky GD, Hard ($\kappa \approx 245$)
1	51	271
4	87	402
8	91	408

Tabulka 3.15: Efektivní počet kroků gradientního sestupu, který dosáhne stejné predikční chyby jako L -vrstvý PFN. Už jediná vrstva odpovídá desítkám kroků. Počet potřebných kroků navíc silně roste s podmíněností κ .

Tabulka ukazuje dvě věci najednou. Za prvé, jedna vrstva odpovídá řádově desítkám kroků obyčejného gradientního sestupu. Naivní hypotéza „vrstva = jeden krok“ je tedy vyvrácena o celé řády. Za druhé, počet potřebných kroků prudce roste s podmíněností, přesně jak předpovídá $O(\kappa)$ chování gradientního sestupu. Naproti tomu PFN dosahuje srovnatelné přesnosti stejnými osmi vrstvami v obou režimech. Tuto asymetrii, kdy GD škáluje s κ a PFN nikoli, prozkoumáme v následující části na třech režimech.

3.4.4 Necitlivost PFN na podmíněnost

Rozhodujícím rozdílem mezi PFN a gradientním sestupem je tedy jejich závislost na podmíněnosti. Abychom ji vykreslili co nejostřeji, přidáme třetí, silně špatně podmíněný režim $\sigma^2 = 0,01$ (vlastní sada modelů trénovaná stejným způsobem), který posouvá κ zhruba na 2400. Pro každý ze tří režimů zafixujeme cílovou přesnost na úrovni, kterou dosáhne osmivrstvý PFN, a změříme, kolik kroků gradientního sestupu je k ní potřeba. Výsledek shrnuje tabulka 3.16 a obrázek 3.20.

Režim	κ	PFN nMSE ($L=8$)	kroky GD ($L=8$)
$\sigma^2 = 0,5$	50	0,0040	91
$\sigma^2 = 0,1$	245	0,0049	408
$\sigma^2 = 0,01$	2424	0,00066	5723

Tabulka 3.16: Škálování s podmíněností přes tři režimy. Počet kroků gradientního sestupu roste zhruba lineárně s κ ($91 \rightarrow 408 \rightarrow 5723$, tedy $\approx 2\kappa$), zatímco osmivrstvý PFN dosahuje srovnatelné přesnosti stále stejnými osmi vrstvami. V nejhůře podmíněném režimu je PFN dokonce nejpřesnější.

Výsledek je jednoznačný. Počet kroků gradientního sestupu roste zhruba lineárně s κ , přesně jak předpovídá $O(\kappa)$ konvergence obyčejného řešiče. PFN je naproti tomu vůči podmíněnosti prakticky necitlivý. Stejných osm vrstev stačí ve všech třech režimech a v nejhůře podmíněném z nich je model dokonce nejpřesnější. Tuto vlastnost budeme dále nazývat **κ -necitlivost**. Poznamenejme, že κ -necitlivost je vidět při srovnání mezi režimy. Uvnitř jednoho režimu žádný vztah mezi chybou a podmíněností nenajdeme (korelace $|r| \lesssim 0,1$), protože při fixních hyperparametrech se κ mezi instancemi mění jen málo.

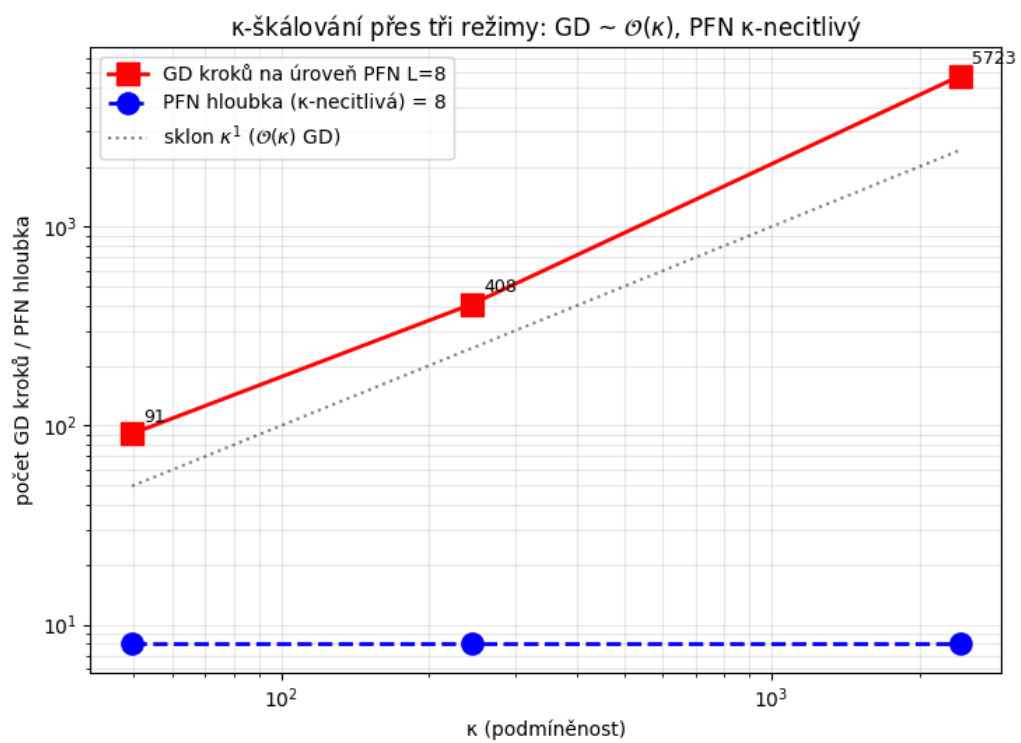
Co nám tato vlastnost říká o řešiči, který se PFN naučil? V numerické matematice existuje standardní postup, jak konvergenci zbavit závislosti na κ , tzv. **preconditioning**. Soustava se před řešením vynásobí vhodnou maticí, která srovná její spektrum, a tím sníží efektivní podmíněnost. Kombinace κ -necitlivosti a rychlosti mnohonásobně převyšující gradientní sestup je proto charakteristickým znakem naučeného *preconditioned* nebo druhořadového (Newtonovského) řešiče, nikoli obyčejného gradientního sestupu.

Toto pozorování má oporu v teorii *in-context* učení. Ahn et al. (2023) dokázali, že globální optimum jednovrstvého lineárního transformeru implementuje jeden krok *preconditioned* gradientního sestupu, jehož *preconditioner* se adaptuje na rozdělení vstupu. Von Oswald et al. (2023) ukázali totéž pro L vrstev jako L iterací. Fu et al. (2023) pak doložili, že transformery dosahují druhořadové konvergence a fungují i na špatně podmíněných datech, kde gradientní sestup selhává. Naše měření je přímým kvantitativním potvrzením tohoto obrazu na úloze GP regrese.

3.4.5 Shrnutí: hloubka jako preconditioned iterace

Tato sekce testovala hypotézu, že hloubka PFN odpovídá počtu iterací řešiče, který invertuje matici $\mathbf{K} + \sigma^2 \mathbf{I}$.

Hloubka skutečně a výrazně zlepšuje aproximaci GP posterioru. Rozklad chyby navíc ukázal, že s hloubkou mizí jak ireducibilní bias, tak rozptylový člen, a to s nasycením kolem čtyř vrstev. Naivní



Obrázek 3.20: Počet kroků gradientního sestupu na úroveň osmivrstvého PFN (červená) jako funkce podmíněnosti κ , v log-log měřítku, přes tři režimy. Tečkovaná čára značí referenční sklon $\mathcal{O}(\kappa)$. Gradientní sestup škáluje lineárně s κ , zatímco hloubka PFN (modrá) zůstává konstantních 8 vrstev napříč celým rozsahem podmíněnosti.

verze hypotézy „vrstva = jeden krok Neumannovy řady“ však neplatí. Férové srovnání se skutečným gradientním sestupem ukázalo, že už jediná vrstva odpovídá desítkám jeho kroků a s horší podmíněností tento počet dále roste, jak je vidět v tabulce 3.15.

Rozhodujícím pozorováním je κ -necitlivost. Napříč třemi řády podmíněnosti potřebuje gradientní sestup stále více kroků, zatímco PFN si vystačí se stejnými osmi vrstvami. Kombinace této necitlivosti a vysoké rychlosti je znakem naučeného *preconditioned* či druhořadového řešiče, přesně jak pro transformery předpovídá teorie *in-context* učení. Dá se tedy říct, že PFN implementuje κ -necitlivou iteraci, ve které jedna vrstva vykoná řádově tolik práce jako desítky kroků obyčejného gradientního sestupu.

Závěr

Text závěru....

Literatura

- [1] S. Allen, J. W. Cahn: *A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening*. Acta Metall., 27:1084-1095, 1979.
- [2] G. Ballabio et al.: *High Performance Systems User Guide*. High Performance Systems Department, CINECA, Bologna, 2005. www.cineca.it
- [3] J. Becker, T. Preusser, M. Rumpf: *PDE methods in flow simulation post processing*. Computing and Visualization in Science, 3(3):159-167, 2000.